

MATERIAL AUTORAL

Model Context Protocol (MCP)

Disciplina 3 • Engenharia de Software com IA Aplicada

Autoria: Erick Wendel

Como usar este material

Este material foi reconstruído a partir das transcrições organizadas da disciplina Model Context Protocol (MCP). O conteúdo-base permanece fiel ao arquivo fornecido; a nova edição acrescenta organização didática, navegação interna, integração com o GitHub, diagramas conceituais e atividades de revisão.

PRINCÍPIO DE FIDELIDADE

- O conteúdo conceitual das aulas foi preservado; a edição atua em organização, hierarquia visual e correções de apresentação.
- Diagramas e quadros apenas reorganizam relações explicitamente descritas nas aulas.
- Gráficos numéricos não são introduzidos sem uma série quantitativa comparável explicitamente fornecida na transcrição.
- Os links do GitHub usam o repositório oficial indicado para a disciplina e os diretórios efetivamente listados nele.

REPOSITÓRIO OFICIAL DA DISCIPLINA

[Model Context Protocol \(MCP\) no GitHub](#)

Use o repositório junto com o texto: localize o projeto associado à unidade, reproduza o comportamento descrito e compare a implementação com a decisão de engenharia apresentada na aula.



COMO ESTUDAR CADA AULA

- Leia os objetivos e observe o fluxo visual da unidade antes de avançar.
- Estude o conteúdo integral, agora organizado por aula e por subtítulos da própria transcrição.
- Abra o diretório do GitHub indicado na unidade e localize os artefatos citados no conteúdo.
- Responda ao checkpoint sem consultar o texto; volte somente aos pontos em que ainda houver dúvida.
- Use o mapa da disciplina para navegar diretamente para qualquer unidade ou aula.

Introdução da disciplina

Ao longo dos últimos anos, a forma como construímos software vem passando por uma transformação profunda. A chegada de modelos de linguagem e sistemas baseados em inteligência artificial mudou não apenas a interface com o usuário, mas também a forma como estruturamos nossas aplicações, expomos capacidades e integramos sistemas.

Dentro desse novo cenário, surge o Model Context Protocol (MCP) como uma peça fundamental. Mais do que um padrão técnico, o MCP representa uma nova forma de pensar a integração entre inteligência artificial e software existente. Ele não substitui APIs, não elimina sistemas legados e nem redefine completamente arquiteturas consolidadas. O que ele faz é criar uma camada de adaptação, permitindo que modelos de linguagem interajam com sistemas reais de forma estruturada, segura e compreensível.

Neste material, eu parto de um problema muito comum no mercado: como conectar IA a sistemas legados sem precisar reescrever tudo. Em vez de propor uma solução teórica, eu construo passo a passo um servidor MCP real, evoluindo desde conceitos básicos até um cenário completo de produção.

Ao longo dos capítulos, eu mostro como sair de uma API tradicional e transformá-la em uma interface adequada para consumo por agentes inteligentes. Esse processo envolve não apenas expor funcionalidades, mas também pensar em:

- abstração de domínio, em vez de simples espelhamento de endpoints
- organização de código e arquitetura sustentável
- definição de tools, resources e prompts
- validação de contratos com schemas
- testes automatizados cobrindo integração real
- autenticação e autorização com JWT e RBAC
- uso de Service Tokens para integração programática
- controle de uso com rate limiting

- publicação e distribuição via NPM
- integração com agentes usando LangChain.js

Mais importante do que cada uma dessas etapas isoladamente é entender como elas se conectam. O objetivo não é apenas aprender MCP, mas compreender como construir sistemas que possam ser consumidos por inteligência artificial de forma segura, escalável e reutilizável.

Ao final do material, o MCP deixa de ser apenas um conceito e passa a ser uma peça viva dentro da arquitetura. Ele se torna uma ponte entre o mundo da IA e os sistemas que já existem dentro das empresas, permitindo que agentes tomem decisões, executem ações e orquestrem fluxos reais de negócio.

Este ebook foi pensado para quem já tem experiência com desenvolvimento, especialmente no ecossistema Java e arquitetura de software, e quer entender como aplicar esses novos conceitos de forma prática. A proposta é direta: menos teoria abstrata e mais construção orientada a problemas reais.

Se existe uma mensagem central neste material, é a seguinte: você não precisa abandonar o que já foi construído para adotar IA. Você precisa conectar melhor o que já existe. E o MCP é uma das formas mais eficientes de fazer isso.

Mapa da disciplina

Navegue por uma unidade ou aula. Os títulos abaixo são links internos; cada unidade também apresenta acesso direto ao GitHub associado ao conteúdo.

Unidade 1: Visão Geral do Model Context Protocol (MCP) [↗ ir para a unidade](#)

[Aula 1: Diferença entre MCPs e o modelo clássico de “tools/plugins”](#)

Unidade 2: MCP para APIs, Bancos de Dados e Serviços Internos [↗ ir para a unidade](#)

[Aula 1: Conhecendo o Projeto: Template Inicial e Arquitetura](#)

[Aula 2: Parsing Inteligente: Organizando a Entrada do Cliente](#)

[Aula 3: Orquestração Autônoma com LangChain.js e MCP no MongoDB](#)

[Aula 4: Construindo Uma Tool Customizada no LangChain](#)

[Aula 5: Orquestração Autônoma com LangChain.js e MCP para Gerenciamento de Sistema de Arquivos](#)

[Aula 6: Usando Services Como Tools - Google Trends API com LangChain.js](#)

Unidade 3: MCPs, Agents, Instructions e Skills no Desenvolvimento [↗ ir para a unidade](#)

[Aula 1: Entendendo Agents e Instructions](#)

[Aula 2: Entendendo skills](#)

Unidade 4: CypherSuite MCP: Desenvolvendo Servidores MCP do Zero [↗ ir para a unidade](#)

[Aula 1: Criando um MCP do Zero: Testes Automatizados Via MCP Client, Definindo Tools e Inspeccionando MCP Servers](#)

[Aula 2: Definindo Resources e Prompts em Servidores MCP + Usando Nosso Servidor MCP no VSCode](#)

Unidade 5: Transformando Sistemas Legados em Servidores MCP [↗ ir para a unidade](#)

[Aula 1: Template Inicial e Arquitetura + Boas Práticas de Organização de Código e Estrutura de Projeto](#)

[Aula 2: Como Empresas Usam MCPs para Conectar IA a Sistemas Legados](#)

[Aula 3: Criando Tools de Listagem e Criação de Clientes](#)

[Aula 4: Criando Tools de Update e Delete de Clientes + Usando no VSCode](#)

Unidade 6: Segurança e Governança em MCPs [↗ ir para a unidade](#)

[Aula 1: Template Inicial e Arquitetura + Boas Práticas de Organização de Código e Estrutura de Projeto](#)

[Aula 2: Implementando Autenticação com RBAC e Autenticação com JWT em Web API](#)

[Aula 3: Autenticação e Autorização com Service Tokens em Web APIs](#)

[Aula 4: Controlando Acesso por Tokens com Rate Limiting](#)

[Aula 5: Usando Service Tokens em Nosso Servidor MCP + Rate Limiting](#)

Unidade 7: MCP em Produção: Publicação e Transports [↗ ir para a unidade](#)

[Aula 1: Publicando Servidores MCP em NPM Registry \(público\) e Verdaccio \(privado\)](#)

[Aula 2: Indo Além em Servidores MCP: Diferentes Transports e Ideias para seu Próximo Servidor](#)

Unidade 8: Consumindo o MCP com LangChain.js [↗ ir para a unidade](#)

[Aula 1: Criando um Agente para Gerenciar Operações de Clientes com Nosso Customers MCP Server e LangChain.js](#)

Mapa do GitHub

O repositório oficial lista os diretórios abaixo dentro de modulo03-mcp-na-pratica. A tabela mantém os nomes exatos do GitHub e os relaciona às unidades didáticas desta edição.

Unidade	Conteúdo	GitHub
1	Visão Geral do Model Context Protocol (MCP)	Repositório do módulo
2	MCP para APIs, Bancos de Dados e Serviços Internos	01-multiple-mcp-tools-template 01-multiple-mcp-tools-z 02-google-trends-agent
3	MCPs, Agents, Instructions e Skills no Desenvolvimento	03-dev-instructions-agents/.github/agents 04-skills
4	CypherSuite MCP: Desenvolvendo Servidores MCP do Zero	05-mcps-do-zero-template 05-mcps-do-zero-z
5	Transformando Sistemas Legados em Servidores MCP	06-your-legacy-api-as-mcp
6	Segurança e Governança em MCPs	07-api-security-auth-rate-limiting-template 07-api-security-auth-rate-limiting-z
7	MCP em Produção: Publicação e Transports	08-publishing-mcps-private-npm
8	Consumindo o MCP com LangChain.js	09-using-mcp-with-langchain

Unidade 1

Visão Geral do Model Context Protocol (MCP)

Evolução de plugins e function calling para o MCP, destacando descoberta de capacidades, tools, resources, prompts, abstração de domínio, transporte e relação com APIs tradicionais.



Fluxo visual da Unidade 1 — reorganização dos elementos descritos nas aulas.

GITHUB DA UNIDADE

[Repositório oficial da disciplina](#)

Aula	Tema
1	Diferença entre MCPs e o modelo clássico de “tools/plugins”

Aula 1 — Diferença entre MCPs e o modelo clássico de “tools/plugins”

Unidade 1 • Visão Geral do Model Context Protocol (MCP)

OBJETIVOS DE APRENDIZAGEM

- Compreender a evolução até o MCP.
- Relacionar limitações do modelo de tools e function calling com o que o MCP propõe de diferente.
- Aplicar as decisões apresentadas em a questão da abstração e verificar o comportamento descrito na aula.

GITHUB DA UNIDADE

[Repositório do módulo](#)

CONCEITOS-CHAVE

- A evolução até o MCP
- Limitações do modelo de tools e function calling
- O que o MCP propõe de diferente
- A questão da abstração
- Transporte e processamento sob demanda
- MCP substitui tools?



Diagrama de referência — fluxo descrito na aula.

Quadro comparativo — tools/function calling e MCP

Abordagem	Características apresentadas na aula
Tools / Function Calling	Funções são descritas previamente com nome, descrição e parâmetros; o modelo chama capacidades já definidas.
MCP	Protocolo cliente-servidor que incorpora tools e acrescenta resources, prompts, descoberta e contexto da integração.

Conteúdo da aula

Ao longo da evolução recente da inteligência artificial aplicada, muita gente passou a tratar o Model Context Protocol como se fosse uma ruptura completa com tudo o que existia antes. Como se, de um dia para o outro, tivéssemos saído de um cenário limitado para algo totalmente novo e revolucionário. Mas, na prática, não foi exatamente isso que aconteceu.

O que houve foi uma resposta natural a um problema que já existia há muito tempo no desenvolvimento de software: a complexidade de integrações.

Se você já trabalhou com APIs de terceiros, como APIs de pagamento, mapas ou autenticação, provavelmente lembra do esforço envolvido. Cada integração exigia um entendimento profundo da documentação, múltiplos endpoints, autenticação específica e, muitas vezes, uma quantidade enorme de funcionalidades das quais você usava apenas uma pequena parte. O custo de aprendizado era alto, e o retorno nem sempre compensava.

Esse mesmo problema começou a aparecer no contexto de aplicações com modelos de linguagem.

A evolução até o MCP

Antes do MCP, já existiam iniciativas importantes que buscavam resolver a integração entre modelos de linguagem e sistemas externos.

Em março de 2023, surgiu a proposta de plugins para o ChatGPT, que permitia adicionar integrações diretamente na interface, possibilitando que o modelo buscasse dados ou executasse ações externas.

Poucos meses depois, em junho de 2023, foi introduzido o conceito de Function Calling. Esse modelo consolidou uma abordagem mais estruturada: em vez de plugins acoplados à interface, passamos a fornecer funções diretamente ao modelo.

Nesse cenário, o funcionamento era relativamente simples e eficiente. Eu definia uma lista de funções disponíveis, cada uma com seu nome, descrição e estrutura de entrada. O modelo analisava o contexto do prompt, decidia qual função chamar, gerava os argumentos necessários e, então, o sistema executava essa função.

Esse padrão foi extremamente importante. Ele resolveu um problema real, principalmente no que diz respeito à redução de alucinações, já que o modelo passava a operar com capacidades bem definidas.

No entanto, apesar de eficiente, esse modelo tinha limitações claras.

Limitações do modelo de tools e function calling

O principal ponto é que, nesse modelo, o entendimento da integração é superficial.

O modelo sabe que determinadas funções existem, conhece seus parâmetros e consegue acioná-las. Mas ele não tem uma visão mais ampla do contexto daquela integração. Ele não consegue, por exemplo, explorar de forma estruturada o que aquele sistema externo oferece antes de decidir o que fazer.

Na prática, eu precisava descrever manualmente todas as funções relevantes no código. Isso significava um trabalho repetitivo e, muitas vezes, pouco escalável. Além disso, qualquer mudança na integração exigia atualização direta na definição dessas funções.

Outro ponto importante é que esse modelo não fornece mecanismos padronizados de descoberta. O modelo não consegue “investigar” a integração. Ele apenas consome o que foi previamente definido.

O que o MCP propõe de diferente

O Model Context Protocol surge justamente para elevar esse nível de abstração.

Em vez de tratar integrações apenas como uma lista de funções, o MCP define um protocolo de comunicação entre cliente e servidor. Esse protocolo não expõe apenas tools, mas também outros elementos fundamentais, como recursos e prompts.

Isso muda completamente a forma como o modelo interage com sistemas externos.

Com MCP, eu passo a ter endpoints adicionais que descrevem o contexto da integração. Por exemplo, posso fornecer informações detalhadas sobre o que aquele serviço faz, quais são seus objetivos e até exemplos de uso. Isso permite que o modelo compreenda melhor o cenário antes de executar qualquer ação.

Além disso, posso disponibilizar prompts prontos, que ajudam tanto o modelo quanto o usuário a entender como interagir com aquele serviço. Isso reduz drasticamente a necessidade de leitura de documentação extensa.

Um dos pontos mais interessantes é o mecanismo de descoberta automática. Em vez de eu definir manualmente cada função, o cliente conectado ao servidor MCP consegue explorar os recursos disponíveis, entender quais ações existem e como utilizá-las.

Isso representa uma mudança significativa no fluxo de desenvolvimento.

A questão da abstração

Outro aspecto importante do MCP é o nível de abstração que ele incentiva.

No modelo tradicional, é comum expor diretamente os endpoints da API como funções. Por exemplo, uma função para buscar cliente por ID, outra para listar clientes, outra para obter detalhes específicos.

No MCP, a ideia é diferente. Eu passo a trabalhar com ações mais próximas do domínio do problema. Em vez de expor múltiplos endpoints técnicos, posso criar uma única ação que represente uma intenção de negócio, como “buscar cliente por nome”.

Internamente, o servidor MCP pode orquestrar várias chamadas: listar clientes, filtrar por nome, buscar detalhes adicionais e consolidar a resposta. Toda a complexidade de autenticação, autorização e múltiplos endpoints fica encapsulada.

Isso melhora não apenas a experiência do modelo, mas também a qualidade das integrações.

Transporte e processamento sob demanda

Outro diferencial relevante do MCP está na forma como os dados podem ser transportados.

No modelo tradicional, a comunicação tende a ser baseada em requisição e resposta completa. O cliente faz uma chamada e aguarda o retorno integral.

Com MCP, é possível trabalhar com dados sob demanda. Isso significa que o servidor pode enviar informações de forma incremental, permitindo processamento contínuo. Um exemplo claro seria o processamento de vídeo, onde os dados podem ser enviados em partes e analisados quase em tempo real.

Esse tipo de abordagem abre espaço para aplicações mais eficientes e responsivas.

MCP substitui tools?

Uma dúvida comum é se o MCP substitui completamente o conceito de tools.

A resposta é não.

Na prática, o MCP incorpora tools dentro de um protocolo mais amplo. As tools continuam existindo como capacidades expostas ao modelo, mas agora fazem parte de um ecossistema padronizado.

Inclusive, muitos frameworks continuam utilizando o termo “tools”, mesmo quando operam com MCP. Isso acontece porque, no nível de uso, o conceito de ferramenta ainda faz sentido. A diferença é que, por trás, existe um protocolo mais rico organizando essas capacidades.

Comparação com APIs tradicionais

Quando comparamos MCP com abordagens tradicionais, como APIs REST, a diferença fica ainda mais evidente.

Em APIs REST, é comum termos documentações extensas, muitas vezes representadas por arquivos Swagger com dezenas ou centenas de endpoints. Para que um modelo de linguagem utilizasse essa API, seria necessário enviar uma quantidade enorme de informação a cada interação.

Isso tem um custo alto, especialmente considerando que modelos de linguagem são cobrados por volume de dados processados.

O MCP resolve esse problema ao trabalhar com ações em vez de endpoints. O modelo consome apenas o que é necessário naquele momento, solicitando mais informações sob demanda. Isso reduz o consumo de tokens e torna a interação mais eficiente.

Engenharia continua sendo essencial

Apesar de todos os avanços, é importante reforçar que o MCP não elimina a necessidade de uma boa engenharia de software.

Um servidor MCP mal projetado pode ser lento, inseguro e difícil de manter, assim como qualquer outro sistema. A qualidade da modelagem das ações, a organização dos recursos e a eficiência das integrações continuam sendo fatores críticos.

Ou seja, o protocolo facilita, mas não substitui decisões arquiteturais bem feitas.

Conclusão

O MCP não surgiu como uma ruptura isolada, mas como uma evolução natural após a validação do uso de tools e function calling.

Enquanto tools e plugins resolveram o problema de integração no nível das aplicações, o MCP busca resolver no nível do ecossistema. Ele fornece um padrão que permite ao modelo compreender melhor o contexto, explorar capacidades disponíveis e tomar decisões mais informadas antes de executar ações.

Na prática, isso se traduz em mais eficiência, menor custo de processamento e maior flexibilidade na construção de soluções com inteligência artificial.

E, ao utilizar frameworks modernos, é possível explorar todo esse potencial com relativa simplicidade, permitindo que o modelo encadeie chamadas, reutilize resultados e atue de forma realmente inteligente em cenários de integração.

NA PRÁTICA

Abra o(s) diretório(s) da Unidade 1 usando os links do GitHub exibidos no início desta aula. Localize os artefatos citados no conteúdo, reproduza o fluxo apresentado e observe o impacto em testes, estado, tracing, custo ou comportamento, conforme o tema da aula.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “A evolução até o MCP” no fluxo apresentado?
- Que decisão de engenharia está associada a “Limitações do modelo de tools e function calling” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório, nos testes ou no tracing para confirmar que o comportamento descrito na aula está funcionando?

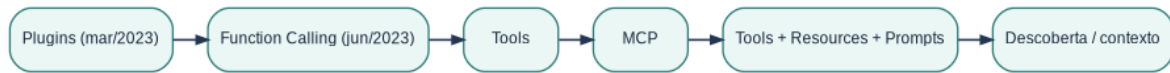
RESUMO DA AULA

- A evolução até o MCP.
- Limitações do modelo de tools e function calling.
- O que o MCP propõe de diferente.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Revisão da Unidade 1

Use esta página como revisão rápida antes de avançar. Você deve conseguir relacionar as aulas da unidade em um único fluxo de engenharia.



Fluxo visual da Unidade 1.

CHECKLIST DA UNIDADE

- [Revisar a Aula 1: Diferença entre MCPs e o modelo clássico de “tools/plugins”](#)

GITHUB DA UNIDADE

[Repositório oficial da disciplina](#)

Unidade 2

MCP para APIs, Bancos de Dados e Serviços Internos

Construção de uma aplicação com múltiplos MCPs e LangChain.js, parsing estruturado da entrada, MongoDB, tool customizada para CSV, File System e reaproveitamento de services como tools.



Fluxo visual da Unidade 2 — reorganização dos elementos descritos nas aulas.

GITHUB DA UNIDADE

[Repositório oficial da disciplina](#)

[01-multiple-mcp-tools-template](#)

[01-multiple-mcp-tools-z](#)

[02-google-trends-agent](#)

Aula	Tema
1	Conhecendo o Projeto: Template Inicial e Arquitetura
2	Parsing Inteligente: Organizando a Entrada do Cliente
3	Orquestração Autônoma com LangChain.js e MCP no MongoDB
4	Construindo Uma Tool Customizada no LangChain
5	Orquestração Autônoma com LangChain.js e MCP para Gerenciamento de Sistema de Arquivos
6	Usando Services Como Tools - Google Trends API com LangChain.js

Aula 1 — Conhecendo o Projeto: Template Inicial e Arquitetura

Unidade 2 • MCP para APIs, Bancos de Dados e Serviços Internos

OBJETIVOS DE APRENDIZAGEM

- Compreender o objetivo da aplicação.
- Relacionar autonomia do modelo e múltiplos MCPs com estrutura inicial do template.
- Aplicar as decisões apresentadas na experiência do usuário final e verificar o comportamento descrito na aula.

GITHUB DA UNIDADE

[01-multiple-mcp-tools-template](#)

[01-multiple-mcp-tools-z](#)

[02-google-trends-agent](#)

CONCEITOS-CHAVE

- Objetivo da aplicação
- Autonomia do modelo e múltiplos MCPs
- Estrutura inicial do template
- Experiência do usuário final
- Primeira etapa: entendimento da intenção
- Definição de ferramentas disponíveis

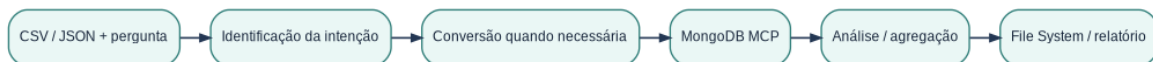


Diagrama de referência — fluxo descrito na aula.

Conteúdo da aula

Nesta etapa do módulo, eu começo apresentando uma aplicação mais completa, pensada justamente para demonstrar, na prática, o potencial do uso de múltiplos servidores MCP trabalhando em conjunto.

A ideia aqui não é apenas mostrar uma integração simples, mas sim mudar a forma como pensamos o fluxo de execução dentro de uma aplicação com inteligência artificial. Em vez de controlar rigidamente cada passo no código, como fizemos anteriormente, eu passo a delegar essa responsabilidade para o modelo, dando a ele autonomia para decidir quando e como utilizar ferramentas, serviços e servidores MCP.

Objetivo da aplicação

O projeto foi estruturado para simular um cenário bastante comum: análise de dados a partir de um relatório.

Nesse caso específico, eu utilizo um relatório de vendas como exemplo, mas o conceito se aplica a qualquer tipo de dado estruturado. O ponto central é permitir que o usuário envie dados e faça perguntas sobre eles, enquanto o sistema decide automaticamente como processar essas informações.

Diferente de abordagens tradicionais, onde eu teria que escrever múltiplos fluxos de execução, aqui eu trabalho com um único prompt bem estruturado. Esse prompt contém tanto a pergunta quanto os dados, e o modelo fica responsável por orquestrar todo o restante.

Autonomia do modelo e múltiplos MCPs

Uma das principais mudanças introduzidas neste projeto é o uso de múltiplos servidores MCP operando de forma coordenada.

Eu não determino explicitamente a ordem de execução das ações no código. Em vez disso, forneço ao modelo um conjunto de ferramentas disponíveis e instruções claras. A partir disso, ele decide:

- Qual ferramenta utilizar
- Em que momento utilizá-la
- Como encadear diferentes operações

Isso representa uma evolução significativa em relação ao modelo anterior, onde o fluxo era rigidamente controlado por lógica imperativa.

Estrutura inicial do template

Para facilitar o desenvolvimento, eu parto de um template já preparado, disponível em repositório. Esse template já contém:

- Estrutura base da aplicação
- Integrações iniciais configuradas
- Serviços MCP prontos para uso
- Ambiente preparado para execução local

Antes de implementar qualquer funcionalidade nova, eu faço uma análise detalhada dessa estrutura, entendendo como cada parte contribui para o funcionamento geral do sistema.

Experiência do usuário final

Do ponto de vista do usuário, a interação é extremamente simples.

Ele fornece:

- Um conjunto de dados, que pode estar em formato CSV ou JSON
- Uma pergunta sobre esses dados

Por exemplo, ele pode solicitar um ranking dos produtos mais vendidos. O sistema, então, interpreta a intenção, processa os dados e retorna a resposta.

O diferencial é que o usuário não precisa especificar formato, estrutura ou etapas intermediárias. Toda essa inteligência fica a cargo do modelo.

Primeira etapa: entendimento da intenção

O fluxo começa com a identificação da intenção do usuário.

A partir do prompt recebido, o sistema extrai:

- O objetivo da solicitação
- O tipo de dado fornecido
- O conteúdo do arquivo
- Um nome apropriado para o arquivo
- O formato do dado

Essa etapa é fundamental, pois orienta todas as decisões subsequentes. Inclusive, eu introduzo a possibilidade de pular etapas com base no contexto. Por exemplo, se o dado já estiver em JSON, não há necessidade de conversão.

Esse tipo de decisão dinâmica é um dos pontos fortes do uso de LLMs com MCP.

Definição de ferramentas disponíveis

No projeto, eu disponibilizo três principais tipos de ferramentas para o modelo:

1. **File System:** Permite ler, escrever e manipular arquivos no disco.
2. **Conversor CSV para JSON:** Uma ferramenta customizada, criada para demonstrar como integrar bibliotecas externas e expandir as capacidades do sistema.
3. **Integração com MongoDB:** Utilizada para armazenamento, consulta e agregação de dados.

O mais importante aqui é que eu não controlo diretamente quando cada ferramenta será usada. Eu apenas descrevo suas capacidades, e o modelo decide como utilizá-las.

Pipeline de execução

O fluxo completo da aplicação é dividido em etapas bem definidas:

1. **Conversão de dados:** Se o conteúdo estiver em CSV, o modelo utiliza a ferramenta de conversão para transformá-lo em JSON. Caso já esteja em JSON, essa etapa é ignorada.
2. **Persistência opcional em arquivo:** Se o usuário solicitar, o sistema salva o conteúdo em disco utilizando o File System.
3. **Inserção no banco de dados:** Os dados são enviados para o MongoDB, onde o modelo define automaticamente o nome da coleção.
4. **Processamento e análise:** O modelo executa consultas no banco de dados para gerar as informações solicitadas, utilizando operações de agregação.
5. **Geração de relatório:** O resultado final é salvo como um arquivo de texto na pasta de relatórios.

Esse pipeline não é rigidamente programado. Ele é descrito em alto nível, e o modelo decide como executá-lo.

Tratamento de falhas e retentativas

Um ponto extremamente relevante é a capacidade de re-tentativa automática.

Se o modelo executar uma operação incorreta, como uma query inválida no banco de dados, ele pode:

- Consultar novamente o servidor MCP
- Buscar exemplos de uso
- Ajustar a execução
- Tentar novamente

Esse comportamento aproxima o sistema de um modelo mais resiliente e autônomo, reduzindo a necessidade de intervenção manual.

Arquitetura baseada em agentes

A aplicação utiliza uma arquitetura baseada em agentes.

Existem dois componentes principais:

- Um agente responsável por entender a intenção
- Um agente responsável por executar as ações

Entre eles, há uma estrutura condicional que interrompe o fluxo caso haja erro na interpretação inicial.

Essa separação melhora a organização do sistema e facilita a evolução futura.

Camada de serviços e integração com MCP

A arquitetura inclui uma camada específica para agregação de servidores MCP.

Essa camada centraliza:

- Conexão com múltiplos MCPs
- Disponibilização das tools
- Integração com o restante da aplicação

Além disso, o serviço principal passa a suportar dois tipos de retorno:

- Dados estruturados em JSON
- Execução de tools via MCP

Essa separação é importante porque define claramente quando o modelo deve apenas interpretar dados e quando deve executar ações externas.

Observabilidade e monitoramento

Para facilitar o entendimento do comportamento do sistema, eu adiciono mecanismos de monitoramento.

Esses mecanismos permitem acompanhar:

- Início e fim das execuções do modelo
- Chamadas de ferramentas
- Entradas e saídas de cada operação
- Tentativas e retentativas

Isso é essencial para a depuração e também para compreender como o modelo está tomando decisões.

Estrutura de diretórios e recursos

O projeto inclui alguns diretórios importantes:

- Pasta de tools, preparada para novas ferramentas
- Pasta de relatórios, onde os resultados são armazenados
- Conjunto de dados de exemplo, utilizado para desenvolvimento

Também há configurações específicas para integração com modelos e serviços externos, incluindo variáveis de ambiente necessárias para autenticação.

Uso de banco de dados e infraestrutura

A aplicação utiliza MongoDB como banco de dados principal.

Além disso, há uma ferramenta visual para inspeção dos dados, facilitando o desenvolvimento e a análise.

A infraestrutura é provisionada via Docker, permitindo subir rapidamente todos os serviços necessários.

Considerações sobre contexto e desempenho

Um ponto importante ao trabalhar com LLMs é o limite de contexto.

Os dados enviados não podem exceder a capacidade do modelo. Por isso, durante o desenvolvimento, utilizo conjuntos reduzidos de dados. Em produção, é necessário pensar em estratégias para lidar com volumes maiores.

Conclusão

Nesta aula, eu apresento a base de uma arquitetura mais avançada para aplicações com inteligência artificial.

O principal ganho está na mudança de paradigma: em vez de controlar o fluxo manualmente, eu passo a definir capacidades e deixar o modelo orquestrar a execução.

Com o uso de múltiplos servidores MCP, ferramentas customizadas e uma arquitetura bem estruturada, é possível construir sistemas muito mais flexíveis, escaláveis e inteligentes.

NA PRÁTICA

Abra o(s) diretório(s) da Unidade 2 usando os links do GitHub exibidos no início desta aula. Localize os artefatos citados no conteúdo, reproduza o fluxo apresentado e observe o impacto em testes, estado, tracing, custo ou comportamento, conforme o tema da aula.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Objetivo da aplicação” no fluxo apresentado?
- Que decisão de engenharia está associada a “Autonomia do modelo e múltiplos MCPs” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório, nos testes ou no tracing para confirmar que o comportamento descrito na aula está funcionando?

RESUMO DA AULA

- Objetivo da aplicação.
- Autonomia do modelo e múltiplos MCPs.
- Estrutura inicial do template.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 2 — Parsing Inteligente: Organizando a Entrada do Cliente

Unidade 2 • MCP para APIs, Bancos de Dados e Serviços Internos

OBJETIVOS DE APRENDIZAGEM

- Compreender a preparação do ambiente.
- Relacionar inicialização da aplicação com estrutura do nó de intenção.
- Aplicar as decisões apresentadas em uso de geração estruturada e verificar o comportamento descrito na aula.

GITHUB DA UNIDADE

[01-multiple-mcp-tools-template](#)

[01-multiple-mcp-tools-z](#)

[02-google-trends-agent](#)

CONCEITOS-CHAVE

- Preparação do ambiente
- Inicialização da aplicação
- Estrutura do nó de intenção
- Uso de geração estruturada
- Validação dos dados extraídos
- Tratamento de inconsistências

Conteúdo da aula

Nesta etapa do projeto, eu começo a estruturar um dos pontos mais importantes de toda a aplicação: o parsing inteligente da entrada do cliente.

Até aqui, já temos uma arquitetura preparada, com múltiplos servidores MCP e um fluxo orientado por agentes. Agora, o foco passa a ser garantir que o modelo consiga interpretar corretamente o que o usuário está enviando, organizando essas informações de forma estruturada para os próximos passos.

Esse momento é crítico, porque qualquer erro na interpretação inicial compromete todo o restante do pipeline.

Preparação do ambiente

Antes de implementar a lógica de parsing, eu começo garantindo que o ambiente esteja corretamente configurado.

Eu valido a versão do Node.js, assegurando compatibilidade com o projeto. Em seguida, restauro as dependências utilizando o gerenciador de pacotes, garantindo que todas as bibliotecas estejam disponíveis.

Também utilizo uma ferramenta para listar e executar scripts do projeto, o que facilita bastante a interação com os comandos definidos. A partir disso, eu subo a infraestrutura necessária, incluindo o banco de dados MongoDB e uma interface visual para inspeção dos dados.

Essa preparação é importante porque o parsing não acontece isoladamente. Ele faz parte de um fluxo maior, que depende de serviços externos funcionando corretamente.

Inicialização da aplicação

Com o ambiente pronto, eu inicio os serviços da aplicação e verifico se os grafos e fluxos estão ativos.

Nesse ponto, mesmo sem lógica completa implementada, já consigo observar que os dados estão sendo enviados para o sistema. Isso permite validar rapidamente se a comunicação básica está funcionando antes de avançar.

Essa abordagem incremental é essencial quando trabalhamos com sistemas baseados em LLMs, pois facilita a identificação de problemas logo nas primeiras etapas.

Estrutura do nó de intenção

O primeiro componente que implemento é o nó responsável por identificar a intenção do usuário.

Esse nó tem como responsabilidade transformar uma entrada não estruturada em um objeto estruturado, contendo informações essenciais para o restante do sistema.

Para isso, utilizo um serviço de geração estruturada, que recebe:

Um prompt de sistema, contendo as instruções de como interpretar a entrada

Um prompt do usuário, com a pergunta e os dados enviados

Um schema que define exatamente qual estrutura deve ser retornada

Esse schema é fundamental, pois orienta o modelo a produzir uma saída previsível, reduzindo ambiguidades.

Uso de geração estruturada

Ao utilizar geração estruturada, eu consigo garantir que o retorno siga um formato específico.

Esse retorno inclui, por exemplo:

- A intenção do usuário

- O tipo do arquivo fornecido

- O conteúdo do arquivo

- Um nome sugerido para o arquivo

Essa padronização é essencial para evitar erros nas etapas seguintes.

No entanto, é importante observar que o serviço pode retornar diferentes tipos de resposta. Por isso, eu faço uma validação explícita do conteúdo retornado.

Validação dos dados extraídos

Após receber o resultado do modelo, eu realizo uma série de validações.

Se a intenção não for identificada corretamente, ou se o tipo de arquivo não estiver presente, eu interrompo o fluxo imediatamente. Não faz sentido continuar o processamento sem essas informações básicas.

Essa validação funciona como uma barreira de segurança, garantindo que apenas dados consistentes avancem no pipeline.

Tratamento de inconsistências

Um ponto interessante que observo durante a implementação é que nem sempre o modelo consegue inferir corretamente todas as informações.

Um exemplo disso é o nome do arquivo. Em alguns casos, o modelo não consegue gerar um nome adequado apenas com base no contexto.

Para lidar com isso, eu implemento um fallback. Caso o nome do arquivo não seja fornecido, eu atribuo um valor padrão baseado no tipo do arquivo.

Essa estratégia garante que o fluxo continue funcionando mesmo diante de pequenas inconsistências na interpretação.

Organização dos dados para o próximo estágio

Depois de validar e ajustar os dados, eu organizo as informações em uma estrutura que será utilizada pelos próximos nós do sistema.

Essa estrutura inclui:

- A intenção interpretada

- O conteúdo do arquivo

- O nome do arquivo

A partir desse ponto, o restante da aplicação passa a trabalhar com dados estruturados, o que simplifica significativamente a lógica de execução.

Uso de logs e depuração

Durante o desenvolvimento, eu utilizo logs e pontos de interrupção para acompanhar o comportamento do sistema.

Isso me permite observar:

- O que está sendo enviado para o modelo

- O que está sendo retornado

- Como os dados estão sendo transformados

Essa visibilidade é fundamental, principalmente quando lidamos com modelos probabilísticos, onde o comportamento pode variar dependendo do contexto.

Importância do parsing inteligente

O parsing inteligente não é apenas uma etapa inicial, mas sim a base de todo o sistema.

Se essa etapa for bem construída, todo o restante do fluxo se torna mais simples, previsível e robusto. Caso contrário, erros se propagam e tornam o sistema difícil de manter.

Ao estruturar corretamente a entrada do cliente, eu consigo:

- Reduzir ambiguidades

- Melhorar a tomada de decisão do modelo

Facilitar a integração com ferramentas externas

Aumentar a confiabilidade do sistema

Integração com o restante da arquitetura

Esse nó de parsing se conecta diretamente com o agente responsável pela execução das ações.

Caso a intenção seja identificada com sucesso, o fluxo segue normalmente. Caso contrário, o processo é interrompido.

Essa separação de responsabilidades mantém a arquitetura limpa e facilita a evolução do sistema.

Conclusão

Nesta etapa, eu estabeleço as bases para um sistema realmente inteligente.

Ao transformar entradas não estruturadas em dados organizados, eu permito que o modelo atue de forma mais precisa e eficiente.

O uso de geração estruturada, validações rigorosas e tratamento de inconsistências garante que o sistema seja resiliente e preparado para lidar com diferentes tipos de entrada.

Esse é um dos pontos mais importantes de toda a aplicação, pois define a qualidade de todas as decisões que serão tomadas a partir daqui.

NA PRÁTICA

Abra o(s) diretório(s) da Unidade 2 usando os links do GitHub exibidos no início desta aula. Localize os artefatos citados no conteúdo, reproduza o fluxo apresentado e observe o impacto em testes, estado, tracing, custo ou comportamento, conforme o tema da aula.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Preparação do ambiente” no fluxo apresentado?
- Que decisão de engenharia está associada a “Inicialização da aplicação” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório, nos testes ou no tracing para confirmar que o comportamento descrito na aula está funcionando?

RESUMO DA AULA

- Preparação do ambiente.
- Inicialização da aplicação.
- Estrutura do nó de intenção.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 3 — Orquestração Autônoma com LangChain.js e MCP no MongoDB

Unidade 2 • MCP para APIs, Bancos de Dados e Serviços Internos

OBJETIVOS DE APRENDIZAGEM

- Compreender a transição do parsing para execução.
- Relacionar execução sem tools: comportamento inicial com introdução das tools via MCP.
- Aplicar as decisões apresentadas em cuidados com segurança em MCP e verificar o comportamento descrito na aula.

GITHUB DA UNIDADE

[01-multiple-mcp-tools-template](#)

[01-multiple-mcp-tools-z](#)

[02-google-trends-agent](#)

CONCEITOS-CHAVE

- Transição do parsing para execução
- Execução sem tools: comportamento inicial
- Introdução das tools via MCP
- Cuidados com segurança em MCP
- Configuração do MongoDB como tool
- Camada de agregação de MCPs



Diagrama de referência — fluxo descrito na aula.

Conteúdo da aula

Com a intenção do cliente já estruturada, eu avanço para a etapa onde, de fato, começo a explorar o potencial da orquestração autônoma utilizando LangChain.js em conjunto com servidores MCP.

Até aqui, o sistema já é capaz de entender o que o usuário deseja. Agora, o próximo passo é permitir que ele execute ações reais, utilizando ferramentas externas de forma inteligente e independente.

Transição do parsing para execução

No fluxo da aplicação, saio do nó de identificação de intenção e entro no nó responsável pela execução, que é o agente.

Nesse ponto, eu já tenho disponível:

- A intenção do usuário
- O nome do arquivo
- O conteúdo dos dados

Com essas informações em mãos, eu construo o prompt que será enviado ao modelo. Esse prompt contém todo o contexto necessário para que ele tome decisões sobre quais ações executar.

Diferente da etapa anterior, aqui eu não utilizo um schema estruturado. Isso acontece porque agora quero permitir que o modelo utilize tools, e não apenas retorne dados organizados.

Essa decisão é fundamental, pois habilita o comportamento autônomo do sistema.

Execução sem tools: comportamento inicial

Antes de integrar qualquer ferramenta, eu executo o fluxo apenas com o modelo, sem disponibilizar tools.

O resultado é interessante: o modelo consegue interpretar os dados, converter CSV para JSON e até gerar agregações por conta própria.

Isso demonstra a capacidade do modelo de resolver problemas de forma isolada. No entanto, esse comportamento não é ideal para produção.

Existem dois motivos principais para isso:

- O modelo não foi projetado para executar cálculos complexos com precisão garantida
- O processamento dentro do modelo consome tokens, o que impacta custo e desempenho

Por isso, a estratégia correta é delegar essas responsabilidades para ferramentas externas.

Introdução das tools via MCP

A partir desse ponto, começo a integrar ferramentas reais utilizando o padrão MCP.

A primeira ferramenta que adiciono é o servidor MCP do MongoDB. Esse servidor permite que o modelo:

- Crie e remova coleções
- Insira dados
- Execute consultas
- Realize agregações

Essa integração é feita através de uma configuração que define como o servidor MCP será acessado.

Cuidados com segurança em MCP

Antes de utilizar qualquer servidor MCP, eu reforço um ponto importante: segurança.

Nem todo repositório disponível é confiável. Como essas ferramentas podem ser executadas diretamente no ambiente local, existe o risco de exposição de dados sensíveis.

Por isso, sempre priorizo:

- Repositórios oficiais
- Ferramentas mantidas pelos próprios fornecedores
- Fontes confiáveis

Esse cuidado é essencial em ambientes reais.

Configuração do MongoDB como tool

A integração com o MongoDB segue um padrão bem definido.

Eu crio uma função responsável por retornar a configuração do servidor MCP. Essa configuração inclui:

- Nome da ferramenta
- Tipo de transporte
- Forma de execução
- Permissões de leitura e escrita

Também defino o banco de dados que será utilizado. Caso ele não exista, o próprio MongoDB se encarrega de criá-lo automaticamente.

Esse nível de abstração simplifica bastante o processo.

Camada de agregação de MCPs

Para organizar melhor o sistema, eu utilizo uma camada intermediária responsável por gerenciar todos os servidores MCP.

Essa camada:

- Centraliza a configuração das ferramentas
- Inicializa os servidores MCP
- Converte os recursos disponíveis em tools utilizáveis pelo modelo

Essa conversão é feita utilizando adaptadores, que permitem que frameworks como LangChain trabalhem com MCPs de forma transparente.

Integração com o agente

Com o MongoDB configurado, eu adiciono essa tool ao agente.

A partir desse momento, o modelo passa a ter acesso a essa capacidade. E aqui acontece algo interessante: eu não preciso dizer explicitamente quando usar o banco de dados.

Eu apenas descrevo as instruções no prompt e disponibilizo a ferramenta. O modelo decide sozinho quando utilizá-la.

Execução autônoma na prática

Ao executar o fluxo com a tool integrada, o comportamento muda significativamente.

O modelo passa a:

- Limpar o banco de dados automaticamente
- Inserir os dados recebidos
- Executar consultas
- Gerar resultados com base nas agregações

Tudo isso sem que eu tenha escrito uma lógica imperativa detalhando cada passo.

Essa é a essência da orquestração autônoma.

Observabilidade do processo

Durante a execução, eu consigo acompanhar tudo o que está acontecendo através de logs.

Consigo visualizar:

- Quando o modelo decide chamar uma tool
- Qual ferramenta foi utilizada
- Quais parâmetros foram enviados
- Qual foi o resultado da execução

Essa visibilidade é fundamental para entender o comportamento do sistema e validar se as decisões estão corretas.

Validação dos resultados

Após a execução, eu posso validar os dados diretamente no banco.

Utilizando uma interface visual, verifico:

- Se os dados foram inseridos corretamente
- Se a quantidade de registros está correta
- Se as agregações fazem sentido

Essa etapa confirma que o modelo não apenas executou ações, mas fez isso de forma consistente.

Tratamento de falhas

Um dos pontos mais interessantes é a capacidade do modelo de lidar com falhas.

Mesmo quando uma ferramenta não está disponível, como no caso de operações de escrita em arquivo ainda não implementadas, o modelo continua o fluxo.

Ele tenta alternativas, utiliza outras ferramentas disponíveis e segue até completar o processo.

Esse comportamento demonstra um nível avançado de resiliência.

Encadeamento de decisões

Outro aspecto importante é o encadeamento de ações.

O modelo não executa apenas uma operação isolada. Ele consegue:

- Interpretar o problema
- Planejar uma sequência de passos
- Executar cada etapa
- Reutilizar resultados intermediários

Isso transforma o modelo em um verdadeiro orquestrador de processos.

Limitações e próximos passos

Apesar do avanço significativo, ainda existem pontos a melhorar.

Por exemplo:

- A conversão de CSV para JSON ainda está sendo feita pelo modelo
- O sistema ainda não possui integração com file system
- Algumas operações podem ser otimizadas com tools específicas

Esses pontos serão resolvidos nas próximas etapas, com a adição de novos MCPs e ferramentas customizadas.

NA PRÁTICA

Abra o(s) diretório(s) da Unidade 2 usando os links do GitHub exibidos no início desta aula. Localize os artefatos citados no conteúdo, reproduza o fluxo apresentado e observe o impacto em testes, estado, tracing, custo ou comportamento, conforme o tema da aula.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Transição do parsing para execução” no fluxo apresentado?
- Que decisão de engenharia está associada a “Execução sem tools: comportamento inicial” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório, nos testes ou no tracing para confirmar que o comportamento descrito na aula está funcionando?

RESUMO DA AULA

- Transição do parsing para execução.
- Execução sem tools: comportamento inicial.
- Introdução das tools via MCP.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 4 — Construindo Uma Tool Customizada no LangChain

Unidade 2 • MCP para APIs, Bancos de Dados e Serviços Internos

OBJETIVOS DE APRENDIZAGEM

- Compreender por que criar uma tool customizada.
- Relacionar o papel da tool no fluxo da aplicação com continuidade do ambiente já montado.
- Aplicar as decisões apresentadas na escolha da biblioteca para conversão e verificar o comportamento descrito na aula.

GITHUB DA UNIDADE

[01-multiple-mcp-tools-template](#)

[01-multiple-mcp-tools-z](#)

[02-google-trends-agent](#)

CONCEITOS-CHAVE

- Por que criar uma tool customizada
- O papel da tool no fluxo da aplicação
- Continuidade do ambiente já montado
- Escolha da biblioteca para conversão
- Estrutura da tool no LangChain
- Nome, descrição e schema

Conteúdo da aula

Nesta aula, eu passo a trabalhar em um ponto essencial para quem quer sair do uso básico de modelos de linguagem e começar a construir aplicações realmente úteis: a criação de tools customizadas.

Até aqui, já vimos como integrar ferramentas externas por meio de MCPs e como permitir que o modelo orquestre chamadas com certo grau de autonomia. Agora, eu dou um passo intermediário muito importante. Antes mesmo de construir um servidor MCP completo, eu mostro como criar uma ferramenta menor, focada e específica, capaz de executar uma única tarefa com precisão.

A proposta aqui é simples, mas extremamente didática: criar uma tool responsável por converter conteúdo CSV em JSON.

Por que criar uma tool customizada

Em muitos cenários, eu não preciso necessariamente de um servidor MCP inteiro. Às vezes, tudo o que preciso é de uma função especializada, que execute uma tarefa pontual dentro do fluxo da aplicação.

Esse é exatamente o caso desta aula.

O modelo já está conduzindo o processo geral, entendendo a intenção do usuário e se conectando a outras ferramentas, como o MongoDB. No entanto, deixar a conversão de CSV para JSON inteiramente nas mãos da LLM não é a melhor escolha.

Embora o modelo consiga fazer essa conversão em muitos casos, essa abordagem traz limitações importantes:

- o processamento fica mais sujeito a inconsistências
- a precisão pode variar conforme o contexto
- há consumo desnecessário de tokens
- uma tarefa técnica e determinística acaba sendo tratada como inferência probabilística

Ao criar uma tool específica para isso, eu retiro essa responsabilidade da LLM e coloco a conversão em uma camada técnica mais confiável.

O papel da tool no fluxo da aplicação

A tool construída nesta etapa assume uma função clara dentro do pipeline.

O usuário envia os dados em formato CSV. O modelo entende o contexto e decide que precisa transformar aquele conteúdo em uma estrutura JSON. Em vez de fazer isso diretamente por inferência, ele chama a tool customizada, que recebe o texto do CSV, realiza a conversão e devolve o resultado em formato adequado para o restante do processo.

Depois disso, o fluxo continua normalmente, com os dados já estruturados sendo enviados para o MongoDB e utilizados nas etapas seguintes.

Essa separação de responsabilidades melhora muito a robustez da aplicação.

Continuidade do ambiente já montado

Nesta aula, eu não recomeço o projeto do zero. Continuo exatamente do ponto em que o ambiente já estava funcionando.

Isso significa que:

- o MongoDB já está ativo
- os serviços anteriores permanecem de pé
- o projeto continua configurado
- a estrutura de tools já está disponível

Essa continuidade é importante porque mostra que a adição de uma tool customizada não exige reorganizar toda a arquitetura. Ela se encaixa naturalmente no sistema já construído.

Escolha da biblioteca para conversão

Para fazer a conversão de CSV em JSON, eu utilizo uma biblioteca já preparada para lidar com dados textuais.

Esse ponto é relevante porque demonstra que uma tool customizada não precisa reinventar a roda. Posso aproveitar bibliotecas do ecossistema JavaScript, especialmente aquelas já consolidadas, e integrá-las ao fluxo do LangChain.

Isso amplia bastante as possibilidades. Sempre que eu quiser incorporar uma capacidade técnica específica, posso encapsular uma biblioteca existente dentro de uma tool e expor essa funcionalidade ao modelo.

Estrutura da tool no LangChain

A construção da tool segue o padrão esperado pelo LangChain.

Primeiro, eu importo a função utilitária responsável por criar tools. Depois, importo também a biblioteca de conversão de CSV para JSON e o mecanismo de validação que será usado para definir o schema de entrada.

A tool, então, passa a ser construída a partir de dois blocos principais:

- a função que executa a lógica
- o objeto de configuração que descreve a tool

Essa separação é importante porque não basta apenas implementar o comportamento técnico. Também é necessário explicar ao modelo o que a tool faz, como ela deve ser chamada e quais parâmetros ela espera receber.

Nome, descrição e schema

Para que a LLM consiga usar a tool corretamente, eu defino metadados claros.

Entre esses metadados, estão:

- o nome da ferramenta
- uma descrição objetiva da sua finalidade
- o schema de entrada
- mensagens associadas à validação

O nome precisa ser claro e consistente, porque é a partir dele que o modelo identifica a capacidade disponível.

A descrição também tem um papel central. Ela orienta o modelo a entender que essa tool existe justamente para converter CSV em JSON, evitando que ele tente fazer a conversão por conta própria.

Já o schema define a estrutura esperada na chamada. Neste caso, a entrada é basicamente um texto contendo os dados em CSV.

Validação com schema

A validação da entrada é um dos aspectos mais importantes dessa implementação.

Ao definir um schema, eu garanto que o modelo só conseguirá chamar a tool se fornecer os parâmetros esperados no formato correto. Isso evita chamadas malformadas, reduz erros em tempo de execução e aumenta a previsibilidade do sistema.

Na prática, a validação funciona como um contrato entre a LLM e a função executada no meu ambiente.

Esse ponto é decisivo porque, quando começamos a permitir que modelos chamem funções diretamente, precisamos criar barreiras claras entre a liberdade do modelo e a segurança da aplicação.

Implementação da lógica de conversão

A lógica interna da tool é bastante simples.

Depois de receber o texto CSV validado, eu utilizo a biblioteca já instalada para processar esse conteúdo e produzir uma saída em JSON.

Além disso, posso registrar logs para observar o comportamento da conversão, como a quantidade de itens processados e o momento em que a operação termina. Isso ajuda muito na depuração e no entendimento do fluxo.

No final, o resultado é retornado em formato de texto.

Esse detalhe é importante. Embora o processamento gere um objeto estruturado, a comunicação com a LLM funciona melhor quando a resposta é convertida para string, já que esse é o formato mais natural para o intercâmbio entre o modelo e as tools.

Integração da tool ao serviço

Com a tool pronta, eu a adiciono à camada de serviço que centraliza as ferramentas disponíveis para o modelo.

Esse passo é extremamente simples: basta importar a função da nova tool e incluí-la na lista de recursos que o agente pode utilizar.

A partir desse momento, a LLM passa a enxergar essa nova capacidade como parte do ambiente disponível.

O interessante é que eu não preciso reescrever a lógica principal da aplicação. A arquitetura já foi preparada para receber novas tools, e isso torna a expansão do sistema muito natural.

Observando a chamada em tempo real

Um dos momentos mais interessantes da aula é quando eu deixo um ponto de interrupção no código e observo a LLM chamando a tool.

Isso mostra, de forma muito concreta, o que está acontecendo por trás da abstração.

O modelo analisa o problema, entende que precisa converter o conteúdo, chama a ferramenta, envia o texto CSV como entrada e aguarda o retorno para continuar o processo.

Ver essa sequência acontecendo no depurador ajuda muito a entender que não se trata de magia. O modelo realmente está interagindo com funções definidas no meu ambiente, e isso abre espaço para integrar regras de negócio, bibliotecas próprias da empresa e processos internos com muita flexibilidade.

Benefícios práticos dessa abordagem

Ao construir uma tool customizada, eu ganho várias vantagens imediatas.

A primeira é a confiabilidade. Em vez de depender da interpretação do modelo para uma tarefa técnica, eu centralizo essa responsabilidade em uma função específica.

A segunda é a economia de tokens. O modelo deixa de gastar contexto tentando executar algo que pode ser resolvido por código de forma muito mais eficiente.

A terceira é a modularidade. Cada nova capacidade pode ser adicionada como uma ferramenta independente, sem que eu precise transformar tudo em um servidor MCP completo.

Isso é especialmente útil para tarefas menores, onde o custo de criar um protocolo inteiro não se justifica.

Diferença entre tool customizada e servidor MCP

Esta aula também ajuda a esclarecer uma distinção importante.

Uma tool customizada não é, necessariamente, um servidor MCP.

Ela é uma função exposta ao modelo dentro do contexto da aplicação. Já o MCP representa uma camada mais ampla de protocolo, descoberta, transporte e integração entre cliente e servidor.

Ou seja, a tool customizada é mais direta, mais localizada e mais simples de construir. Ela atende muito bem a tarefas específicas, especialmente quando eu quero encapsular lógica interna sem a necessidade de um servidor dedicado.

Por isso, esta etapa funciona como uma ponte entre o uso básico de tools e a construção posterior de MCPs completos.

Resiliência do fluxo

Mesmo com a nova tool funcionando, ainda existem outras partes do sistema que não foram integradas, como o File System.

E o comportamento do modelo diante disso é muito interessante. Ele percebe que certas capacidades ainda não estão disponíveis, registra os erros correspondentes e continua avançando com o que tem à disposição.

Esse comportamento reforça a ideia de que estamos trabalhando com um sistema progressivamente enriquecido. A cada nova tool adicionada, o modelo passa a operar com mais competência, sem que seja necessário reconstruir a lógica geral.

NA PRÁTICA

Abra o(s) diretório(s) da Unidade 2 usando os links do GitHub exibidos no início desta aula. Localize os artefatos citados no conteúdo, reproduza o fluxo apresentado e observe o impacto em testes, estado, tracing, custo ou comportamento, conforme o tema da aula.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Por que criar uma tool customizada” no fluxo apresentado?
- Que decisão de engenharia está associada a “O papel da tool no fluxo da aplicação” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório, nos testes ou no tracing para confirmar que o comportamento descrito na aula está funcionando?

RESUMO DA AULA

- Por que criar uma tool customizada.
- O papel da tool no fluxo da aplicação.
- Continuidade do ambiente já montado.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 5 — Orquestração Autônoma com LangChain.js e MCP para Gerenciamento de Sistema de Arquivos

Unidade 2 • MCP para APIs, Bancos de Dados e Serviços Internos

OBJETIVOS DE APRENDIZAGEM

- Compreender a expansão das capacidades da IA.
- Relacionar escolha do MCP de File System com configuração da tool de File System.
- Aplicar as decisões apresentadas em controle de escopo e impacto no contexto e verificar o comportamento descrito na aula.

GITHUB DA UNIDADE

[01-multiple-mcp-tools-template](#)

[01-multiple-mcp-tools-z](#)

[02-google-trends-agent](#)

CONCEITOS-CHAVE

- Expansão das capacidades da IA
- Escolha do MCP de File System
- Configuração da tool de File System
- Controle de escopo e impacto no contexto
- Integração com a camada de MCP
- Execução prática com File System

Conteúdo da aula

Para fechar a construção desse projeto, eu avanço para a última peça fundamental da arquitetura: permitir que o modelo interaja com o sistema de arquivos.

Até aqui, o sistema já é capaz de interpretar a intenção do usuário, converter dados estruturais com uma tool customizada e persistir informações em banco de dados utilizando MCP. O que falta agora é dar ao modelo a capacidade de materializar resultados, escrevendo e lendo arquivos diretamente no disco.

Essa etapa completa o ciclo de uma aplicação real.

Expansão das capacidades da IA

Quando eu adiciono acesso ao sistema de arquivos, eu deixo de ter apenas um sistema que responde perguntas ou manipula dados em memória. Eu passo a ter um sistema que:

- Gera artefatos persistentes
- Organiza resultados em diretórios
- Permite auditoria e rastreabilidade
- Pode ser integrado a outros fluxos fora da aplicação

Isso muda completamente o tipo de solução que posso construir.

Escolha do MCP de File System

Para implementar essa funcionalidade, eu utilizo um servidor MCP específico para File System.

Esse servidor já disponibiliza uma série de capacidades prontas, como:

- Leitura de arquivos
- Escrita de arquivos
- Listagem de diretórios
- Movimentação de arquivos
- Consulta de metadados

O mais importante aqui é entender que eu não preciso implementar essas funcionalidades manualmente. Eu apenas integro um servidor MCP já existente e exponho essas capacidades ao modelo.

Configuração da tool de File System

Assim como fiz com o MongoDB, eu crio uma função responsável por fornecer a configuração do MCP de File System.

Essa configuração inclui:

- Nome da ferramenta
- Forma de execução
- Tipo de transporte

- Diretório acessível

Esse último ponto é extremamente importante.

Controle de escopo e impacto no contexto

Ao definir o diretório que o modelo pode acessar, eu estou delimitando o escopo da operação.

Se eu conceder acesso a toda a estrutura do projeto, o modelo pode tentar explorar esse contexto, lendo arquivos desnecessários. Isso pode causar dois problemas:

- Aumento no consumo de tokens
- Perda de foco na tarefa principal

Por isso, eu restrinjo o acesso apenas à pasta de relatórios.

Dessa forma, o modelo tem acesso somente ao que é necessário para cumprir sua função: salvar os resultados finais.

Essa decisão é fundamental para manter eficiência e previsibilidade.

Integração com a camada de MCP

Com a configuração pronta, eu adiciono essa nova tool à mesma camada de agregação de MCPs utilizada anteriormente.

Isso significa que:

- Não preciso alterar o fluxo principal
- Não preciso reescrever o agente
- Apenas amplio o conjunto de capacidades disponíveis

Essa modularidade é uma das maiores vantagens da arquitetura adotada.

Execução prática com File System

Com o File System integrado, eu executo novamente o fluxo da aplicação.

Agora, além de:

- Interpretar a intenção
- Converter dados
- Persistir no banco

O modelo também consegue:

- Gerar um relatório final
- Escrever esse relatório em arquivo
- Organizar esse arquivo dentro do diretório definido

E tudo isso acontece de forma autônoma.

O modelo entende que precisa salvar o resultado e escolhe a ferramenta adequada para isso.

Observando o comportamento do sistema

Ao analisar os logs e os resultados, eu consigo ver claramente o fluxo completo acontecendo:

- O banco de dados é limpo
- Os dados são processados
- As agregações são realizadas
- O resultado é gerado
- O arquivo é criado no sistema

Essa sequência demonstra o nível de maturidade da aplicação construída.

Validação dos resultados gerados

Após a execução, eu posso acessar diretamente o diretório de relatórios e verificar os arquivos gerados.

Os arquivos contêm:

- O resultado da análise
- Informações processadas pelo modelo
- Estrutura organizada e legível

Isso permite validar não apenas a execução, mas também a qualidade do output.

Testes com diferentes cenários

Com a estrutura completa, eu começo a testar diferentes tipos de perguntas.

Por exemplo:

- Cálculo de receita total
- Agregações específicas
- Filtros por categoria

O modelo consegue lidar bem com cenários mais simples, especialmente quando os dados são limitados.

No entanto, quando aumento significativamente o volume de dados, começam a surgir limitações.

Limitações com grandes volumes de dados

Ao trabalhar com arquivos maiores, especialmente em modelos gratuitos, observo alguns comportamentos importantes:

- Dificuldade no parsing de grandes volumes
- Falhas na conversão de dados
- Erros em inserções no banco
- Perda de contexto durante a execução

Esses problemas não estão relacionados à arquitetura em si, mas às limitações do modelo utilizado.

Isso reforça uma prática importante: para dados grandes, o ideal é não enviar tudo diretamente no prompt.

Estratégia recomendada para dados maiores

Em cenários reais, o ideal é:

- Fazer upload de arquivos
- Utilizar tools para leitura sob demanda
- Processar dados em partes
- Persistir incrementalmente

Essa abordagem evita sobrecarga no modelo e melhora significativamente a performance.

Comportamento adaptativo do modelo

Mesmo diante de erros, o modelo demonstra um comportamento interessante.

Ele tenta:

- Reexecutar operações
- Ajustar chamadas
- Buscar alternativas dentro das ferramentas disponíveis

Esse comportamento reforça a ideia de orquestração autônoma.

O sistema não depende de um fluxo rígido. Ele se adapta conforme as condições.

Comparação entre tools e MCP

Ao final desta etapa, fica muito clara a diferença entre tools e MCP.

Tools são funções específicas, com entrada e saída bem definidas. Elas resolvem problemas pontuais.

O MCP, por outro lado, é um protocolo mais amplo. Ele fornece:

- Descoberta de capacidades
- Contexto sobre as ferramentas
- Múltiplos endpoints
- Integração padronizada

No projeto, eu utilizo ambos de forma complementar.

Consolidação da arquitetura

Com a adição do File System, a arquitetura atinge um nível bastante completo.

Agora eu tenho:

- Parsing inteligente de entrada
- Tools customizadas para processamento
- Integração com banco de dados via MCP
- Escrita e leitura de arquivos via MCP
- Orquestração autônoma pelo modelo

Tudo isso funcionando de forma integrada.

Conclusão

Nesta aula, eu concluo a construção de um fluxo completo de aplicação com inteligência artificial aplicada.

Ao integrar o File System via MCP, eu permito que o modelo não apenas processe informações, mas também produza resultados concretos e persistentes.

Esse é um passo essencial para transformar experimentos com LLM em soluções reais de software.

Além disso, fica evidente como a combinação de tools e MCPs permite construir sistemas modulares, extensíveis e altamente adaptáveis.

A partir daqui, o próximo passo natural é avançar para a criação de servidores MCP próprios, elevando ainda mais o nível de controle e personalização das integrações.

NA PRÁTICA

Abra o(s) diretório(s) da Unidade 2 usando os links do GitHub exibidos no início desta aula. Localize os artefatos citados no conteúdo, reproduza o fluxo apresentado e observe o impacto em testes, estado, tracing, custo ou comportamento, conforme o tema da aula.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Expansão das capacidades da IA” no fluxo apresentado?
- Que decisão de engenharia está associada à “Escolha do MCP de File System” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório, nos testes ou no tracing para confirmar que o comportamento descrito na aula está funcionando?

RESUMO DA AULA

- Expansão das capacidades da IA.
- Escolha do MCP de File System.
- Configuração da tool de File System.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 6 — Usando Services Como Tools - Google Trends API com LangChain.js

Unidade 2 • MCP para APIs, Bancos de Dados e Serviços Internos

OBJETIVOS DE APRENDIZAGEM

- Compreender a proposta da aula.
- Relacionar por que usar uma service como tool com o caso do Google Trends.
- Aplicar as decisões apresentadas em estrutura geral do projeto e verificar o comportamento descrito na aula.

GITHUB DA UNIDADE

[01-multiple-mcp-tools-template](#)

[01-multiple-mcp-tools-z](#)

[02-google-trends-agent](#)

CONCEITOS-CHAVE

- A proposta da aula
- Por que usar uma service como tool
- O caso do Google Trends
- Estrutura geral do projeto
- O papel do prompt nessa orquestração
- A service como camada de negócio

Conteúdo da aula

Nesta aula, eu amplio a discussão sobre integração com modelos de linguagem para mostrar um cenário muito prático: como aproveitar services já existentes e expô-las como tools dentro de uma aplicação com LangChain.js.

O objetivo aqui não é construir mais um projeto completo do zero, mas apresentar uma ideia que ajuda a repensar como aplicações com inteligência artificial podem ser desenhadas. Em vez de depender exclusivamente de MCPs prontos, eu mostro que também é possível transformar uma service tradicional em uma tool utilizável pela LLM, preservando a autonomia do modelo e, ao mesmo tempo, reaproveitando regras de negócio já construídas.

A proposta da aula

A ideia central é bastante direta.

Eu imagino um cenário em que o usuário quer obter sugestões de títulos para vídeos com base no que está em alta no Google Trends. Em vez de simplesmente pedir que a LLM invente algumas opções, eu quero enriquecer essa resposta com dados reais de tendência.

Nesse caso, o fluxo passa a ser o seguinte:

- o usuário informa o tema de interesse
- a LLM interpreta a intenção
- extrai palavras-chave relevantes
- chama uma tool que consulta a API do Google Trends
- recebe os dados processados
- e, por fim, responde com sugestões mais embasadas

Esse tipo de arquitetura é muito mais próximo do que encontramos em produção.

Por que usar uma service como tool

Nem sempre existe um MCP pronto para tudo o que eu quero integrar. E, mesmo quando existe, pode ser que eu já tenha uma service própria, com regras de negócio específicas da minha empresa ou do meu produto.

É exatamente aí que essa abordagem se torna poderosa.

Em vez de chamar essa service manualmente dentro do fluxo do nó, eu a exponho como uma tool. Assim, o modelo ganha autonomia para decidir quando usar aquele recurso, mas a lógica técnica continua encapsulada em uma camada de serviço convencional.

Na prática, eu combino o melhor dos dois mundos:

- autonomia da LLM para tomar decisões
- controle da aplicação sobre a lógica de negócio
- reaproveitamento de serviços já existentes
- menor acoplamento entre fluxo e implementação

O caso do Google Trends

Para esse exemplo, eu utilizo uma API externa que agrega serviços do ecossistema Google, incluindo o Google Trends.

Essa API exige chave de acesso e possui um plano gratuito com limite de consultas. Por isso, além da integração técnica, também é preciso tomar cuidado com a forma como o modelo consome esse recurso.

O caso de uso apresentado é muito interessante: um criador de conteúdo quer saber quais temas ou termos relacionados estão em alta, para produzir títulos mais atrativos e com maior potencial de alcance.

Essa é uma aplicação muito concreta de IA aplicada.

Estrutura geral do projeto

O projeto foi organizado de forma simples, com dois papéis principais no fluxo:

- um componente responsável por pesquisar
- outro responsável por responder ao usuário

Essa separação deixa o pipeline mais claro.

Na primeira etapa, a LLM atua como pesquisadora. Ela recebe a pergunta do usuário, interpreta o tema principal e extrai as palavras-chave que serão utilizadas na consulta ao Google Trends.

Na segunda etapa, ela utiliza os resultados dessa pesquisa para montar uma resposta final mais útil, orientada pelos dados retornados.

O papel do prompt nessa orquestração

Os prompts têm um papel central nesse projeto.

No nó de pesquisa, eu configuro a LLM para atuar como uma assistente voltada a criadores de vídeo. Ela precisa identificar o assunto principal da solicitação e extrair as keywords adequadas para uma única chamada à API.

Esse detalhe da chamada única é muito importante.

Como o serviço possui limite de uso, eu preciso impedir que o modelo entre em ciclos de exploração excessiva, repetindo consultas sem necessidade. Então, o prompt não serve apenas para orientar a resposta, mas também para impor limites operacionais e proteger o consumo da API.

Já no nó de resposta, a instrução muda. Agora, a LLM deve analisar os dados recebidos e devolver recomendações coerentes, preferencialmente no idioma do usuário.

A service como camada de negócio

A parte mais importante desta aula está na service que faz a comunicação com a API externa.

Essa service recebe as palavras-chave extraídas pela LLM, consulta a API do Google Trends e executa uma série de regras adicionais para transformar a resposta em algo mais útil.

Ela não apenas repassa os dados brutos. Ela agrega, interpreta e organiza informações como:

- termos relacionados
- tópicos em ascensão
- consultas associadas
- sinais de relevância

Esse ponto é fundamental porque mostra uma prática muito comum em projetos reais: a integração com APIs externas raramente é direta. Normalmente existe uma camada intermediária responsável por adaptar, filtrar e enriquecer os dados.

Expondo a service como tool

Depois que essa service está pronta, eu a transformo em uma tool.

Esse passo é conceitualmente simples, mas muito poderoso.

Eu defino:

- o nome da tool
- a descrição do que ela faz
- o schema de entrada
- e a função que, internamente, chama a service

Com isso, a LLM passa a enxergar essa capacidade como parte do conjunto de ferramentas disponíveis.

O mais importante é que, em nenhum momento, eu preciso chamar a service manualmente dentro dos nós do grafo. O modelo decide quando utilizá-la com base no contexto do prompt.

Isso reforça a ideia de autonomia controlada.

Diferença em relação ao fluxo tradicional

Em uma abordagem mais tradicional, eu provavelmente faria o seguinte:

- interpretaria a pergunta
- extrairia as keywords manualmente
- chamaria a API do Google Trends no código
- processaria os dados
- e só depois devolveria para a LLM formular a resposta

Aqui, o fluxo muda.

Eu continuo tendo a service para encapsular a regra de negócio, mas a decisão de usá-la sai do código imperativo e passa para o modelo. Isso reduz o acoplamento e deixa a aplicação mais flexível.

Controle de custos e limites

Um detalhe muito importante desta aula é o cuidado com o custo operacional.

Como a API utilizada possui limites no plano gratuito, eu preciso controlar quantas chamadas a LLM faz. Sem esse cuidado, o modelo pode insistir em múltiplas consultas, tentando refinar a resposta indefinidamente.

Por isso, a instrução de executar apenas uma chamada é essencial.

Esse tipo de preocupação é muito real em ambientes de produção. Não basta que a integração funcione. Ela precisa funcionar com previsibilidade, eficiência e respeito aos limites do ambiente.

Testes e comportamento observado

Ao executar o fluxo, a LLM consegue identificar palavras-chave relevantes a partir da pergunta do usuário. No exemplo apresentado, ela extrai termos ligados ao tema solicitado e consulta o serviço de tendências.

Com base nos dados recebidos, o sistema passa a sugerir títulos mais alinhados ao que está sendo buscado pelas pessoas.

O resultado é interessante porque mostra que a resposta não nasce apenas da criatividade do modelo. Ela é enriquecida por dados externos, o que aumenta o valor prático da recomendação.

Aplicação real em projetos autorais e empresariais

Um dos pontos mais valiosos desta aula é perceber que esse padrão pode ser reaproveitado em inúmeros cenários.

No exemplo mostrado, a motivação vem de um caso real de criação de conteúdo. A ideia é usar dados de tendência para melhorar títulos, descrições e posicionamento de vídeos.

Mas o mesmo raciocínio vale para muitos outros contextos, como:

- enriquecimento de cadastro de clientes
- busca de informações em serviços internos
- análise de mercado
- consulta de dados públicos
- integração com sistemas da empresa

Ou seja, qualquer service existente pode, potencialmente, ser transformada em uma tool acionável pela LLM.

Reaproveitamento da arquitetura já construída

Outro ponto importante é que eu não preciso reinventar o projeto para fazer isso.

A arquitetura já construída com LangChain.js continua válida. Eu apenas adiciono uma nova tool ao conjunto disponível e ajusto os prompts para orientar melhor o comportamento do modelo.

Isso mostra como aplicações baseadas em tools e agentes podem crescer de maneira incremental.

O valor dessa abordagem

O aprendizado mais importante desta aula está no padrão de pensamento que ela propõe.

Eu deixo de enxergar a LLM apenas como uma camada de geração de texto e passo a tratá-la como um orquestrador capaz de acionar capacidades da minha aplicação.

Essas capacidades podem vir de:

- tools pequenas e locais
- services internas da empresa
- APIs externas
- servidores MCP

O mecanismo muda pouco. O que muda é a origem da capacidade que eu exponho ao modelo.

Conclusão

Nesta aula, eu mostro que uma service tradicional pode ser promovida ao papel de tool dentro de uma aplicação com LangChain.js.

Essa abordagem permite integrar regras de negócio reais ao fluxo da LLM sem perder autonomia, flexibilidade ou clareza arquitetural. Em vez de depender apenas de MCPs prontos, eu passo a enxergar qualquer service útil como uma capacidade potencialmente acionável pelo modelo.

Esse é um passo muito importante para quem quer construir aplicações de IA de forma profissional, porque aproxima a arquitetura de cenários reais, nos quais a inteligência do sistema depende não só do modelo, mas também da qualidade dos serviços que ele consegue orquestrar.

NA PRÁTICA

Abra o(s) diretório(s) da Unidade 2 usando os links do GitHub exibidos no início desta aula. Localize os artefatos citados no conteúdo, reproduza o fluxo apresentado e observe o impacto em testes, estado, tracing, custo ou comportamento, conforme o tema da aula.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “A proposta da aula” no fluxo apresentado?
- Que decisão de engenharia está associada a “Por que usar uma service como tool” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório, nos testes ou no tracing para confirmar que o comportamento descrito na aula está funcionando?

RESUMO DA AULA

- A proposta da aula.
- Por que usar uma service como tool.
- O caso do Google Trends.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Revisão da Unidade 2

Use esta página como revisão rápida antes de avançar. Você deve conseguir relacionar as aulas da unidade em um único fluxo de engenharia.



Fluxo visual da Unidade 2.

CHECKLIST DA UNIDADE

- [Revisar a Aula 1: Conhecendo o Projeto: Template Inicial e Arquitetura](#)
- [Revisar a Aula 2: Parsing Inteligente: Organizando a Entrada do Cliente](#)
- [Revisar a Aula 3: Orquestração Autônoma com LangChain.js e MCP no MongoDB](#)
- [Revisar a Aula 4: Construindo Uma Tool Customizada no LangChain](#)
- [Revisar a Aula 5: Orquestração Autônoma com LangChain.js e MCP para Gerenciamento de Sistema de Arquivos](#)
- [Revisar a Aula 6: Usando Services Como Tools - Google Trends API com LangChain.js](#)

GITHUB DA UNIDADE

Repositório oficial da disciplina

[01-multiple-mcp-tools-template](#)

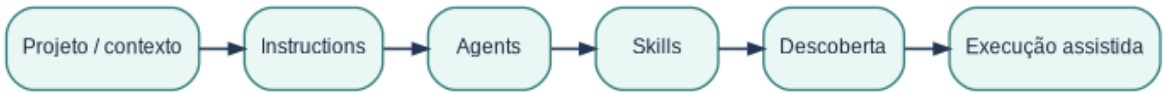
[01-multiple-mcp-tools-z](#)

[02-google-trends-agent](#)

Unidade 3

MCPs, Agents, Instructions e Skills no Desenvolvimento

Contexto persistente para agentes de código, arquivos de instructions, llms.txt, agents e skills como conhecimento modular, com comparação conceitual entre skills e MCP.



Fluxo visual da Unidade 3 — reorganização dos elementos descritos nas aulas.

GITHUB DA UNIDADE

Repositório oficial da disciplina

[03-dev-instructions-agents/.github/agents](#)

[04-skills](#)

Aula	Tema
1	Entendendo Agents e Instructions
2	Entendendo skills

Aula 1 — Entendendo Agents e Instructions

Unidade 3 • MCPs, Agents, Instructions e Skills no Desenvolvimento

OBJETIVOS DE APRENDIZAGEM

- Compreender o que muda com Vibe Coding.
- Relacionar arquivos de instrução: dando contexto ao projeto com estrutura e localização das instruções.
- Aplicar as decisões apresentadas no problema do tamanho e consumo de tokens e verificar o comportamento descrito na aula.

GITHUB DA UNIDADE

[03-dev-instructions-agents/.github/agents](#)

[04-skills](#)

CONCEITOS-CHAVE

- O que muda com Vibe Coding
- Arquivos de instrução: dando contexto ao projeto
- Estrutura e localização das instruções
- O problema do tamanho e consumo de tokens
- Geração automatizada de instruções
- O padrão llms.txt

Conteúdo da aula

Nesta aula, eu faço uma pausa estratégica nas integrações com APIs e MCPs para abordar um tema que impacta diretamente a forma como desenvolvemos com inteligência artificial no dia a dia: o uso de agents e arquivos de instrução.

A proposta aqui é mudar um pouco a perspectiva. Em vez de focar apenas em como integrar sistemas, eu passo a olhar para como orientar melhor a própria IA durante o desenvolvimento, especialmente no contexto do que eu chamo de Vibe Coding.

O que muda com Vibe Coding

No cenário atual, cada vez mais eu deixo de escrever código diretamente e passo a delegar tarefas para a IA. Isso significa que a qualidade do resultado depende muito mais de como eu instruo o modelo do que da minha própria digitação.

Nesse contexto, dois elementos se tornam fundamentais:

- instruções bem definidas sobre o projeto
- agents especializados para tarefas específicas

Sem isso, a IA tende a gerar código inconsistente, fora de padrão ou desalinhado com a arquitetura.

Arquivos de instrução: dando contexto ao projeto

O primeiro conceito importante é o de arquivos de instrução.

A ideia é simples: criar um documento que descreva o projeto de forma clara para a IA. Esse documento pode incluir:

- arquitetura do sistema
- responsabilidades de cada camada
- padrões de nomenclatura
- convenções de código
- decisões técnicas importantes

Esse arquivo funciona como um guia permanente para o modelo.

Sempre que eu pedir para gerar código, ele já terá uma base de referência sobre como o projeto deve ser estruturado.

Estrutura e localização das instruções

Normalmente, esse tipo de instrução é colocado em arquivos específicos dentro do projeto, muitas vezes em diretórios como:

- .github
- arquivos markdown dedicados
- arquivos de configuração do editor

Esses arquivos não são voltados para execução da aplicação, mas sim para orientar ferramentas de IA integradas ao ambiente de desenvolvimento.

O problema do tamanho e consumo de tokens

Um ponto crítico que eu observo é o tamanho desses arquivos.

Em projetos reais, esses documentos podem crescer bastante, chegando a milhares de palavras. Isso significa:

- maior consumo de tokens
- mais tempo de processamento
- maior risco de perda de foco da IA

Esse problema leva a uma necessidade de organização melhor dessas instruções, que será abordada com mais profundidade posteriormente.

Geração automatizada de instruções

Na prática, ninguém escreve tudo isso manualmente.

Uma abordagem comum é utilizar a própria IA para gerar esse arquivo com base no projeto. Existem ferramentas que:

- analisam a estrutura do código
- fazem perguntas sobre o projeto
- geram automaticamente um conjunto de regras

Isso acelera muito o processo e garante maior consistência.

O padrão llms.txt

Um conceito relacionado que surge nesse contexto é o llms.txt.

Esse arquivo funciona como uma forma de expor informações estruturadas para agentes de IA, especialmente em ambientes web.

A ideia é semelhante ao robots.txt, mas voltada para LLMs.

Em vez de obrigar a IA a navegar por páginas HTML, eu forneço um arquivo estruturado contendo:

- descrição do sistema
- links relevantes
- perguntas frequentes
- dados importantes

Isso reduz o consumo de recursos e melhora a qualidade das respostas.

Diferença entre instruções internas e llms.txt

É importante separar dois conceitos:

- arquivos de instrução do projeto, usados dentro do ambiente de desenvolvimento
- llms.txt, usado para expor informações externamente para agentes

Ambos seguem a mesma lógica: reduzir ambiguidade e melhorar a eficiência da IA.

Introdução aos agents

Depois de entender as instruções, eu entro no conceito de agents.

Um agent é, essencialmente, um prompt especializado com uma responsabilidade bem definida.

Em vez de ter um único prompt gigante que faz tudo, eu passo a dividir responsabilidades.

Por exemplo, posso ter agents para:

- desenvolvimento de código
- geração de testes
- revisão de código
- análise de requisitos
- integração com sistemas externos

Cada agent tem seu próprio conjunto de regras, contexto e ferramentas.

Criação de agents no ambiente de desenvolvimento

Ferramentas modernas, como editores de código, permitem criar agents diretamente no ambiente.

Ao criar um agent, eu defino:

- nome
- descrição
- papel
- instruções específicas
- acesso a ferramentas

Depois disso, posso selecionar esse agent e interagir com ele diretamente.

Isso muda completamente a experiência de desenvolvimento.

Exemplo prático de agent de desenvolvimento

No meu fluxo de trabalho, eu utilizo um agent focado em desenvolvimento.

Esse agent segue princípios como:

- uso de TypeScript e Node.js
- aplicação de boas práticas de arquitetura
- responsabilidade única
- imutabilidade
- uso de testes automatizados

Além disso, ele possui regras claras sobre quando uma tarefa pode ser considerada concluída.

Por exemplo:

- não pode haver erros de compilação
- os testes precisam passar
- alterações devem ser validadas antes de finalizar

Esse nível de controle aumenta muito a qualidade do código gerado.

Integração com tools e MCPs

Um agent também pode ter acesso a ferramentas específicas.

Por exemplo:

- leitura e escrita de arquivos
- execução de comandos
- acesso a documentação atualizada
- integração com MCPs

Isso permite que ele não apenas gere código, mas também execute ações no ambiente.

Divisão de responsabilidades entre agents

Um dos principais ganhos dessa abordagem é a divisão de responsabilidades.

Em vez de sobrecarregar um único prompt, eu posso ter:

- um agent para gerar código
- outro para gerar testes
- outro para validar qualidade
- outro para revisar mudanças

Cada um com foco claro e instruções específicas.

Isso melhora a precisão e reduz ambiguidades.

Redução da complexidade dos prompts

Ao dividir tarefas entre agents, eu também reduzo o tamanho dos prompts individuais.

Isso traz benefícios importantes:

- menor consumo de tokens
- respostas mais rápidas
- maior previsibilidade
- menos chance de erro

Essa estratégia é especialmente importante em projetos maiores.

Uso de agents no fluxo real de trabalho

No dia a dia, eu utilizo agents para automatizar tarefas completas.

Por exemplo:

- consultar tarefas em sistemas de gestão
- analisar comentários de revisão de código
- aplicar correções automaticamente
- validar mudanças antes de entrega

Isso transforma o papel do desenvolvedor.

Eu deixo de ser apenas quem escreve código e passo a ser um orquestrador de ferramentas e agentes.

Boas práticas no uso de agents

Alguns pontos que considero fundamentais:

- manter instruções claras e objetivas
- evitar prompts excessivamente grandes
- dividir responsabilidades sempre que possível

- validar resultados automaticamente
- controlar acesso a ferramentas externas

Essas práticas ajudam a manter o sistema previsível e confiável.

NA PRÁTICA

Abra o(s) diretório(s) da Unidade 3 usando os links do GitHub exibidos no início desta aula. Localize os artefatos citados no conteúdo, reproduza o fluxo apresentado e observe o impacto em testes, estado, tracing, custo ou comportamento, conforme o tema da aula.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “O que muda com Vibe Coding” no fluxo apresentado?
- Que decisão de engenharia está associada a “Arquivos de instrução: dando contexto ao projeto” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório, nos testes ou no tracing para confirmar que o comportamento descrito na aula está funcionando?

RESUMO DA AULA

- O que muda com Vibe Coding.
- Arquivos de instrução: dando contexto ao projeto.
- Estrutura e localização das instruções.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 2 — Entendendo skills

Unidade 3 • MCPs, Agents, Instructions e Skills no Desenvolvimento

OBJETIVOS DE APRENDIZAGEM

- Compreender o problema dos prompts grandes.
- Relacionar a ideia por trás dos Skills com skills como conhecimento modular.
- Aplicar as decisões apresentadas em estrutura de um Skill e verificar o comportamento descrito na aula.

GITHUB DA UNIDADE

[03-dev-instructions-agents/agents](https://github.com/03-dev-instructions-agents/agents)

[04-skills](#)

CONCEITOS-CHAVE

- O problema dos prompts grandes
- A ideia por trás dos Skills
- Skills como conhecimento modular
- Estrutura de um Skill
- Comparação com MCP
- Uso prático no desenvolvimento

Conteúdo da aula

Em software, sempre existiram múltiplas formas de resolver o mesmo problema. Quando entramos no mundo da inteligência artificial aplicada, isso se torna ainda mais evidente, especialmente quando começamos a trabalhar com prompts e o que hoje já chamamos de engenharia de prompts.

Nesta aula, eu apresento um padrão que vem ganhando bastante força justamente para lidar com uma das principais limitações dos modelos de linguagem: o limite de contexto e a dificuldade de manter qualidade quando o volume de informação cresce demais.

O problema dos prompts grandes

Ao longo do curso, já trabalhamos com prompts complexos, instruções detalhadas e até arquivos inteiros de contexto. No entanto, existe um ponto crítico:

Quanto maior o prompt, maior a chance da IA se perder.

Isso acontece porque:

- há limite de tokens
- o modelo não consegue priorizar bem todas as informações
- partes importantes podem ser ignoradas
- a resposta pode perder consistência

Esse cenário exige uma nova abordagem.

A ideia por trás dos Skills

Os Skills surgem como uma solução para esse problema.

Em vez de eu criar um único prompt gigantesco com tudo que o modelo precisa saber, eu passo a dividir esse conhecimento em unidades menores, especializadas e reutilizáveis.

Um Skill é, basicamente:

- um conjunto de instruções focadas em uma tarefa específica
- com exemplos, boas práticas e contexto técnico
- organizado em arquivos estruturados, geralmente em markdown

Essa abordagem lembra muito o que já fizemos com agents, mas com um foco ainda mais granular.

Skills como conhecimento modular

Cada Skill representa uma habilidade específica.

Por exemplo:

- como escrever queries eficientes em banco de dados
- como usar uma determinada biblioteca
- como interagir com APIs específicas
- como executar tarefas no sistema operacional

Em vez de ensinar tudo de uma vez, eu ensino apenas o necessário no momento certo.

Isso muda completamente a forma como a IA consome contexto.

Estrutura de um Skill

Um Skill não é apenas um texto simples.

Ele costuma conter:

- descrição da funcionalidade
- exemplos práticos
- templates reutilizáveis
- boas práticas
- referências para outros arquivos

Além disso, um Skill pode apontar para outros documentos, criando uma estrutura navegável de conhecimento.

Ou seja, em vez de carregar tudo na memória, a IA pode acessar o que precisa sob demanda.

Comparação com MCP

Existe uma semelhança interessante entre Skills e MCP.

Enquanto MCP organiza ferramentas e capacidades externas, os Skills organizam conhecimento.

Ambos seguem a mesma filosofia:

- evitar sobrecarga de contexto
- trabalhar sob demanda
- permitir descoberta incremental

A diferença é que:

- MCP lida com execução e integração
- Skills lidam com instrução e conhecimento

Uso prático no desenvolvimento

No dia a dia, isso traz um ganho enorme.

Suponha que eu esteja trabalhando com uma tecnologia específica, como um banco de dados que exige otimização de queries.

Em vez de confiar apenas no conhecimento pré-treinado do modelo, eu posso fornecer um Skill atualizado com:

- padrões de performance
- exemplos corretos
- armadilhas comuns
- recomendações específicas

Com isso, a qualidade do código gerado melhora significativamente.

Descoberta de Skills

Um dos pontos mais interessantes é a existência de repositórios de Skills.

Esses repositórios funcionam como um ecossistema onde eu posso:

- buscar habilidades prontas
- instalar no meu projeto
- reutilizar em diferentes contextos

Isso lembra muito o conceito de gerenciadores de pacotes, como o que já utilizamos no desenvolvimento tradicional.

Integração com o ambiente de desenvolvimento

Os Skills podem ser integrados diretamente ao editor ou ao projeto.

Quando isso acontece, o fluxo é o seguinte:

- eu faço uma solicitação
- o sistema identifica quais Skills são relevantes
- o modelo consulta esses Skills
- a resposta é construída com base nesse conhecimento

Diferente dos agents, eu não preciso selecionar manualmente qual Skill usar. O sistema faz essa escolha automaticamente.

Exemplo prático com ferramentas

Um dos exemplos mais interessantes é o uso de Skills para ferramentas técnicas.

Por exemplo, ao trabalhar com edição de vídeo:

- o modelo pode utilizar um Skill que ensina como usar uma ferramenta específica
- esse Skill contém comandos, exemplos e padrões

● o modelo aplica esse conhecimento diretamente na execução
Isso evita que ele tente “inventar” soluções e garanta mais precisão.

Aplicação em banco de dados

Outro exemplo relevante é o uso de Skills para banco de dados.

Com um Skill adequado, o modelo passa a:

- gerar queries mais eficientes
- respeitar boas práticas de performance
- considerar estrutura e normalização
- evitar erros comuns

Esse tipo de melhoria é extremamente valioso em ambientes reais.

Uso combinado com Agents

Skills e Agents não competem entre si. Eles se complementam.

Uma forma prática de pensar nisso é:

- Agents representam papéis
- Skills representam habilidades

Por exemplo:

- um agent desenvolvedor pode usar skills de banco de dados, testes e APIs
- um agent de QA pode usar skills de automação e validação
- um agent de produto pode usar skills de análise e priorização

Essa combinação cria um sistema muito mais flexível e poderoso.

Redução de complexidade

Ao adotar Skills, eu reduzo drasticamente a necessidade de prompts gigantes.

Em vez de centralizar tudo em um único lugar, eu distribuo conhecimento.

Isso traz benefícios claros:

- menor consumo de tokens
- respostas mais rápidas
- maior precisão
- melhor organização do conhecimento

Uso em projetos reais

Em projetos reais, eu posso criar Skills próprios da empresa.

Por exemplo:

- padrões internos de API
- regras de negócio específicas
- convenções de arquitetura
- fluxos operacionais

Esses Skills podem ser compartilhados entre equipes, garantindo consistência no desenvolvimento.

Integração com LangChain

Embora nesta aula o foco seja mais conceitual, é importante entender que Skills também podem ser utilizados dentro de aplicações com LangChain.

Na prática, eles podem ser tratados como:

- fontes de contexto adicionais
- documentos acessíveis via ferramentas
- referências que o modelo consulta dinamicamente

Isso abre espaço para arquiteturas ainda mais sofisticadas.

Evolução natural do uso de IA

O que fica claro aqui é que estamos evoluindo de:

Os Skills são um passo importante nessa evolução.

Conclusão

Nesta aula, eu apresento os Skills como uma solução prática para lidar com limitações de contexto e melhorar a qualidade das respostas da IA.

Ao fragmentar o conhecimento em unidades menores e especializadas, eu consigo construir sistemas mais eficientes, organizados e escaláveis.

Combinando Skills com Agents e MCPs, eu passo a ter uma arquitetura completa, onde:

- o modelo entende o contexto
- acessa conhecimento relevante
- utiliza ferramentas externas
- e executa tarefas com autonomia

Esse é o tipo de abordagem que realmente eleva o nível de aplicações com inteligência artificial no desenvolvimento de software.

NA PRÁTICA

Abra o(s) diretório(s) da Unidade 3 usando os links do GitHub exibidos no início desta aula. Localize os artefatos citados no conteúdo, reproduza o fluxo apresentado e observe o impacto em testes, estado, tracing, custo ou comportamento, conforme o tema da aula.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “O problema dos prompts grandes” no fluxo apresentado?
- Que decisão de engenharia está associada a “A ideia por trás dos Skills” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório, nos testes ou no tracing para confirmar que o comportamento descrito na aula está funcionando?

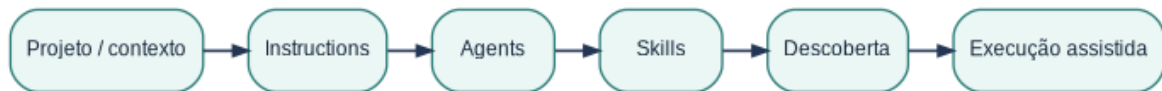
RESUMO DA AULA

- O problema dos prompts grandes.
- A ideia por trás dos Skills.
- Skills como conhecimento modular.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Revisão da Unidade 3

Use esta página como revisão rápida antes de avançar. Você deve conseguir relacionar as aulas da unidade em um único fluxo de engenharia.



Fluxo visual da Unidade 3.

CHECKLIST DA UNIDADE

- [Revisar a Aula 1: Entendendo Agents e Instructions](#)
- [Revisar a Aula 2: Entendendo skills](#)

GITHUB DA UNIDADE

[Repositório oficial da disciplina](#)

[03-dev-instructions-agents/.github/agents](#)

[04-skills](#)

Unidade 4

CypherSuite MCP: Desenvolvendo Servidores MCP do Zero

Implementação de um servidor MCP próprio com testes via MCP Client, definição de tools, resources e prompts, inspeção do servidor e consumo no VS Code.



Fluxo visual da Unidade 4 — reorganização dos elementos descritos nas aulas.

GITHUB DA UNIDADE

[Repositório oficial da disciplina](#)

[05-mcps-do-zero-template](#)

[05-mcps-do-zero-z](#)

Aula	Tema
1	Criando um MCP do Zero: Testes Automatizados Via MCP Client, Definindo Tools e Inspeccionando MCP Servers
2	Definindo Resources e Prompts em Servidores MCP + Usando Nosso Servidor MCP no VSCode

Aula 1 — Criando um MCP do Zero: Testes Automatizados Via MCP Client, Definindo Tools e Inspeccionando MCP Servers

Unidade 4 • CypherSuite MCP: Desenvolvendo Servidores MCP do Zero

OBJETIVOS DE APRENDIZAGEM

- Compreender um projeto pensado para focar no protocolo.
- Relacionar a preparação do ambiente com o primeiro passo: criar o servidor MCP.
- Aplicar as decisões apresentadas no transporte e a forma de conexão e verificar o comportamento descrito na aula.

GITHUB DA UNIDADE

[05-mcps-do-zero-template](#)

[05-mcps-do-zero-z](#)

CONCEITOS-CHAVE

- Um projeto pensado para focar no protocolo
- Preparação do ambiente
- O primeiro passo: criar o servidor MCP
- O transporte e a forma de conexão
- Construindo antes os testes
- Simulando um cliente MCP em teste

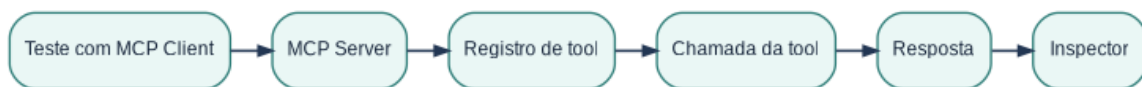


Diagrama de referência — fluxo descrito na aula.

Conteúdo da aula

Depois de entender como criar tools e expor funções para serem chamadas de forma autônoma por modelos de linguagem, eu chego agora a um ponto muito importante do módulo: a construção de um servidor MCP do zero. Nesta aula, eu saio do papel de consumidor de servidores MCP prontos e passo a construir a minha própria implementação, entendendo cada camada do processo e como ela se conecta com o restante da aplicação.

A proposta do projeto é simples, mas extremamente didática. Eu crio uma aplicação capaz de criptografar e descriptografar mensagens. O usuário envia uma mensagem e uma chave. O servidor MCP utiliza essa chave para criptografar o conteúdo. Depois, com a mesma chave, a mensagem pode ser revertida para o conteúdo original. O objetivo não é apenas trabalhar com criptografia, mas usar esse exemplo para mostrar, na prática, como definir tools, como estruturar um servidor MCP, como testá-lo e como inspecionar o que ele está expondo.

Um projeto pensado para focar no protocolo

Diferente de outras etapas do módulo, aqui eu preparo um template mais simples. A intenção é reduzir distrações e concentrar o aprendizado na construção do servidor MCP. A lógica de criptografia e descriptografia já está pronta em uma camada de serviço, porque o foco desta aula não é o algoritmo em si, mas a forma como eu exponho essas capacidades através do protocolo.

Isso é importante porque me permite trabalhar diretamente sobre os conceitos centrais do MCP:

- registro do servidor
- definição das tools
- validação de entrada e saída
- comunicação entre cliente e servidor
- testes automatizados
- inspeção do comportamento exposto

Preparação do ambiente

Antes de começar a construir o servidor, eu preparo o ambiente do projeto.

Garanto que a versão do Node.js esteja alinhada com o que o projeto precisa, restauro as dependências e verifico se há vulnerabilidades conhecidas nos pacotes instalados. Quando aparecem alertas de segurança, eu aproveito para reforçar uma prática importante: manter as dependências atualizadas faz parte da qualidade da aplicação, mesmo em um projeto de demonstração.

Também observo os scripts disponíveis no projeto, justamente para entender como vou iniciar o ambiente de desenvolvimento e executar os testes. Essa organização inicial ajuda a manter o fluxo mais previsível.

O primeiro passo: criar o servidor MCP

A primeira responsabilidade real desta etapa é criar o servidor MCP.

Eu inicio definindo um arquivo específico para o servidor e instancio a estrutura base do MCP com algumas informações fundamentais, como nome e versão. Esse ponto pode parecer simples, mas ele marca a transição entre uma aplicação comum e um serviço que passa a falar o protocolo MCP.

Ao fazer isso, eu estou dizendo que essa aplicação agora pode ser descoberta e consumida por clientes compatíveis, como frameworks, editores ou outras aplicações que saibam se comunicar nesse padrão.

O transporte e a forma de conexão

Depois de criar o servidor, eu preciso definir como ele será exposto.

Nesta aula, eu escolho o transporte via STDIO. Essa escolha tem um motivo claro: quero simular um cenário em que o MCP é executado localmente, como um pacote ou processo rodando na máquina do cliente.

Esse é o mesmo padrão que já vimos anteriormente em integrações locais. O servidor fica ativo, aguardando conexões, e o cliente se comunica com ele por entrada e saída padrão. É uma forma simples e muito eficiente de trabalhar quando o MCP não precisa ficar exposto como um serviço HTTP externo.

Ao usar STDIO, eu também reforço uma ideia importante: MCP não é sinônimo de API web pública. Ele pode ser apenas um processo bem estruturado rodando localmente e se comportando como um servidor de capacidades.

Construindo antes os testes

Uma das decisões mais importantes desta aula é começar pelos testes.

Em vez de sair registrando tools imediatamente, eu crio primeiro a estrutura de testes automatizados. Isso deixa claro que um servidor MCP não deve ser tratado como algo “mágico” ou difícil de validar. Ele continua sendo software, e software precisa ser testado.

Eu preparo uma suíte de testes voltada especificamente para as tools que vou expor. Inicialmente, quero validar duas operações:

- criptografar uma mensagem
- descriptografar uma mensagem

Esses testes servem como contrato de comportamento. Antes mesmo de implementar as tools, eu já deixo definido o que espero delas.

Simulando um cliente MCP em teste

Para que os testes façam sentido, eu preciso de um cliente capaz de se conectar ao servidor MCP durante a execução da suíte.

Então eu crio um helper responsável por iniciar esse cliente de teste. A ideia é simular exatamente o comportamento de um consumidor real do servidor. Esse cliente sobe um processo apontando para a aplicação que contém o MCP e se conecta a ele usando o mesmo transporte definido anteriormente.

Esse ponto é extremamente importante porque mostra que eu não estou testando apenas funções internas. Eu estou validando a integração real entre cliente e servidor, como ela aconteceria em produção.

Na prática, eu passo a ter duas partes distintas:

- o servidor MCP rodando em um processo separado
- o cliente de teste se conectando a ele e chamando as tools expostas

O primeiro teste falha de propósito

Com o cliente pronto, eu escrevo o primeiro teste de criptografia.

A lógica é simples: envio uma mensagem e uma chave, peço que o cliente chame uma tool específica e verifico se recebo uma mensagem criptografada de volta. Mas como a tool ainda não existe, a expectativa é justamente que esse teste falhe.

Essa falha é desejada. Ela confirma que o teste está cobrindo um comportamento real e que ainda falta implementar a tool correspondente.

Esse momento é didaticamente muito importante, porque reforça o ciclo correto de desenvolvimento:

- definir o comportamento esperado
- observar a falha inicial
- implementar a funcionalidade
- validar que o teste passa

Registrando a primeira tool

Com o teste falhando, eu volto ao servidor MCP e registro a primeira tool: a responsável por criptografar a mensagem.

Aqui aparece uma das partes mais valiosas do MCP. Ao registrar uma tool, eu não entrego apenas uma função executável. Eu também descrevo formalmente essa capacidade.

Eu defino:

- o nome da tool
- a descrição do que ela faz
- o schema de entrada
- o schema de saída
- a implementação da lógica

Essa combinação é o que torna a integração muito mais rica do que uma simples function calling informal. O consumidor do servidor não recebe apenas um nome de função, mas também uma descrição bem estruturada de como usá-la.

A importância da validação com schema

Para registrar a tool, eu utilizo schemas de validação.

Isso garante que o cliente só consiga chamar a funcionalidade se enviar exatamente os dados esperados. No caso da criptografia, são necessários dois campos:

- a mensagem
- a chave de criptografia

Também defino a estrutura de saída, informando que a resposta conterá a mensagem criptografada.

Essa validação tem dois papéis muito importantes. O primeiro é técnico: impedir chamadas inválidas. O segundo é semântico: ajudar a IA ou qualquer consumidor a entender como interagir corretamente com a tool.

Tratamento de sucesso e erro

Outro aspecto importante da implementação é o tratamento explícito de erro.

Quando a operação falha, eu retorno uma resposta indicando que houve erro e envio um conteúdo textual explicando o problema. Em uma aplicação real, eu evitaria expor tantos detalhes internos ao cliente final, mas aqui isso é útil para fins didáticos e de depuração.

Quando a operação funciona, eu retorno dois formatos de conteúdo:

- uma representação textual
- uma estrutura JSON organizada

Isso é relevante porque mostra que o MCP pode atender diferentes tipos de consumo. Alguns clientes podem querer apenas texto; outros podem preferir dados estruturados para continuar processando automaticamente.

A tool de descriptografia

Depois de concluir a tool de criptografia, eu repito o mesmo processo para a descriptografia.

O padrão é praticamente o mesmo, mas agora a entrada passa a exigir:

- a mensagem criptografada
- a mesma chave usada no processo original

Na saída, o que espero receber é a mensagem descriptografada.

Esse espelhamento entre as duas tools é útil porque ajuda a consolidar o padrão de implementação. Depois que entendo uma, a outra se torna apenas uma variação de comportamento.

Testes autossuficientes

Na construção dos testes, eu reforço uma prática importante: cada teste precisa ser autossuficiente.

Isso significa que o teste de descriptografia não deve depender do resultado do teste anterior. Mesmo que o fluxo lógico envolva primeiro criptografar e depois descriptografar, o teste deve produzir seu próprio cenário completo.

Essa abordagem evita o acoplamento entre testes e melhora muito a confiabilidade da suíte. Cada caso valida o que precisa validar sem pressupor que outro já passou antes.

O valor de pensar MCP como software normal

Um aprendizado importante desta aula é perceber que um servidor MCP continua sendo software como qualquer outro.

Eu continuo precisando de:

- organização de arquivos
- separação de responsabilidades
- testes automatizados

- validação de dados
- tratamento de falhas

O que muda é o protocolo de comunicação e a forma como exponho as capacidades.

Essa percepção é importante porque tira o MCP de um campo abstrato e o coloca no terreno da engenharia de software tradicional. Isso facilita tanto o desenvolvimento quanto a manutenção.

Inspecionando o servidor MCP

Depois de registrar as tools e validar os testes, eu parto para um passo muito útil: inspecionar o servidor MCP.

Ao conectar uma ferramenta de inspeção ao serviço, eu consigo visualizar o que está sendo exposto. Nesse momento, fica muito claro que o MCP vai além de tools. A interface mostra que existem outras áreas possíveis, como resources e prompts, mesmo que ainda não tenham sido definidas nesta etapa.

Por enquanto, o que está ativo são as tools. E isso já permite explorar o servidor manualmente, chamando as funções, preenchendo parâmetros e observando as respostas.

Esse tipo de inspeção é excelente para depuração, documentação e validação rápida do comportamento do serviço.

Testando os cenários manualmente

Na inspeção manual, eu consigo testar os dois fluxos:

- criptografar uma mensagem com uma chave
- descriptografar usando a mesma chave

Quando utilizo a chave correta, o resultado volta ao texto original. Quando uso uma chave incorreta, o erro é retornado conforme o comportamento que defini no servidor.

Também posso testar conteúdos inválidos e observar como o MCP responde. Isso é importante porque mostra, de forma concreta, que os schemas e o tratamento de erro estão funcionando como esperado.

Tools são só uma parte do MCP

Ao final da aula, eu reforço uma ideia que atravessa todo o módulo: tools são apenas uma parte do padrão MCP.

É muito comum que as pessoas confundam os dois conceitos, porque no uso do dia a dia a interação mais visível acontece justamente por meio das tools. Mas o protocolo é mais amplo. Ele também permite expor resources, prompts e outras formas de descoberta e organização de contexto.

Nesta etapa, eu trabalho apenas com tools porque quero consolidar o básico primeiro. Mas a própria inspeção já mostra que existe um espaço maior para evoluir o servidor.

NA PRÁTICA

Abra o(s) diretório(s) da Unidade 4 usando os links do GitHub exibidos no início desta aula. Localize os artefatos citados no conteúdo, reproduza o fluxo apresentado e observe o impacto em testes, estado, tracing, custo ou comportamento, conforme o tema da aula.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Um projeto pensado para focar no protocolo” no fluxo apresentado?
- Que decisão de engenharia está associada a “Preparação do ambiente” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório, nos testes ou no tracing para confirmar que o comportamento descrito na aula está funcionando?

RESUMO DA AULA

- Um projeto pensado para focar no protocolo.
- Preparação do ambiente.
- O primeiro passo: criar o servidor MCP.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 2 — Definindo Resources e Prompts em Servidores MCP + Usando Nosso Servidor MCP no VSCode

Unidade 4 • CypherSuite MCP: Desenvolvendo Servidores MCP do Zero

OBJETIVOS DE APRENDIZAGEM

- Compreender completando a visão do protocolo.
- Relacionar começando por testes automatizados com registrando o primeiro resource.
- Aplicar as decisões apresentadas em o valor dos resources no uso real e verificar o comportamento descrito na aula.

GITHUB DA UNIDADE

[05-mcps-do-zero-template](#)

[05-mcps-do-zero-z](#)

CONCEITOS-CHAVE

- Completando a visão do protocolo
- Começando por testes automatizados
- Registrando o primeiro resource
- O valor dos resources no uso real
- Validando o resource com o Inspector
- Sobre resource templates

Conteúdo da aula

Na aula anterior, eu mostrei como registrar tools dentro de um servidor MCP e como essas tools podem ser chamadas tanto por um cliente automatizado quanto por ferramentas de inspeção. Agora eu avanço para completar a implementação do protocolo, adicionando mais duas peças fundamentais: resources e prompts. E, para fechar o ciclo com uma demonstração realmente prática, eu também conecto o nosso servidor MCP diretamente ao VSCode, para mostrar como ele pode ser usado no fluxo real de desenvolvimento.

Até aqui, o nosso servidor já consegue criptografar e descriptografar mensagens. Isso por si só já é útil, mas ainda está incompleto do ponto de vista do protocolo. Um servidor MCP não se resume a tools. Ele pode, e em muitos casos deve, expor também recursos descritivos e prompts prontos, que ajudam tanto o modelo quanto o usuário a entender melhor o que aquele serviço faz e como ele deve ser usado.

Completando a visão do protocolo

Quando eu trabalho apenas com tools, o cliente consegue descobrir quais ações estão disponíveis e quais parâmetros cada uma espera. Mas ainda falta algo importante: contexto.

Esse contexto pode ser oferecido de duas formas complementares:

- por meio de resources, que funcionam como documentos ou fontes de informação que descrevem aspectos do servidor
- por meio de prompts, que funcionam como templates prontos de instrução para orientar o uso das tools

Essa distinção é importante.

As tools dizem o que o servidor consegue fazer.

Os resources ajudam a explicar o serviço.

Os prompts ajudam a mostrar como pedir ao modelo que use esse serviço.

Quando esses três elementos trabalham juntos, o servidor MCP deixa de ser apenas um conjunto de funções e passa a ser uma interface muito mais completa de capacidades.

Começando por testes automatizados

Como venho fazendo ao longo de todo o projeto, eu começo novamente pelos testes.

Antes de registrar qualquer novo recurso no servidor, eu escrevo um caso de teste para validar que determinado resource será listado corretamente. A ideia aqui é simples: se quero expor uma informação institucional sobre o meu servidor de criptografia, primeiro preciso garantir, por teste, que esse recurso realmente será publicado.

Esse resource inicial é um tipo de documento informativo. Ele vai explicar o que o servidor faz, qual algoritmo está sendo usado, quais são os requisitos da chave e que tipo de saída o cliente deve esperar.

Na prática, é como se eu estivesse criando uma página de documentação embutida dentro do próprio MCP.

O teste faz a listagem dos recursos disponíveis, procura por uma URI específica e valida que esse item realmente existe. Como o resource ainda não foi implementado, o teste falha. E essa falha é exatamente o comportamento esperado, porque ela marca o ponto de partida da implementação.

Registrando o primeiro resource

Com o teste falhando, eu volto para o servidor e faço o registro do resource.

Esse registro exige alguns elementos bem claros:

- um nome para o resource
- uma URI que o identifica
- uma descrição
- uma função que devolve o conteúdo

O ponto mais importante aqui é entender que o resource não é uma tool. Ele não executa uma ação. Ele retorna contexto.

No caso desta aula, eu crio um resource de informação sobre a criptografia. Esse recurso explica, em linguagem objetiva, qual é a lógica usada pelo servidor, como a chave deve ser tratada e qual estrutura de saída o cliente pode esperar. Em vez de obrigar o consumidor a procurar essa informação fora do sistema, eu a incorporo ao próprio servidor MCP.

Isso é extremamente útil quando o cliente é um modelo de linguagem, porque o modelo pode acessar esse conteúdo e usar essa informação para tomar decisões melhores antes de chamar uma tool.

O valor dos resources no uso real

Muita gente tende a enxergar resources como algo secundário, mas eles têm um papel muito importante.

Quando eu exponho resources, eu permito que o cliente:

- descubra documentação sob demanda
- entenda limitações do serviço
- recupere contexto sem depender de prompts gigantes
- use o próprio servidor como fonte de referência

Isso ajuda especialmente em cenários onde o modelo precisa decidir se deve ou não usar uma determinada tool, ou quando ele precisa compreender melhor o formato esperado de entrada e saída.

No nosso caso, o resource de criptografia funciona quase como uma nota técnica do servidor. Ele explica o suficiente para qualquer consumidor entender a finalidade daquele MCP.

Validando o resource com o Inspector

Depois de registrar o resource, eu executo novamente os testes e confirmo que ele agora está sendo listado corretamente. Em seguida, eu uso a ferramenta de inspeção do MCP para verificar visualmente o resultado.

A diferença é imediata.

Antes, o inspector exibia apenas as tools. Agora, uma nova aba aparece com os resources disponíveis. Ao abrir esse recurso, eu consigo ver o conteúdo textual sendo retornado, exatamente como foi definido no servidor.

Esse momento é importante porque deixa evidente que o protocolo está ficando mais rico. Já não se trata apenas de ações. Existe agora também uma camada de descrição embutida no servidor.

Sobre resource templates

Durante a demonstração, aparece também a diferença entre resources comuns e resource templates.

Resources comuns são estáticos. Eles têm um endereço conhecido e retornam sempre um conteúdo específico.

Já resource templates servem para cenários dinâmicos, em que eu posso construir uma URI parametrizada, por exemplo com identificadores ou outros valores variáveis.

Nesta aula, eu não exploro esse segundo caso porque o projeto não exige isso. O nosso resource é fixo e suficiente para demonstrar o conceito. Mas a simples presença dessa categoria no protocolo já mostra que o MCP foi desenhado para lidar também com contextos mais dinâmicos.

Avançando para os prompts

Depois de concluir a parte de resources, eu passo para o terceiro bloco do protocolo: os prompts.

Aqui a ideia muda um pouco.

Enquanto o resource descreve o serviço, o prompt descreve uma forma sugerida de usar o serviço. Ele funciona como um template pronto que pode ser exibido em ferramentas compatíveis, ajudando o usuário ou a IA a iniciar uma conversa correta com o MCP.

No nosso exemplo, eu crio um prompt voltado para a criptografia de mensagens.

Esse prompt recebe parâmetros como:

- a mensagem a ser criptografada
- a chave de criptografia

E, com base nisso, gera um texto que orienta a chamada da tool correspondente.

Criando o prompt a partir de testes

Mais uma vez, eu começo pelo teste.

A lógica é parecida com a adotada para resources. Eu preparo uma chamada ao cliente MCP para obter um prompt específico, envio os argumentos necessários e verifico se o conteúdo retornado contém exatamente o texto esperado.

Como o prompt ainda não existe, o teste falha.

A partir daí, implemento o registro do prompt no servidor.

Registrando um prompt no servidor MCP

O registro de prompt exige:

- um nome
- uma descrição
- um schema de argumentos
- uma função que monta a mensagem

Esse ponto é especialmente interessante porque aproxima bastante o MCP da experiência de uso em editores e clientes visuais.

Ao definir o schema de argumentos, eu deixo claro quais dados o usuário precisa preencher para gerar aquele prompt. No caso do nosso servidor, os campos obrigatórios são justamente a mensagem e a chave de criptografia.

Depois disso, eu implemento a função que constrói a mensagem final.

Essa mensagem é formatada como uma instrução pronta, com papel de usuário e conteúdo textual explícito, orientando o modelo a usar a tool de criptografia com os valores informados.

Na prática, o prompt vira um atalho operacional.

Por que prompts são tão úteis

A utilidade dos prompts fica muito clara quando penso na experiência de quem vai consumir o servidor.

Sem prompts, a pessoa precisa adivinhar como pedir para o modelo usar corretamente o MCP. Com prompts, eu posso oferecer exemplos prontos e guiados.

Isso traz várias vantagens:

- reduz erros de uso
- facilita a descoberta das capacidades do servidor
- ajuda modelos menos robustos a seguirem o caminho certo
- melhora a experiência em editores e interfaces gráficas

Em outras palavras, o prompt não substitui a tool, mas melhora muito a forma como ela é acionada.

Inspecionando os prompts no MCP Inspector

Depois da implementação, eu volto ao inspector e agora vejo uma nova seção: prompts.

Ao abrir essa seção, encontro o prompt registrado e posso preenchê-lo diretamente com valores de exemplo. O inspector então monta o texto final com base nesses argumentos.

Esse comportamento mostra, na prática, que o servidor não está só expondo capacidades de execução. Ele também está oferecendo uma camada de usabilidade sobre essas capacidades.

Isso torna o MCP mais amigável tanto para humanos quanto para clientes automatizados.

Levando o nosso MCP para o VSCode

Depois de definir tools, resources e prompts, eu faço a parte mais empolgante da aula: plugar o nosso servidor MCP diretamente no VSCode.

Essa etapa é importante porque ela tira o projeto do ambiente de teste e leva para o fluxo de trabalho real. Em vez de usar apenas testes automatizados e ferramentas de inspeção, eu passo a usar o meu próprio servidor como parte do editor.

Para isso, eu crio um arquivo de configuração dentro da pasta.vscode, definindo os servidores MCP que o editor deve carregar ao abrir o projeto.

Nessa configuração, eu informo:

- o nome do servidor
- o comando que deve ser executado
- os argumentos necessários para iniciar o processo

Na prática, eu faço no editor a mesma coisa que já havia feito nos testes com o cliente MCP. A diferença é que, agora, o consumidor não é uma suíte automatizada, e sim o próprio VSCode.

O editor como cliente MCP

Assim que salvo a configuração, o editor detecta o novo servidor.

A partir daí, ele passa a:

- iniciar o processo localmente
- descobrir as tools expostas
- carregar os prompts disponíveis
- permitir o uso interativo dessas capacidades

Esse é um dos momentos mais poderosos de toda a aula, porque fica evidente que o MCP não é uma abstração teórica. Ele realmente se conecta ao ambiente de desenvolvimento e passa a funcionar como parte dele.

Usando o prompt dentro do editor

Com o servidor carregado no VSCode, eu consigo abrir a interface de prompts e selecionar o prompt que acabei de criar.

O editor então me pede os parâmetros definidos no schema:

- a mensagem
- a chave

Depois de preencher esses valores, ele monta automaticamente o texto final do prompt. E, quando eu envio essa instrução, o editor usa o servidor MCP para chamar a tool correspondente.

Ou seja, o fluxo completo funciona assim:

- eu escolho o prompt
- preencho os parâmetros
- o prompt gera a instrução correta
- o modelo usa o MCP
- a tool é chamada
- a resposta volta para o editor

Isso mostra com muita clareza o papel complementar de cada parte do protocolo.

Testando o fluxo completo de criptografia e descriptografia

Na sequência, eu uso esse mesmo contexto no editor para pedir a descriptografia da mensagem gerada.

Como o histórico já contém os dados relevantes, o modelo consegue reutilizar a informação e chamar a tool adequada. O resultado volta corretamente, mostrando a mensagem original descriptografada.

Esse teste é simples, mas muito significativo. Ele comprova que o nosso MCP está pronto para uso real:

- possui tools funcionais
- possui resource descritivo
- possui prompt reutilizável
- pode ser conectado ao editor
- responde corretamente ao fluxo completo

O que essa aula deixa claro sobre o MCP

Ao final desta etapa, a estrutura do protocolo fica muito mais fácil de entender.

As tools representam ações.

Os resources representam contexto e documentação.

Os prompts representam formas guiadas de usar essas ações.

Quando tudo isso é combinado, o servidor MCP deixa de ser apenas uma camada técnica de integração e passa a ser uma interface completa de capacidades para clientes inteligentes.

Esse é o ponto em que o MCP realmente se diferencia de abordagens mais simples, como uma lista isolada de funções.

NA PRÁTICA

Abra o(s) diretório(s) da Unidade 4 usando os links do GitHub exibidos no início desta aula. Localize os artefatos citados no conteúdo, reproduza o fluxo apresentado e observe o impacto em testes, estado, tracing, custo ou comportamento, conforme o tema da aula.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Completando a visão do protocolo” no fluxo apresentado?
- Que decisão de engenharia está associada a “Começando por testes automatizados” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório, nos testes ou no tracing para confirmar que o comportamento descrito na aula está funcionando?

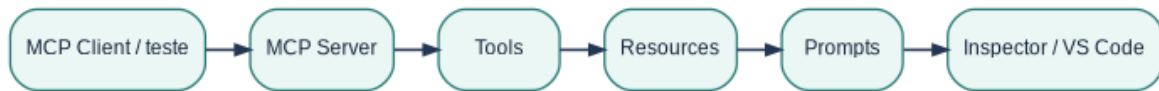
RESUMO DA AULA

- Completando a visão do protocolo.
- Começando por testes automatizados.
- Registrando o primeiro resource.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Revisão da Unidade 4

Use esta página como revisão rápida antes de avançar. Você deve conseguir relacionar as aulas da unidade em um único fluxo de engenharia.



Fluxo visual da Unidade 4.

CHECKLIST DA UNIDADE

- [Revisar a Aula 1: Criando um MCP do Zero: Testes Automatizados Via MCP Client, Definindo Tools e Inspeccionando MCP Servers](#)
- [Revisar a Aula 2: Definindo Resources e Prompts em Servidores MCP + Usando Nosso Servidor MCP no VSCode](#)

GITHUB DA UNIDADE

[Repositório oficial da disciplina](#)

[05-mcps-do-zero-template](#)

[05-mcps-do-zero-z](#)

Unidade 5

Transformando Sistemas Legados em Servidores MCP

Uso do MCP como camada de adaptação sobre uma API existente, com modelagem orientada ao domínio, HTTP Client, camada de serviço e tools para operações de clientes.



Fluxo visual da Unidade 5 — reorganização dos elementos descritos nas aulas.

GITHUB DA UNIDADE

[Repositório oficial da disciplina](#)

[06-your-legacy-api-as-mcp](#)

Aula	Tema
1	Template Inicial e Arquitetura + Boas Práticas de Organização de Código e Estrutura de Projeto
2	Como Empresas Usam MCPs para Conectar IA a Sistemas Legados
3	Criando Tools de Listagem e Criação de Clientes
4	Criando Tools de Update e Delete de Clientes + Usando no VSCode

Aula 1 — Template Inicial e Arquitetura + Boas Práticas de Organização de Código e Estrutura de Projeto

Unidade 5 • Transformando Sistemas Legados em Servidores MCP

OBJETIVOS DE APRENDIZAGEM

- Compreender o objetivo do projeto.
- Relacionar MCP não é espelhamento de endpoint com visão orientada ao cliente final.
- Aplicar as decisões apresentadas em segurança desde a concepção e verificar o comportamento descrito na aula.

GITHUB DA UNIDADE

[06-your-legacy-api-as-mcp](#)

CONCEITOS-CHAVE

- O objetivo do projeto
- MCP não é espelhamento de endpoint
- Visão orientada ao cliente final
- Segurança desde a concepção
- Estrutura inicial do projeto
- Separação entre API legada e camada MCP

Conteúdo da aula

Neste módulo, eu passo a trabalhar com um cenário extremamente relevante para quem já atua em empresas ou quer construir produtos próprios: como transformar APIs públicas ou legadas em servidores MCP bem estruturados, seguros e realmente úteis para consumo por modelos de linguagem.

A proposta aqui vai além de simplesmente expor endpoints existentes. O foco é construir uma camada de abstração bem planejada, organizada e aderente às boas práticas de arquitetura, para que o MCP represente ações de negócio de forma clara e eficiente.

O objetivo do projeto

Para este módulo, eu preparo um cenário bastante realista. Em vez de começar do zero com uma aplicação nova, eu utilizo uma API legada que já existia anteriormente. Essa API foi construída para operações básicas de CRUD, ou seja:

- criação
- leitura
- atualização
- remoção

Esse tipo de aplicação é muito comum no mercado. Quase toda empresa tem algum serviço parecido, seja para clientes, alunos, produtos ou qualquer outra entidade de negócio.

O que eu quero mostrar é como pegar esse tipo de sistema e transformá-lo em algo mais adequado ao mundo de AI aplicada.

MCP não é espelhamento de endpoint

Um ponto central desta aula é o cuidado para não tratar MCP como uma cópia direta da API existente.

Se eu tenho uma API com vários endpoints, não significa que eu devo criar uma tool para cada rota e reproduzir exatamente a mesma estrutura. Essa abordagem até funcionaria tecnicamente, mas desperdiçaria grande parte do valor do protocolo.

A ideia do MCP é abstrair.

Ou seja, em vez de expor detalhes internos da API, eu devo expor ações que façam sentido para quem vai consumir o serviço. Isso significa que uma única ação do MCP pode, internamente, chamar vários endpoints diferentes, combinar respostas, aplicar regras de negócio e esconder complexidades técnicas.

Esse é um dos pontos mais importantes de toda a arquitetura.

Visão orientada ao cliente final

Pensando no cliente final, o que importa não é saber se existem três ou quatro rotas por trás da operação. O que importa é conseguir pedir algo como:

- buscar um cliente por nome
- obter uma visão consolidada de dados
- executar uma ação de negócio específica

Se a API original não oferece isso diretamente, o MCP pode oferecer.

Esse tipo de modelagem é o que diferencia uma integração apenas funcional de uma integração bem desenhada.

Segurança desde a concepção

Além da abstração, eu também antecipo um segundo tema muito importante: segurança.

A API legada utilizada neste projeto ainda não possui autenticação nem autorização adequadas. As rotas estão abertas, o que é aceitável em um cenário inicial de demonstração, mas não em uma aplicação real.

Por isso, já deixo claro que a evolução do projeto incluirá:

- autenticação
- autorização baseada em papéis
- controle de permissões por cliente ou perfil

Essa preocupação é essencial porque, ao publicar um MCP, eu estou criando uma camada de acesso que pode ser consumida por outras pessoas, sistemas ou modelos de linguagem. Isso exige muito cuidado com governança.

Estrutura inicial do projeto

Para facilitar o desenvolvimento, eu já deixo preparado um template específico para servidores MCP.

Esse template não vem com toda a implementação pronta, mas já traz a base de organização que eu costumo utilizar nos meus próprios projetos. A ideia é fornecer uma estrutura coerente de diretórios e responsabilidades, permitindo que o desenvolvimento siga de forma mais limpa e previsível.

Essa organização inicial ajuda bastante porque evita que o projeto cresça de forma desordenada.

Separação entre API legada e camada MCP

Um ponto arquitetural importante é que eu mantenho a API legada e o template MCP como partes separadas.

Isso é extremamente saudável do ponto de vista de arquitetura, porque a camada MCP passa a funcionar como uma interface de adaptação, e não como uma modificação direta da aplicação original.

Na prática, eu preservoo:

- a API legada como sistema de origem
- o MCP como camada de tradução e abstração

Essa separação facilita a manutenção, testes e evolução futura.

Preparação do ambiente

Antes de começar a implementar qualquer funcionalidade, eu subo o ambiente da aplicação legada.

Esse ambiente inclui:

- a API em execução
- o banco de dados necessário
- a infraestrutura via Docker

Com isso, eu consigo validar que o sistema original está funcionando corretamente antes de construir qualquer camada sobre ele.

Essa validação inicial é fundamental. Não faz sentido criar uma abstração sobre uma base que ainda não foi confirmada como estável.

Testando a aplicação legada

Depois de subir o ambiente, eu executo chamadas básicas para verificar o funcionamento da API.

Faço testes de listagem e criação de registros para garantir que:

- a aplicação está respondendo
- os dados estão sendo persistidos
- a comunicação com o banco está correta

Esse passo, embora simples, é muito importante para evitar que problemas de infraestrutura sejam confundidos com erros na camada MCP.

Preparando o template MCP

Com a API funcionando, eu passo a trabalhar no template MCP.

Esse template já contém:

- a estrutura base do servidor
- a configuração inicial
- uma URL base apontando para a API original
- uma organização de pastas pronta para evolução

Aqui, a proposta não é complicar com variáveis de ambiente ou múltiplas abstrações logo de início. O foco desta aula é estabelecer a base arquitetural e preparar o terreno para as implementações seguintes.

Organização de código como decisão arquitetural

Um dos temas centrais desta aula é a organização do código.

Mais do que escolher nomes de pastas, estou tomando decisões sobre como o projeto vai crescer. Uma boa estrutura evita acoplamento excessivo, facilita testes e melhora a clareza da aplicação.

A ideia de dividir o projeto em camadas e pastas específicas ajuda a separar responsabilidades, por exemplo:

- servidor MCP
- integrações com APIs externas
- regras de negócio
- testes
- utilitários e suporte

Essa divisão não precisa ser idêntica em todos os projetos, mas precisa fazer sentido e ser consistente.

Template como ponto de partida, não como prisão

Também é importante notar que o template não deve ser visto como uma regra rígida.

Ele serve como ponto de partida, com uma estrutura que eu considero saudável e prática, mas cada projeto pode evoluir de forma diferente dependendo do contexto. O mais importante é manter coerência arquitetural ao longo do desenvolvimento.

Ou seja, não se trata de decorar uma estrutura fixa, mas de entender por que ela existe.

Inicialização e inspeção do servidor MCP

Depois de restaurar as dependências do template MCP, eu inicializo o servidor e faço uma primeira inspeção.

Nesse momento, o servidor ainda não expõe tools, resources ou prompts. Ele apenas está de pé, com a estrutura mínima pronta para ser evoluída.

Essa inspeção inicial é útil justamente para confirmar que o ambiente está funcionando e que o ponto de partida está limpo.

Uso de configuração local para integração com o editor

Outro detalhe importante é a presença da configuração para uso do MCP localmente no editor.

Isso prepara o terreno para etapas futuras, quando o servidor já estiver com funcionalidades expostas e poderá ser consumido diretamente a partir do ambiente de desenvolvimento.

Mesmo que ainda não exista nada funcional para o editor utilizar, deixar essa configuração pronta desde o início ajuda a consolidar o fluxo de trabalho.

Boas práticas reforçadas nesta etapa

Esta aula reforça várias boas práticas importantes para projetos de MCP:

A primeira é não expor a API de forma ingênua. O MCP deve abstrair o domínio, e não apenas repetir endpoints.

A segunda é separar bem as camadas. A API original continua sendo uma coisa; o MCP é outra.

A terceira é validar a infraestrutura antes de construir por cima dela.

A quarta é começar com uma estrutura organizada, mesmo antes da implementação completa.

A quinta é pensar em segurança desde cedo, mesmo que ela ainda não esteja implementada.

O que essa arquitetura prepara para as próximas aulas

Ao final desta etapa, eu deixo pronto o cenário ideal para evoluir o projeto.

Agora eu tenho:

- uma API legada funcional
- uma camada MCP separada
- uma estrutura de projeto organizada
- um template preparado para testes e crescimento
- um servidor inicial pronto para começar a expor capacidades

A partir daqui, posso começar a implementar tools, autenticação, autorização e abstrações mais inteligentes sobre a API original.

Conclusão

Nesta aula, eu estabeleço a base arquitetural de um projeto muito mais próximo da realidade de mercado.

Em vez de criar um exemplo isolado, eu parto de uma aplicação legada e preparo uma camada MCP organizada, pensada para abstrair operações, melhorar a experiência de consumo e permitir evolução segura ao longo do tempo.

Mais do que iniciar um novo projeto, eu começo a mostrar como transformar software existente em uma interface moderna para inteligência artificial, sem cair na armadilha de simplesmente espelhar endpoints.

Essa visão de arquitetura, organização e responsabilidade de código é o que sustenta um servidor MCP realmente útil, publicável e sustentável no longo prazo.

NA PRÁTICA

Abra o(s) diretório(s) da Unidade 5 usando os links do GitHub exibidos no início desta aula. Localize os artefatos citados no conteúdo, reproduza o fluxo apresentado e observe o impacto em testes, estado, tracing, custo ou comportamento, conforme o tema da aula.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “O objetivo do projeto” no fluxo apresentado?
- Que decisão de engenharia está associada a “MCP não é espelhamento de endpoint” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório, nos testes ou no tracing para confirmar que o comportamento descrito na aula está funcionando?

RESUMO DA AULA

- O objetivo do projeto.
- MCP não é espelhamento de endpoint.
- Visão orientada ao cliente final.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 2 — Como Empresas Usam MCPs para Conectar IA a Sistemas Legados

Unidade 5 • Transformando Sistemas Legados em Servidores MCP

OBJETIVOS DE APRENDIZAGEM

- Compreender pensando como uma empresa desenha APIs.
- Relacionar por que não fazer um mapeamento direto com MCP como camada de proxy inteligente.
- Aplicar as decisões apresentadas em modelando o domínio e verificar o comportamento descrito na aula.

GITHUB DA UNIDADE

[06-your-legacy-api-as-mcp](#)

CONCEITOS-CHAVE

- Pensando como uma empresa desenha APIs
- Por que não fazer um mapeamento direto
- MCP como camada de proxy inteligente
- Modelando o domínio
- Criando o HTTP Client
- Separação de responsabilidades

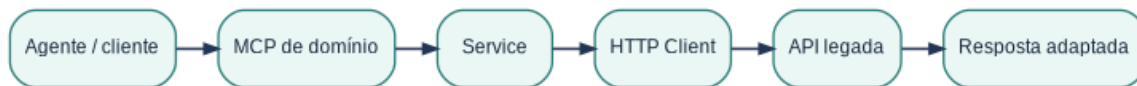


Diagrama de referência — fluxo descrito na aula.

Conteúdo da aula

Neste momento do módulo, eu passo a abordar um dos cenários mais relevantes no uso de Model Context Protocol dentro de empresas: a integração com sistemas legados.

Aqui, o foco deixa de ser apenas técnico e passa a ser também arquitetural. Não se trata apenas de como implementar um MCP, mas de como pensar essa camada como um ponto de conexão entre inteligência artificial e sistemas que já existem há anos dentro das organizações.

Pensando como uma empresa desenha APIs

Antes de escrever qualquer código, eu começo refletindo sobre como APIs são desenhadas.

Em um cenário corporativo, normalmente já existe uma aplicação funcionando. Essa aplicação expõe endpoints REST com diversas operações. No nosso caso, temos um conjunto relativamente pequeno de endpoints, mas em sistemas reais isso pode facilmente chegar a dezenas ou centenas.

A primeira tentação seria transformar cada endpoint em uma tool dentro do MCP. Existem, inclusive, ferramentas que fazem esse mapeamento automaticamente.

Mas essa não é a abordagem correta.

Por que não fazer um mapeamento direto

Fazer um depara direto de REST para MCP pode até funcionar, mas gera um problema sério: excesso de granularidade.

Isso significa que o consumidor do MCP, seja uma LLM ou um sistema, precisaria:

- conhecer muitos endpoints
- entender como combiná-los
- fazer múltiplas chamadas sequenciais
- lidar com detalhes técnicos que não deveriam ser expostos

Isso vai contra o propósito do MCP.

O MCP deve abstrair, não replicar.

MCP como camada de proxy inteligente

Neste projeto, eu começo a construir uma camada intermediária que funciona como um proxy.

Mas não é um proxy simples. Ele não apenas repassa chamadas. Ele organiza, transforma e prepara os dados para consumo.

Essa camada fica entre:

- a API legada
- e o cliente que vai consumir via MCP

Essa separação é fundamental porque permite evoluir o MCP sem alterar o sistema original.

Modelando o domínio

O primeiro passo prático é modelar o domínio.

Mesmo que eu não esteja criando entidades completas com comportamento, eu preciso definir claramente os tipos que representam os dados que vou manipular.

No exemplo, eu crio um tipo de cliente com campos como:

- identificador
- nome
- telefone

Essa definição simples já ajuda a manter consistência ao longo do projeto.

Criando o HTTP Client

Depois disso, eu implemento uma camada de infraestrutura responsável por se comunicar com a API original.

Essa camada é o HTTP Client.

Ela tem uma responsabilidade muito clara:

- fazer chamadas HTTP
- receber respostas
- transformar em estruturas internas

E, principalmente, não conter regra de negócio.

Ela é apenas um adaptador técnico.

Separação de responsabilidades

Essa separação é extremamente importante.

Eu passo a ter:

- uma camada de infraestrutura que fala com a API
- uma camada de serviço que organiza a lógica
- uma camada MCP que expõe capacidades

Cada uma com seu papel bem definido.

Isso evita o acoplamento e facilita a manutenção.

Validando a integração com testes

Antes mesmo de expor qualquer funcionalidade via MCP, eu valido o HTTP Client.

Faço uma chamada simples de listagem de clientes e verifico se os dados estão sendo retornados corretamente.

Esse passo garante que a base está funcionando antes de subir mais uma camada de abstração.

Construindo a camada de serviço

Com o client funcionando, eu crio a camada de serviço.

Essa camada representa a lógica da aplicação.

No exemplo, ela ainda é simples e apenas repassa chamadas para o client. Mas o ponto importante é estrutural: é aqui que, em cenários reais, eu colocaria regras de negócio.

Essa camada é o lugar certo para:

- agregações
- validações
- transformações de dados
- composição de múltiplas chamadas

Começando pelos testes da tool

Seguindo o padrão que venho adotando, eu começo pela definição do teste da tool.

Eu descrevo o comportamento esperado:

- chamar uma tool chamada listCustomers
- receber um array de clientes

Nesse momento, a tool ainda não existe. E isso é proposital.

O teste falha, e essa falha guia a implementação.

Definindo a primeira tool

Com o teste pronto, eu implemento a tool.

Essa tool:

- recebe o server MCP
- recebe a service
- registra a capacidade com nome e descrição
- define o schema de saída
- implementa o callback

Aqui aparece um ponto importante: mesmo quando não há entrada, eu ainda preciso definir claramente o output.

Isso ajuda tanto na validação quanto na compreensão do modelo.

Uso de schema para padronização

Para garantir consistência, eu utilizo schema na definição da tool.

Esse schema descreve exatamente o formato do retorno.

Isso traz benefícios importantes:

- validação automática
- documentação implícita
- melhor entendimento por parte da LLM

Fluxo completo da chamada

Quando a tool é executada, o fluxo é o seguinte:

- o cliente chama o MCP
- o MCP chama a tool
- a tool chama a service
- a service chama o HTTP Client
- o HTTP Client chama a API original
- a resposta percorre o caminho inverso

Esse encadeamento mostra claramente o papel de cada camada.

Registrando a tool no servidor

Um detalhe importante é que a tool só passa a existir quando é registrada no servidor.

Não basta criar a função. É necessário conectá-la ao MCP.

Esse tipo de erro é comum e reforça a importância de entender o ciclo completo.

Introduzindo resources na prática

Depois de implementar a tool, eu avanço para a criação de um resource.

Esse resource tem um papel muito específico: descrever a API que está sendo utilizada.

Ele funciona como uma documentação embutida no MCP.

Resource como documentação viva

No exemplo, eu crio um resource que descreve:

- a base URL da API
- os endpoints disponíveis
- o comportamento esperado

Isso permite que qualquer consumidor entenda rapidamente o contexto.

Para uma LLM, isso é ainda mais valioso, porque reduz a necessidade de inferência.

Testando o resource

Assim como fiz com tools, eu também crio testes para o resource.

Valido que:

- ele está sendo listado
- possui a URI correta
- contém a descrição esperada

Isso garante que o comportamento está consistente.

Inspecionando com MCP Inspector

Após implementar, eu utilizo uma ferramenta de inspeção para validar visualmente o servidor.

Consigo ver:

- tools disponíveis
- resources registrados
- respostas retornadas

Essa etapa é essencial para a depuração e validação.

O papel estratégico do MCP nas empresas

Ao final desta aula, fica muito claro como as empresas podem usar MCPs na prática.

Elas não precisam reescrever sistemas existentes.

Elas podem:

- criar uma camada MCP sobre sistemas legados
- abstrair complexidades
- expor ações de negócio
- permitir que IA interaja com esses sistemas

Isso reduz custo, risco e tempo de adoção.

Evitando armadilhas comuns

Alguns erros que eu evito explicitamente:

- mapear endpoints diretamente sem abstração
- misturar regra de negócio com infraestrutura
- não validar com testes
- não documentar o comportamento

Esses erros comprometem a escalabilidade da solução.

Evolução natural do projeto

Neste ponto, o projeto já possui:

- integração com API legada
- camada de serviço
- tool funcional
- resource descritivo
- testes automatizados

A partir daqui, posso evoluir para:

- novas tools
- autenticação e autorização
- agregações mais complexas
- prompts para facilitar uso

NA PRÁTICA

Abra o(s) diretório(s) da Unidade 5 usando os links do GitHub exibidos no início desta aula. Localize os artefatos citados no conteúdo, reproduza o fluxo apresentado e observe o impacto em testes, estado, tracing, custo ou comportamento, conforme o tema da aula.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Pensando como uma empresa desenha APIs” no fluxo apresentado?
- Que decisão de engenharia está associada a “Por que não fazer um mapeamento direto” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório, nos testes ou no tracing para confirmar que o comportamento descrito na aula está funcionando?

RESUMO DA AULA

- Pensando como uma empresa desenha APIs.
- Por que não fazer um mapeamento direto.
- MCP como camada de proxy inteligente.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 3 — Criando Tools de Listagem e Criação de Clientes

Unidade 5 • Transformando Sistemas Legados em Servidores MCP

OBJETIVOS DE APRENDIZAGEM

- Compreender reforçando o conceito de abstração.
- Relacionar evoluindo a tool de listagem com começando pela criação com testes.
- Aplicar as decisões apresentadas entendendo o retorno da API e verificar o comportamento descrito na aula.

GITHUB DA UNIDADE

[06-your-legacy-api-as-mcp](#)

CONCEITOS-CHAVE

- Reforçando o conceito de abstração
- Evoluindo a tool de listagem
- Começando pela criação com testes
- Entendendo o retorno da API
- Definindo o tipo de retorno
- Implementando a tool de criação

Conteúdo da aula

Nesta etapa do projeto, eu começo a consolidar um dos pontos mais importantes do uso de MCP em aplicações reais: a criação de tools que representam ações de negócio, e não apenas um espelhamento direto de endpoints da API.

Até aqui, já temos uma base arquitetural bem definida, com separação entre HTTP client, service e camada MCP. Agora, o foco passa a ser evoluir essa estrutura com novas capacidades, começando pelas operações mais comuns: listagem e criação de clientes.

Reforçando o conceito de abstração

Antes de implementar qualquer coisa, eu reforço um ponto que é central em todo o módulo.

O MCP não deve ser um reflexo direto da API.

Mesmo que a API tenha endpoints específicos, o MCP pode agrupar, combinar e abstrair essas operações. Em vez de obrigar o cliente a fazer várias chamadas, eu posso oferecer uma única ação que resolve o problema completo.

Essa mentalidade começa a aparecer mais claramente a partir desta aula.

Evoluindo a tool de listagem

A tool de listagem de clientes já estava implementada anteriormente, mas agora ela passa a ter um papel mais importante dentro do fluxo.

Ela deixa de ser apenas um exemplo e passa a ser reutilizada como base para outras operações, como busca e validação.

Isso mostra como tools bem definidas podem ser reutilizadas dentro do próprio sistema.

Começando pela criação com testes

Seguindo o padrão adotado ao longo do projeto, eu começo pela criação de um teste automatizado.

Neste caso, eu defino o comportamento esperado da nova tool:

- receber nome e telefone
- criar um cliente
- retornar um identificador e uma mensagem de confirmação

Antes mesmo da implementação, eu já sei exatamente o que espero da tool.

Como ela ainda não existe, o teste falha. E essa falha orienta o desenvolvimento.

Entendendo o retorno da API

Antes de implementar a tool, eu valido como a API original se comporta.

Ao fazer uma chamada de criação, percebo que ela não retorna o objeto completo do cliente. Em vez disso, retorna:

- uma mensagem de confirmação
- o identificador gerado

Esse detalhe é importante porque define o contrato da tool.

Eu não devo inventar um formato diferente. Eu devo respeitar o comportamento real da API, adaptando apenas o necessário.

Definindo o tipo de retorno

Com base nisso, eu crio um tipo específico para representar o resultado da criação.

Esse tipo contém:

- id
- message

Essa separação é importante porque nem toda operação retorna o mesmo tipo de dado. Criar tipos específicos melhora a clareza e evita confusão no uso das tools.

Implementando a tool de criação

A implementação segue o mesmo padrão da tool de listagem.

Eu defino:

- nome da tool
- descrição
- schema de entrada
- schema de saída
- função de execução

No schema de entrada, eu deixo claro que são necessários:

- nome
- telefone

No schema de saída, defino:

- id
- message

Essa formalização ajuda tanto na validação quanto na interpretação pelo modelo.

Integração com a camada de serviço

Depois de definir a tool, eu preciso conectá-la à camada de serviço.

A service passa a ter um método responsável por criar clientes, que simplesmente delega a operação para o HTTP client.

Embora neste momento pareça apenas um repasse, essa estrutura permite evoluir facilmente depois, adicionando regras de negócio.

Uso de tipos utilitários

Um detalhe interessante nesta etapa é o uso de recursos do TypeScript para melhorar a modelagem.

Ao criar um cliente, eu sei que o identificador não deve ser fornecido pelo usuário. Então, eu utilizo uma estratégia para omitir esse campo do tipo de entrada.

Isso evita erros e reforça o contrato da operação.

Implementação no HTTP client

Na camada de infraestrutura, eu implemento a chamada HTTP responsável pela criação.

Diferente da listagem, aqui preciso:

- definir o método como POST
- configurar headers
- enviar o corpo da requisição

Essa camada continua sem regra de negócio. Ela apenas executa a comunicação com a API.

Validando a execução

Com todas as camadas conectadas, eu executo o teste novamente.

Agora, o fluxo completo acontece:

- o teste chama a tool
- a tool chama a service
- a service chama o HTTP client
- o HTTP client chama a API
- a resposta retorna até o teste

O resultado contém o identificador e a mensagem esperada.

Isso confirma que a tool está funcionando corretamente.

Introduzindo busca como ação composta

Com a criação funcionando, eu avanço para um cenário mais interessante: a busca de clientes.

Aqui começa a ficar mais evidente o valor da abstração.

A API original não possui um endpoint flexível para busca por múltiplos critérios. Então, eu construo essa capacidade dentro do MCP.

Testando o fluxo de busca

Para validar a busca, eu primeiro crio um cliente.

Depois, utilizo a tool de busca para encontrá-lo.

Esse encadeamento mostra como as tools podem ser utilizadas em conjunto para compor fluxos mais complexos.

Definindo múltiplos critérios de entrada

A tool de busca permite múltiplos critérios:

- id
- nome
- telefone

Todos opcionais.

Isso dá flexibilidade ao cliente e reduz a necessidade de múltiplas tools específicas.

Implementando a lógica de busca

Na camada de serviço, eu implemento a lógica de busca.

O comportamento é o seguinte:

- se o id for informado, busco diretamente
- caso contrário, recupero todos os clientes
- aplico filtros em memória com base nos critérios

Essa abordagem simula um comportamento mais avançado, mesmo com uma API limitada.

Comparação dinâmica de atributos

Para realizar o filtro, eu percorro os campos informados na query e comparo com os dados dos clientes.

Essa lógica permite:

- busca parcial
- combinação de critérios
- maior flexibilidade

Mesmo sendo simples, ela demonstra como o MCP pode enriquecer a API original.

Tratamento de ausência de resultados

Se nenhum cliente for encontrado, a tool retorna nulo.

Isso é importante para deixar explícito que a busca não encontrou resultados, evitando ambiguidades.

Criando o prompt de busca

Depois de implementar a tool, eu crio um prompt para facilitar seu uso.

Esse prompt:

- recebe parâmetros de busca
- constrói uma instrução clara
- orienta o modelo a usar a tool corretamente

Isso melhora muito a experiência de uso, especialmente em interfaces como editores.

Validando o prompt

Assim como nas outras partes, eu crio um teste para validar o prompt.

Verifico se:

- ele é registrado corretamente
- gera o texto esperado
- utiliza os parâmetros corretamente

Isso garante consistência na experiência.

Testando no ambiente real

Com tudo implementado, eu conecto o MCP ao editor e executo testes manuais.

Consigo:

- criar clientes
- buscar por nome

- buscar por telefone
- verificar múltiplos resultados

O sistema funciona de ponta a ponta.

Observando o comportamento incremental

Um detalhe interessante é que, a cada execução dos testes, novos clientes são inseridos.

Isso mostra que o sistema está realmente persistindo dados e não apenas simulando respostas.

Consolidando o aprendizado

Nesta aula, eu consolido vários conceitos importantes:

- tools representam ações de negócio
- testes guiam a implementação
- MCP pode abstrair limitações da API
- múltiplas camadas trabalham em conjunto
- prompts melhoram a usabilidade

Tudo isso dentro de uma estrutura organizada e escalável.

Conclusão

Nesta etapa, eu avanço significativamente na construção de um MCP realista.

Ao implementar tools de listagem, criação e busca de clientes, eu mostro como transformar uma API simples em uma interface mais inteligente, flexível e adequada para consumo por modelos de linguagem.

Mais do que executar operações, o MCP passa a oferecer uma experiência de uso mais rica, escondendo complexidades e permitindo que o cliente interaja com o sistema de forma mais natural.

Esse é exatamente o tipo de evolução que empresas buscam ao integrar IA com sistemas existentes.

NA PRÁTICA

Abra o(s) diretório(s) da Unidade 5 usando os links do GitHub exibidos no início desta aula. Localize os artefatos citados no conteúdo, reproduza o fluxo apresentado e observe o impacto em testes, estado, tracing, custo ou comportamento, conforme o tema da aula.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Reforçando o conceito de abstração” no fluxo apresentado?
- Que decisão de engenharia está associada à “Evoluindo a tool de listagem” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório, nos testes ou no tracing para confirmar que o comportamento descrito na aula está funcionando?

RESUMO DA AULA

- Reforçando o conceito de abstração.
- Evoluindo a tool de listagem.
- Começando pela criação com testes.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 4 — Criando Tools de Update e Delete de Clientes + Usando no VSCode

Unidade 5 • Transformando Sistemas Legados em Servidores MCP

OBJETIVOS DE APRENDIZAGEM

- Compreender começando pelo teste do update.
- Relacionar evoluindo o modelo de dados para mutações com criando um schema específico para update.
- Aplicar as decisões apresentadas em implementando a tool de update e verificar o comportamento descrito na aula.

GITHUB DA UNIDADE

[06-your-legacy-api-as-mcp](#)

CONCEITOS-CHAVE

- Começando pelo teste do update
- Evoluindo o modelo de dados para mutações
- Criando um schema específico para update
- Implementando a tool de update
- Integração com a camada de serviço
- Implementando o update no HTTP client

Conteúdo da aula

Para concluir a construção do nosso MCP, eu implemento as duas últimas operações do CRUD: update e delete. Essa etapa fecha o ciclo completo de manipulação de dados e, mais importante do que isso, consolida o padrão arquitetural que venho utilizando desde o início do módulo.

Aqui, eu reforço ainda mais a ideia de reaproveitamento de estrutura, consistência de contratos e integração completa com o ambiente de desenvolvimento.

Começando pelo teste do update

Assim como nas etapas anteriores, eu inicio pela definição do teste.

Eu descrevo o comportamento esperado da operação de update:

- receber um identificador
- receber novos dados do cliente
- atualizar o registro existente
- retornar uma mensagem de confirmação
- retornar o mesmo identificador atualizado

Um detalhe importante aqui é que o update depende da existência de um cliente previamente criado. Para manter o teste confiável, eu preparo esse cenário dentro do próprio teste.

Isso evita dependência entre testes e garante isolamento.

Evoluindo o modelo de dados para mutações

Neste momento, eu percebo um padrão importante.

Tanto a criação quanto a atualização retornam estruturas muito semelhantes:

- id
- message
- possibilidade de erro

Com isso, eu crio um tipo mais genérico para representar mutações.

Esse tipo passa a encapsular:

- identificador
- mensagem de confirmação
- indicador de erro

Essa padronização simplifica bastante o desenvolvimento, porque agora todas as operações de escrita seguem o mesmo contrato.

Criando um schema específico para update

Diferente da criação, o update exige o identificador.

Então, eu crio um schema específico onde:

- o id é obrigatório
- os demais campos seguem o mesmo padrão do cliente

Essa separação é importante porque evita ambiguidades e garante que o contrato da operação esteja correto.

Implementando a tool de update

A implementação da tool segue o mesmo padrão já estabelecido:

- registro no servidor MCP
- definição de descrição
- uso de schema de entrada e saída
- implementação do callback

Aqui, eu aproveito um ganho importante: reutilização.

Em vez de redefinir todos os campos manualmente, eu reutilizo os schemas já criados anteriormente. Isso reduz a duplicação e melhora a consistência.

Integração com a camada de serviço

Na camada de serviço, o método de update é bastante direto.

Ele recebe o objeto de atualização e repassa para o HTTP client.

Embora simples, essa estrutura mantém a separação de responsabilidades e permite evolução futura.

Implementando o update no HTTP client

Na camada de infraestrutura, eu implemento a chamada HTTP para atualização.

Alguns pontos importantes aqui:

- uso do método PUT
- envio do identificador na URL
- envio do restante dos dados no corpo da requisição

Durante a implementação, surge um problema comum: envio duplicado do identificador.

Eu estava enviando o id tanto na URL quanto no corpo, o que gerava erro na API.

A correção foi separar:

- id na URL
- restante dos dados no corpo

Esse tipo de ajuste é comum em integrações reais.

Lidando com validações da API

Outro ponto importante foi perceber que a API exige todos os campos no update.

Ou seja, não é um patch parcial.

Com isso, eu preciso garantir que:

- nome e telefone sejam enviados
- mesmo que apenas um campo esteja sendo alterado

Essa restrição impacta diretamente o design da tool.

Validando o fluxo completo

Após os ajustes, eu executo novamente o teste.

O fluxo funciona corretamente:

- cliente é criado
- cliente é atualizado
- id permanece o mesmo
- mensagem é retornada corretamente

Isso confirma que a tool está funcionando como esperado.

Implementando a tool de delete

Com o update finalizado, eu parto para o delete.

Essa é uma das implementações mais simples do projeto.

A operação exige apenas:

- o identificador do cliente

E retorna:

- mensagem de confirmação
- id do registro removido

Definindo o schema de delete

O schema de entrada é mínimo:

- id obrigatório

O schema de saída reutiliza o padrão de mutação já definido.

Isso mostra novamente o valor da padronização.

Implementação nas camadas

A implementação segue o mesmo fluxo:

- tool registrada no MCP
- service delegando a operação
- HTTP client executando a requisição DELETE

Sem surpresas, mas com consistência.

Testando o delete

No teste, eu sigo o mesmo padrão:

- crio um cliente
- capturo o id
- executo o delete
- valido a mensagem retornada

Depois disso, posso validar também que o cliente não existe mais.

Uso do MCP Inspector

Antes de levar para o editor, eu utilizo o inspector para validar:

- tools disponíveis
- execução manual
- retorno das operações

Consigo ver claramente:

- list
- get
- create
- update
- delete

Todas funcionando corretamente.

Integração com o VSCode

Com tudo implementado, eu levo o MCP para o VSCode.

O editor passa a atuar como cliente MCP e consegue:

- descobrir as tools
- executar operações
- interagir com o servidor

Esse é o momento em que o projeto deixa de ser apenas código e passa a ser uma ferramenta utilizável.

Testando com linguagem natural

Um dos pontos mais interessantes dessa etapa é o uso via linguagem natural.

Eu faço uma solicitação como:

- remover cliente com determinado nome

Mesmo sem especificar:

- qual tool usar
- qual sequência executar

O sistema consegue:

- identificar que precisa buscar o cliente
- recuperar o id
- executar o delete

Esse comportamento mostra o poder da combinação entre:

- MCP bem estruturado
- descrições claras
- contexto adequado

Abstração além da API original

Um detalhe importante é que a API original não possuía busca por nome.

Mesmo assim, o MCP consegue oferecer essa capacidade.

Isso acontece porque:

- a lógica foi implementada na camada de serviço
- a tool abstrai essa complexidade

Esse é um exemplo claro de como MCP adiciona valor sobre sistemas legados.

Consolidando o CRUD completo

Neste ponto, o sistema possui:

- listagem de clientes
- criação de clientes
- busca por múltiplos critérios
- atualização de dados
- remoção de registros

Tudo acessível via MCP.

O papel das descrições e schemas

Outro aprendizado importante é o impacto das descrições.

Ao definir bem:

- nomes de tools
- descrições
- schemas

Eu facilito o trabalho da IA.

Ela consegue:

- entender o que fazer
- escolher a tool correta
- encadear operações

Sem necessidade de instruções explícitas.

Próximos passos naturais

Ao final desta aula, ainda existem possibilidades de evolução:

- criação de prompts para todas as tools
- melhoria nas mensagens de retorno
- implementação de autenticação
- tratamento mais robusto de erros

Mas a base já está completa.

Conclusão

Nesta aula, eu finalizo a implementação do CRUD completo dentro do MCP, adicionando as operações de update e delete e validando seu funcionamento tanto em testes automatizados quanto no VSCode.

Mais do que apenas completar funcionalidades, eu demonstro como manter consistência arquitetural, reaproveitar estruturas e construir um sistema que realmente funciona em um ambiente real.

O resultado final é um MCP robusto, organizado e capaz de conectar uma IA a um sistema legado de forma eficiente, segura e escalável.

NA PRÁTICA

Abra o(s) diretório(s) da Unidade 5 usando os links do GitHub exibidos no início desta aula. Localize os artefatos citados no conteúdo, reproduza o fluxo apresentado e observe o impacto em testes, estado, tracing, custo ou comportamento, conforme o tema da aula.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Começando pelo teste do update” no fluxo apresentado?
- Que decisão de engenharia está associada à “Evoluindo o modelo de dados para mutações” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório, nos testes ou no tracing para confirmar que o comportamento descrito na aula está funcionando?

RESUMO DA AULA

- Começando pelo teste do update.
- Evoluindo o modelo de dados para mutações.
- Criando um schema específico para update.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Revisão da Unidade 5

Use esta página como revisão rápida antes de avançar. Você deve conseguir relacionar as aulas da unidade em um único fluxo de engenharia.



Fluxo visual da Unidade 5.

CHECKLIST DA UNIDADE

- [Revisar a Aula 1: Template Inicial e Arquitetura + Boas Práticas de Organização de Código e Estrutura de Projeto](#)
- [Revisar a Aula 2: Como Empresas Usam MCPs para Conectar IA a Sistemas Legados](#)
- [Revisar a Aula 3: Criando Tools de Listagem e Criação de Clientes](#)
- [Revisar a Aula 4: Criando Tools de Update e Delete de Clientes + Usando no VSCode](#)

GITHUB DA UNIDADE

[Repositório oficial da disciplina](#)

[06-your-legacy-api-as-mcp](#)

Unidade 6

Segurança e Governança em MCPs

Evolução da segurança da API e do servidor MCP com JWT, RBAC, Service Tokens, rate limiting, controle de autorização e tratamento consistente de erros.



Fluxo visual da Unidade 6 — reorganização dos elementos descritos nas aulas.

GITHUB DA UNIDADE

[Repositório oficial da disciplina](#)

[07-api-security-auth-rate-limiting-template](#)

[07-api-security-auth-rate-limiting-z](#)

Aula	Tema
1	Template Inicial e Arquitetura + Boas Práticas de Organização de Código e Estrutura de Projeto
2	Implementando Autenticação com RBAC e Autenticação com JWT em Web API
3	Autenticação e Autorização com Service Tokens em Web APIs
4	Controlando Acesso por Tokens com Rate Limiting
5	Usando Service Tokens em Nosso Servidor MCP + Rate Limiting

Aula 1 — Template Inicial e Arquitetura + Boas Práticas de Organização de Código e Estrutura de Projeto

Unidade 6 • Segurança e Governança em MCPs

OBJETIVOS DE APRENDIZAGEM

- Compreender a mudança de foco do módulo.
- Relacionar segurança aplicada à API legada com JWT para autenticação de usuários.
- Aplicar as decisões apresentadas em por que MCP costuma usar outro modelo e verificar o comportamento descrito na aula.

GITHUB DA UNIDADE

[07-api-security-auth-rate-limiting-template](#)

[07-api-security-auth-rate-limiting-z](#)

CONCEITOS-CHAVE

- A mudança de foco do módulo
- Segurança aplicada à API legada
- JWT para autenticação de usuários
- Por que MCP costuma usar outro modelo
- O problema das chaves sem expiração
- Service tokens como prática de mercado

Conteúdo da aula

Neste módulo, eu começo a tratar de um tema essencial para qualquer aplicação exposta ao mundo real: segurança. Mais especificamente, eu mostro como estruturar autenticação, autorização e controle de uso em uma API legada e, ao mesmo tempo, preparar esse mesmo raciocínio para servidores MCP.

O ponto central aqui é que publicar uma aplicação na web não é apenas fazê-la funcionar. É garantir que ela funcione com controle, previsibilidade e proteção adequada para quem consome e para quem opera o sistema.

A mudança de foco do módulo

Até aqui, o projeto vinha mostrando como construir MCPs, criar tools, expor resources e prompts, além de integrar tudo isso ao editor e a aplicações em geral. Agora, o foco muda um pouco.

Neste módulo, eu passo a olhar para três pilares fundamentais:

- autenticação
- autorização
- limitação de uso

Esses três elementos são decisivos quando deixamos de trabalhar apenas em ambiente local e começamos a pensar em publicação real.

Segurança aplicada à API legada

A base do trabalho continua sendo a API legada que já vinha sendo usada nos módulos anteriores.

Sobre essa API, eu passo a implementar autenticação com JWT e autorização baseada em papéis. A ideia é simples:

- usuários autenticados recebem um token
- esse token é usado para acessar as rotas
- dependendo do papel do usuário, algumas ações são permitidas e outras não

Com isso, a aplicação começa a sair de um cenário puramente funcional e passa a operar com regras reais de acesso.

JWT para autenticação de usuários

Na parte tradicional da API, eu utilizo JWT para autenticação.

Esse modelo é bastante conhecido no desenvolvimento web. O usuário informa suas credenciais, recebe um token assinado e passa a utilizá-lo nas próximas chamadas. Esse token pode expirar e, justamente por isso, oferece uma camada importante de proteção caso seja vazado.

Essa escolha faz sentido para autenticação de usuários humanos e para fluxos clássicos de login.

Por que MCP costuma usar outro modelo

Quando eu trago a discussão para servidores MCP, a lógica costuma ser diferente.

No mundo de MCP, normalmente não faz tanto sentido obrigar o cliente a:

- informar usuário e senha
- solicitar token com frequência
- lidar com expiração constante

Na prática, o padrão mais comum é o uso de API keys permanentes, armazenadas em variáveis de ambiente, arquivos de configuração ou outros mecanismos seguros de provisionamento.

Isso é muito parecido com o que já utilizamos em integrações com serviços externos ao longo do curso.

O problema das chaves sem expiração

Esse modelo, embora prático, traz um risco importante.

Se uma API key vaza, ela não expira sozinha como um JWT tradicional. Isso significa que, sem uma estratégia adicional, o dano potencial é muito maior.

É exatamente por isso que neste módulo eu introduzo a ideia de service tokens.

Service tokens como prática de mercado

Os service tokens funcionam como uma forma mais controlada de acesso programático.

Com eles, eu consigo:

- identificar o cliente que está consumindo o serviço
- definir quais ações esse cliente pode executar
- limitar quantas chamadas ele pode fazer
- bloquear ou revogar acesso quando necessário

Esse modelo é muito comum em ambientes corporativos, principalmente quando o consumidor da aplicação não é um usuário final, mas outro serviço, integração ou agente automatizado.

Controle de autorização por papel

Dentro do template, eu também preparo a aplicação para trabalhar com papéis diferentes.

No exemplo apresentado, eu utilizo dois perfis:

- membro
- admin

A distinção entre eles segue um padrão simples e bastante realista:

- membro tem acesso somente de leitura
- admin pode escrever, atualizar e remover

Essa separação mostra como a autorização não é a mesma coisa que a autenticação.

Autenticação responde à pergunta: quem é você?

Autorização responde à pergunta: o que você pode fazer?

Template preparado para evolução

Para facilitar o desenvolvimento, eu já parto de um template pronto.

Esse template reaproveita bastante do que já foi construído anteriormente, mas adiciona novas peças voltadas à segurança. Entre elas, destaco:

- bibliotecas para JWT
- mecanismo de rate limiting
- nova camada dedicada a autenticação
- bateria de testes preparada para os novos cenários

Ou seja, eu não começo do zero. Eu evoluo uma base já conhecida e organizada.

Novas dependências e atualização do ambiente

Como parte dessa evolução, também atualizo os pacotes do projeto.

A aplicação passa a utilizar uma versão mais recente do ambiente, e isso exige atenção com compatibilidade. Além disso, são adicionadas bibliotecas específicas para autenticação e controle de limite de chamadas.

Essa atualização é importante porque mostra uma preocupação realista com manutenção de projeto, e não apenas com demonstração pontual.

Estrutura de código e nova camada de autenticação

Uma das mudanças mais relevantes na arquitetura é a introdução de uma camada específica de autenticação.

Nessa camada, eu começo a organizar elementos como:

- usuários simulados
- papéis de acesso
- regras de autenticação

- estrutura para geração e validação de tokens

Essa separação é importante porque evita misturar segurança com lógica de negócio ou com infraestrutura de transporte.

É uma decisão de organização de código, mas também uma decisão arquitetural.

O valor dos testes já preparados

Outro ponto forte do template é a presença de uma bateria grande de testes.

Esses testes já deixam preparado o terreno para validar:

- autenticação
- autorização
- service tokens
- integração com servidores MCP

Muitos deles estão inicialmente desabilitados, justamente porque a implementação será feita ao longo do módulo. Ainda assim, eles já servem como contrato do comportamento esperado.

Essa abordagem é muito útil porque permite implementar com objetivo claro, sem depender apenas de verificação manual.

Preparação do ambiente de execução

Antes de começar a codificar, eu também organizo o ambiente.

A orientação é subir apenas o banco de dados, deixando o restante do sistema sob controle do fluxo de desenvolvimento. Isso ajuda a:

- evitar interferência de containers antigos
- garantir ambiente limpo
- validar os testes com previsibilidade

Essa preparação é simples, mas reforça uma prática importante: segurança e infraestrutura andam juntas.

Reaproveitamento consciente do projeto anterior

Algo que considero valioso nesta etapa é que eu não abandono o que já foi construído.

Eu pego o mesmo MCP das aulas anteriores e o adapto para um novo contexto. Isso mostra que uma boa estrutura de projeto permite evolução incremental.

Em vez de recriar tudo, eu amplio a arquitetura existente com novas responsabilidades.

Organização de código como base para segurança

Esta aula também reforça um ponto que costuma passar despercebido: a segurança não começa só no token. Ela começa na organização do código.

Quando eu separo corretamente:

- autenticação
- autorização
- regras de negócio
- integração com MCP
- testes

eu torno o sistema mais legível, mais auditável e mais fácil de evoluir.

Projetos inseguros quase sempre também são projetos mal organizados.

O que fica preparado para as próximas etapas

Ao final desta aula, o que eu deixo pronto não é apenas um template novo. Eu deixo uma fundação.

Essa fundação inclui:

- API legada pronta para receber autenticação
- base para autorização por papéis
- estrutura para service tokens
- ambiente de testes preparado
- arquitetura pronta para aplicar rate limiting

Com isso, as próximas aulas podem focar diretamente na implementação dos comportamentos de segurança, em vez de perder tempo com reorganização estrutural.

NA PRÁTICA

Abra o(s) diretório(s) da Unidade 6 usando os links do GitHub exibidos no início desta aula. Localize os artefatos citados no conteúdo, reproduza o fluxo apresentado e observe o impacto em testes, estado, tracing, custo ou comportamento, conforme o tema da aula.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “A mudança de foco do módulo” no fluxo apresentado?
- Que decisão de engenharia está associada a “Segurança aplicada à API legada” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório, nos testes ou no tracing para confirmar que o comportamento descrito na aula está funcionando?

RESUMO DA AULA

- A mudança de foco do módulo.
- Segurança aplicada à API legada.
- JWT para autenticação de usuários.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 2 — Implementando Autenticação com RBAC e Autenticação com JWT em Web API

Unidade 6 • Segurança e Governança em MCPs

OBJETIVOS DE APRENDIZAGEM

- Compreender o papel do JWT na aplicação.
- Relacionar registrando o plugin de autenticação com separando a autenticação em uma camada própria.
- Aplicar as decisões apresentadas em usuários simulados para a demonstração e verificar o comportamento descrito na aula.

GITHUB DA UNIDADE

[07-api-security-auth-rate-limiting-template](#)
[07-api-security-auth-rate-limiting-z](#)

CONCEITOS-CHAVE

- O papel do JWT na aplicação
- Registrando o plugin de autenticação
- Separando a autenticação em uma camada própria
- Usuários simulados para a demonstração
- Diferença entre autenticação e autorização
- Definindo rotas públicas

Conteúdo da aula

Neste ponto do projeto, eu começo a tratar de um tema que muda completamente o nível de maturidade da aplicação: autenticação e autorização. Até aqui, a API já funcionava, o CRUD já estava operando e o MCP já conseguia se comunicar com o sistema legado. Mas faltava algo essencial para qualquer cenário real de publicação: controlar quem pode acessar o quê.

É exatamente aqui que entram dois conceitos fundamentais: JWT para autenticação e RBAC para autorização. A combinação dessas duas estratégias me permite validar a identidade do usuário e, em seguida, restringir o que ele pode fazer com base no papel que ocupa dentro do sistema.

O papel do JWT na aplicação

A primeira decisão que eu tomo é como vou trabalhar com o token JWT.

Assim como em outros cenários onde preciso assinar ou proteger dados, aqui também preciso de uma chave secreta. Essa chave é o elemento que garante a integridade do token gerado. Sem ela, não é possível assinar corretamente os tokens nem verificar se eles foram adulterados.

No projeto, eu defino essa chave de forma simples para fins didáticos, mas o ponto importante é entender o papel dela. Em uma aplicação real, essa chave precisa ser forte, protegida e mantida fora do código-fonte, normalmente em variáveis de ambiente ou mecanismos mais robustos de gestão de segredo.

O JWT entra como mecanismo de autenticação. Em outras palavras, ele me permite dizer: esse usuário fez login com sucesso, então agora ele tem um token válido que o representa nas próximas chamadas.

Registrando o plugin de autenticação

Como a aplicação está usando Fastify, eu aproveito o ecossistema do próprio framework para registrar o plugin de JWT.

Esse passo é importante porque integra a autenticação diretamente ao ciclo de vida da aplicação. A partir desse momento, o Fastify já passa a oferecer recursos prontos para:

- assinar tokens
- validar tokens recebidos
- anexar informações do usuário à requisição
- facilitar o fluxo de autenticação

Isso simplifica bastante a implementação, porque eu não preciso construir toda essa base manualmente.

Separando a autenticação em uma camada própria

Um ponto que considero muito importante nesta etapa é a organização do código.

Em vez de misturar toda a lógica de autenticação diretamente no arquivo principal da aplicação, eu crio uma camada específica para isso. Essa decisão ajuda a manter o projeto mais limpo, mais fácil de entender e mais simples de evoluir.

Essa camada dedicada à autenticação passa a concentrar responsabilidades como:

- definição de rotas públicas
- lógica de login
- verificação de token
- autorização por papel
- integração com os hooks do framework

Esse tipo de separação faz diferença principalmente em projetos maiores, onde a manutenção se torna mais sensível.

Usuários simulados para a demonstração

Para a demonstração, eu utilizo um conjunto fixo de usuários.

A ideia aqui não é construir um sistema completo de cadastro e gerenciamento de identidade, mas sim demonstrar a arquitetura e o comportamento da autenticação. Por isso, defino usuários simples em memória, com credenciais e papéis já conhecidos.

No exemplo, trabalho com dois perfis principais:

- admin
- member

Essa distinção é a base do controle de acesso da aplicação.

Diferença entre autenticação e autorização

Aqui vale reforçar uma distinção que é muito importante.

Autenticação responde à pergunta: quem é você?

Autorização responde à pergunta: o que você pode fazer?

Na prática, o JWT resolve a primeira parte. Ele me diz que o usuário é quem afirma ser, desde que tenha feito login com credenciais válidas e esteja portando um token legítimo.

Já o RBAC resolve a segunda parte. Ele define se esse usuário, já autenticado, pode ou não executar determinada ação.

Definindo rotas públicas

Antes de bloquear tudo, eu preciso decidir quais rotas devem permanecer públicas.

Essa é uma prática muito comum em aplicações web. Normalmente, algumas rotas ficam abertas por necessidade funcional. No nosso caso, por exemplo, faz sentido manter públicas rotas como:

- health check
- login
- endpoint de emissão de token de serviço

Essas rotas precisam ser acessíveis sem autenticação prévia, porque ou servem para monitoramento da aplicação ou são justamente o ponto de entrada para o processo de autenticação.

Usando hooks do Fastify

Para aplicar a verificação de acesso antes que a requisição chegue às rotas protegidas, eu utilizo um recurso muito importante do Fastify: os hooks.

O hook escolhido aqui é `onRequest`, porque ele é executado logo no início do ciclo de uma requisição. Isso significa que consigo interceptar a chamada antes que ela alcance qualquer lógica de negócio.

Esse ponto é muito importante. Segurança deve acontecer o mais cedo possível no fluxo. Se eu deixo a requisição andar demais antes de validar, abro espaço para comportamento indesejado.

Lógica inicial de proteção

A primeira regra que aplico é simples:

- se a rota for pública, a requisição segue normalmente
- se a rota não for pública, ela precisa ser autenticada

Essa decisão já muda completamente o comportamento da aplicação. Em vez de eu marcar manualmente rota por rota como protegida, passo a adotar um modelo mais seguro por padrão. Tudo é privado, exceto o que eu explicitamente libero.

Esse é um padrão que considero muito saudável, porque reduz o risco de esquecer uma rota exposta sem proteção.

Criando a rota de login

Com a estrutura básica pronta, eu parto para a implementação da rota de login.

Essa rota é responsável por:

- receber credenciais do usuário
- validar se o usuário existe
- validar se a senha está correta
- gerar um token JWT em caso de sucesso

Como se trata de uma rota pública, ela não precisa exigir token. Pelo contrário, ela é justamente a porta de entrada para obter esse token.

Validando a entrada da requisição

Um detalhe importante aqui é que eu também estruturei a validação da entrada.

Na rota de login, o corpo da requisição precisa conter:

- username
- password

Em vez de aceitar qualquer formato de entrada, eu defino explicitamente esse contrato. Isso melhora a previsibilidade da API e reduz a chance de comportamento inconsistente.

Além disso, também deixo descrita a estrutura da resposta esperada, incluindo os cenários de sucesso e falha.

Procurando o usuário e validando credenciais

Depois de receber os dados, eu procuro o usuário na lista definida internamente.

Faço essa busca com uma pequena preocupação de robustez: trato o nome de usuário de forma case insensitive, para evitar que diferenças entre maiúsculas e minúsculas atrapalhem a autenticação. Já a senha continua exigindo correspondência exata, como deve ser.

Se o usuário não for encontrado, ou se a senha estiver errada, retorno uma resposta de falha de autenticação.

Esse ponto é central porque é ele que impede o acesso não autorizado logo na entrada do sistema.

Gerando o token JWT

Quando o usuário é validado com sucesso, eu gero o token.

E aqui há um detalhe muito importante: eu não coloco no token apenas o nome do usuário. Eu também incluo o papel dele. Isso significa que o token já carrega a informação necessária para que, nas etapas seguintes, o sistema saiba se aquele usuário é admin ou member.

Esse cuidado simplifica bastante a autorização posterior, porque a role já acompanha a identidade autenticada do usuário.

O token como identidade transportada

Depois de emitido, o token passa a acompanhar o usuário em todas as chamadas protegidas.

Isso normalmente acontece por meio do header de autorização, no formato padrão Bearer. A cada nova requisição, a aplicação pode verificar esse token e reconstruir o contexto do usuário.

Na prática, isso significa que o sistema não precisa perguntar novamente quem é o usuário a cada chamada. O token já leva essa informação consigo.

Validando o token nas rotas privadas

Com o login funcionando, eu volto ao hook global e implemento a segunda parte do fluxo: a verificação do token nas rotas privadas.

Essa verificação é feita usando o próprio mecanismo disponibilizado pelo plugin do Fastify. Quando o token é válido, a requisição segue. Quando ele é inválido, expirado ou ausente, a aplicação interrompe o fluxo e devolve erro de autorização.

Esse passo é o que protege efetivamente as rotas.

O usuário autenticado dentro da requisição

Depois que o token é validado, o contexto do usuário fica disponível na própria requisição.

Isso é extremamente útil porque, a partir daí, qualquer camada posterior da aplicação pode consultar dados como:

- username
- role

Sem precisar reprocessar token ou buscar informações adicionais.

Esse contexto é o elo entre autenticação e autorização.

Introduzindo RBAC

Com o usuário autenticado e a role disponível, eu começo a implementar RBAC.

Role-Based Access Control é um modelo simples e muito eficaz de autorização. Em vez de criar regras soltas e espalhadas, eu passo a tomar decisões com base no papel do usuário.

No nosso caso, a regra é clara:

- member pode apenas ler
- admin pode escrever, atualizar e remover

Esse tipo de política é bastante comum em sistemas corporativos.

Usando pre-handler para autorização

Para aplicar RBAC nas rotas específicas, eu utilizo outro recurso importante do Fastify: `preHandler`.

Se `onRequest` protege a entrada geral, `preHandler` me permite fazer verificações específicas antes de um endpoint individual ser executado.

Isso é ideal para autorização, porque nem toda rota protegida exige a mesma permissão. Algumas aceitam membros, outras exigem administradores.

Criando uma função reutilizável para papel

Em vez de escrever a mesma verificação em toda rota, eu abstraio a lógica em uma função reutilizável.

Essa função recebe a role necessária para a execução e compara com a role do usuário autenticado. Se a role for suficiente, a chamada segue. Se não for, a aplicação responde com erro de permissão insuficiente.

Esse tipo de abstração melhora muito a legibilidade e reduz duplicação.

Aplicando a autorização nas rotas de escrita

A partir daí, eu começo a marcar as rotas conforme o nível de acesso necessário.

As rotas de escrita, como:

- criação
- atualização
- remoção

passam a exigir role de admin.

Já as rotas de leitura continuam acessíveis a usuários com papel de member.

Esse mapeamento traduz a política de segurança diretamente para o comportamento da API.

Testando com admin e member

Depois de implementar a lógica, eu valido o comportamento usando os dois perfis.

Quando utilizo um usuário admin:

- o login funciona
- o token é gerado
- a criação de clientes funciona
- as rotas protegidas de escrita respondem normalmente

Quando utilizo um usuário member:

- o login também funciona
- o token é gerado corretamente
- as rotas de leitura funcionam
- mas as rotas de escrita passam a retornar erro de permissão

Esse contraste é exatamente o que eu queria demonstrar.

A importância dos testes automatizados

Como venho reforçando ao longo do módulo, eu não paro na validação manual.

Depois de implementar as regras, eu executo a bateria de testes automatizados. É isso que me dá confiança de que o comportamento está correto e que futuras alterações não vão quebrar a segurança sem que eu perceba.

Os testes passam a validar cenários como:

- login com credenciais válidas
- login com credenciais inválidas
- acesso autorizado para leitura
- bloqueio de escrita para member
- liberação de escrita para admin

Esse conjunto de testes é o que transforma a implementação em algo realmente confiável.

O que essa etapa prepara para o próximo passo

Com JWT e RBAC implementados na Web API, eu deixo pronto o terreno para a próxima evolução: autenticação voltada para consumo por serviços e por MCPs.

Isso é importante porque o modelo de autenticação para usuários humanos não é necessariamente o melhor modelo para integrações automatizadas. Em APIs consumidas por clientes programáticos, entram em cena conceitos como service tokens e API keys.

Mas esse próximo passo só faz sentido porque a base de autenticação e autorização já está bem estabelecida aqui.

NA PRÁTICA

Abra o(s) diretório(s) da Unidade 6 usando os links do GitHub exibidos no início desta aula. Localize os artefatos citados no conteúdo, reproduza o fluxo apresentado e observe o impacto em testes, estado, tracing, custo ou comportamento, conforme o tema da aula.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “O papel do JWT na aplicação” no fluxo apresentado?
- Que decisão de engenharia está associada a “Registrando o plugin de autenticação” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório, nos testes ou no tracing para confirmar que o comportamento descrito na aula está funcionando?

RESUMO DA AULA

- O papel do JWT na aplicação.
- Registrando o plugin de autenticação.
- Separando a autenticação em uma camada própria.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 3 — Autenticação e Autorização com Service Tokens em Web APIs

Unidade 6 • Segurança e Governança em MCPs

OBJETIVOS DE APRENDIZAGEM

- Compreender por que Service Tokens existem.
- Relacionar a diferença entre JWT e Service Token com o custo da simplicidade.
- Aplicar as decisões apresentadas criando o conceito de super admin e verificar o comportamento descrito na aula.

GITHUB DA UNIDADE

[07-api-security-auth-rate-limiting-template](#)

[07-api-security-auth-rate-limiting-z](#)

CONCEITOS-CHAVE

- Por que Service Tokens existem
- Diferença entre JWT e Service Token
- O custo da simplicidade
- Criando o conceito de super admin
- Criando a rota de geração de Service Token
- Reutilizando validação de usuário

Conteúdo da aula

Depois de implementar autenticação com JWT e autorização com RBAC, eu avanço para um modelo muito mais comum em integrações com sistemas e, principalmente, em cenários com MCP: o uso de Service Tokens.

Aqui, a ideia muda completamente de perspectiva. Em vez de autenticar um usuário humano a cada requisição, eu passo a trabalhar com uma credencial persistente, que representa um cliente, integração ou aplicação consumidora.

Por que Service Tokens existem

Quando eu trabalho com MCPs ou integrações automatizadas, o fluxo tradicional de login e senha deixa de fazer sentido.

Não é prático exigir que:

- um editor faça login manual
- uma integração renove token constantemente
- um agente automatizado lide com expiração frequente

Nesse contexto, surge o Service Token, que funciona como uma API Key.

Ele permite que o cliente:

- se autentique uma única vez
- reutilize a credencial indefinidamente
- simplifique o fluxo de integração

Esse é exatamente o padrão que vemos em diversas plataformas modernas.

Diferença entre JWT e Service Token

É importante entender claramente a diferença entre os dois modelos.

O JWT:

- expira
- representa uma sessão de usuário
- exige fluxo de login
- é ideal para usuários humanos

O Service Token:

- não expira automaticamente
- representa uma integração
- é reutilizável
- é ideal para sistemas e MCPs

Essa diferença impacta diretamente na arquitetura da aplicação.

O custo da simplicidade

Embora o Service Token seja mais simples de usar, ele traz um risco maior.

Se um token vaza:

- ele continua válido
- não expira sozinho
- pode ser usado indefinidamente

Por isso, sistemas reais precisam de mecanismos como:

- revogação manual
- rotação de tokens
- monitoramento de uso

Nesta aula, eu foco na implementação básica, mas deixo claro esse ponto de atenção.

Criando o conceito de super admin

Para permitir a geração de Service Tokens, eu introduzo um novo conceito: o super secret.

Esse valor funciona como uma credencial adicional, usada apenas para autorizar a criação de novos tokens.

Na prática, ele representa um nível mais alto de privilégio, normalmente associado a:

- administradores do sistema
- interfaces internas
- painéis de gestão

Esse cuidado evita que qualquer usuário consiga gerar tokens indiscriminadamente.

Criando a rota de geração de Service Token

A implementação começa com uma nova rota.

Essa rota recebe:

- username
- password
- admin super secret

Antes de qualquer coisa, eu valido o super secret. Se ele estiver incorreto, a requisição é rejeitada imediatamente.

Esse passo é fundamental para proteger a geração de tokens.

Reutilizando validação de usuário

Depois de validar o super secret, eu reutilizo a mesma lógica de autenticação já implementada:

- verifico se o usuário existe
- verifico se a senha está correta

Isso evita duplicação e mantém consistência.

Gerando o Service Token

Ao invés de gerar um JWT, eu passo a gerar um identificador único.

Para isso, utilizo um mecanismo de geração de identificadores aleatórios. O resultado é uma chave longa, difícil de adivinhar, que será usada como token de acesso.

Esse token é retornado junto com:

- a role do usuário

Esse detalhe é importante porque o sistema precisa saber quais permissões aplicar posteriormente.

Armazenando os tokens emitidos

Diferente do JWT, que é stateless, o Service Token precisa ser armazenado.

Eu crio uma estrutura em memória para guardar:

- o token gerado
- o usuário associado
- a role correspondente

Em um sistema real, isso deveria estar em um banco de dados ou sistema persistente.

Mas, para fins didáticos, essa abordagem é suficiente.

Adaptando o fluxo de autenticação

Agora vem a parte mais importante: integrar o Service Token ao fluxo existente.

Até então, a aplicação validava apenas JWT.

Agora, eu passo a ter dois caminhos possíveis:

- autenticação via Service Token
- autenticação via JWT

Detectando o tipo de token

No hook de autenticação, eu extraio o token do header de autorização.

Em seguida, tento identificar:

- se ele corresponde a um Service Token emitido
- ou se deve ser tratado como JWT

Se o token for encontrado na estrutura de Service Tokens:

- eu considero o usuário autenticado
- atribuo o contexto diretamente à requisição
- não preciso validar JWT

Caso contrário, sigo o fluxo tradicional de JWT.

Unificando o resultado da autenticação

Um ponto muito importante aqui é que, independentemente do tipo de autenticação, o resultado final é o mesmo.

A requisição passa a ter:

- usuário identificado
- role definida

Isso permite que o restante da aplicação funcione sem saber qual método de autenticação foi utilizado.

Essa unificação simplifica bastante o design.

Mantendo compatibilidade com RBAC

Como a role continua sendo parte do contexto, o sistema de RBAC continua funcionando normalmente.

Ou seja:

- um token de admin pode executar operações de escrita
- um token de member fica restrito à leitura

Isso garante consistência de regras, independentemente do método de autenticação.

Uso prático com MCP

Esse modelo é exatamente o que eu preciso para MCP.

Em vez de configurar login e senha no editor, eu posso:

- gerar um Service Token
- armazenar em variável de ambiente
- reutilizar automaticamente

Isso é compatível com padrões já usados em ferramentas como:

- integrações via HTTP
- SDKs
- serviços externos

Header de autenticação

Na implementação, eu opto por reutilizar o mesmo header de autorização.

Embora uma prática comum seja usar headers específicos, como um campo dedicado para Service Token, manter o padrão Bearer simplifica a integração neste momento.

O importante é que o backend saiba diferenciar os tipos.

Tratando ausência de header

Um detalhe importante que surge durante os testes é o tratamento de ausência de header.

Se o header não existir:

- o sistema não deve quebrar
- deve seguir o fluxo e retornar erro de não autorizado

Esse tipo de validação evita erros inesperados em produção.

Validando com testes automatizados

Após implementar o fluxo completo, eu executo a suíte de testes.

Os testes passam a cobrir:

- geração de Service Token
- autenticação com token válido
- falha com token inválido
- integração com RBAC

- compatibilidade com rotas existentes

Isso garante que a nova funcionalidade não quebre o que já estava funcionando.

Convivência entre dois modelos de autenticação

Ao final da implementação, o sistema passa a suportar dois modelos:

- JWT para usuários humanos
- Service Token para integrações

Essa coexistência é muito comum em sistemas reais.

Ela permite atender diferentes tipos de cliente sem duplicar a aplicação.

Preparando para produção

Embora a implementação seja funcional, alguns pontos precisam ser considerados para produção:

- persistência dos tokens
- mecanismo de revogação
- auditoria de uso
- rotação periódica de chaves
- proteção do super secret

Esses elementos são essenciais para garantir segurança em larga escala.

Preparação para o próximo passo

Com Service Tokens implementados, eu deixo o sistema pronto para um cenário ainda mais realista.

Agora, já consigo:

- autenticar usuários humanos
- autenticar integrações
- controlar permissões com RBAC
- integrar com MCP de forma natural

O próximo passo natural é controlar o uso da API, evitando abuso e garantindo estabilidade.

NA PRÁTICA

Abra o(s) diretório(s) da Unidade 6 usando os links do GitHub exibidos no início desta aula. Localize os artefatos citados no conteúdo, reproduza o fluxo apresentado e observe o impacto em testes, estado, tracing, custo ou comportamento, conforme o tema da aula.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Por que Service Tokens existem” no fluxo apresentado?
- Que decisão de engenharia está associada a “Diferença entre JWT e Service Token” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório, nos testes ou no tracing para confirmar que o comportamento descrito na aula está funcionando?

RESUMO DA AULA

- Por que Service Tokens existem.
- Diferença entre JWT e Service Token.
- O custo da simplicidade.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 4 — Controlando Acesso por Tokens com Rate Limiting

Unidade 6 • Segurança e Governança em MCPs

OBJETIVOS DE APRENDIZAGEM

- Compreender o princípio de confiança zero.
- Relacionar por que rate limiting é essencial com rate limiting como camada de proteção.
- Aplicar as decisões apresentadas em configurando limites de uso e verificar o comportamento descrito na aula.

GITHUB DA UNIDADE

[07-api-security-auth-rate-limiting-template](#)

[07-api-security-auth-rate-limiting-z](#)

CONCEITOS-CHAVE

- O princípio de confiança zero
- Por que rate limiting é essencial
- Rate limiting como camada de proteção
- Configurando limites de uso
- Definindo a janela de tempo
- Identificando o cliente corretamente

Conteúdo da aula

Até este ponto, eu já implementei três camadas fundamentais de proteção na aplicação: autenticação com JWT, autorização com RBAC e autenticação programática com Service Tokens. Agora eu avanço para a terceira grande etapa quando pensamos em APIs públicas e consumo por MCPs: controle de uso.

Aqui, o objetivo deixa de ser apenas identificar quem pode acessar e passa a ser também limitar como esse acesso acontece.

O princípio de confiança zero

Quando eu trabalho com APIs expostas publicamente, eu parto sempre do princípio de confiança zero.

Isso significa que eu não posso assumir que o cliente vai se comportar bem. Mesmo que ele seja legítimo, podem acontecer situações como:

- loops acidentais
- erros de implementação
- uso indevido
- ataques automatizados

Por isso, além de autenticar e autorizar, eu preciso controlar o volume de requisições.

Por que rate limiting é essencial

O rate limiting resolve um problema muito prático.

Imagine um cenário onde um cliente normalmente faz:

- 10 requisições por minuto

E, de repente, começa a fazer:

- 1000 requisições por minuto

Isso pode indicar:

- bug na aplicação cliente
- uso abusivo
- tentativa de ataque

Além disso, em muitos casos, a API consome serviços pagos por requisição ou por token, como integrações com modelos de linguagem.

Sem controle, isso pode gerar custos inesperados e até indisponibilidade do sistema.

Rate limiting como camada de proteção

O rate limiting funciona como uma barreira adicional.

Mesmo que o usuário esteja autenticado e autorizado, ele ainda precisa respeitar limites de uso.

Essa é uma prática padrão em qualquer API pública madura.

Configurando limites de uso

Para implementar essa funcionalidade, eu começo definindo uma configuração central.

Nessa configuração, eu estabeleço:

- quantidade máxima de requisições
- janela de tempo

Por exemplo:

- 90 requisições por minuto

Esse tipo de configuração é um equilíbrio entre:

- permitir uso normal
- evitar abuso

A escolha desses valores depende do perfil da aplicação.

Definindo a janela de tempo

A janela de tempo geralmente é definida em minutos.

Isso porque o uso pode variar naturalmente:

- picos momentâneos
- períodos de baixa utilização

Se eu limitar por segundo de forma muito rígida, posso prejudicar a experiência do usuário.

Por isso, o uso de janela por minuto é mais comum e mais tolerante a variações.

Identificando o cliente corretamente

Um dos pontos mais importantes do rate limiting é definir quem é o cliente.

No meu caso, eu utilizo a própria credencial de autenticação como identificador.

Isso significa que o limite é aplicado por:

- JWT
- Service Token

Se o token não estiver presente, eu utilizo o IP como fallback.

Esse comportamento cobre dois cenários importantes:

Integração com o fluxo de autenticação

A lógica de identificação reaproveita o mesmo padrão utilizado na autenticação.

Eu extraio o token do header de autorização, removo o prefixo padrão e utilizo esse valor como chave de identificação.

Se não houver token, uso o IP da requisição.

Essa abordagem garante consistência entre autenticação e rate limiting.

Uso de plugin para simplificar implementação

Para implementar o rate limiting, eu utilizo um plugin específico do framework.

Assim como fiz com JWT, eu aproveito uma solução já consolidada, que resolve:

- controle de contagem
- janela de tempo
- bloqueio automático
- resposta padronizada

Isso evita reinventar lógica complexa e reduz risco de erro.

Registro do plugin na aplicação

A integração é simples.

Eu registro o plugin no servidor e passo as configurações definidas anteriormente.

Um detalhe importante aqui é garantir que o plugin seja carregado corretamente antes do uso, respeitando o ciclo de inicialização da aplicação.

Isso evita problemas onde o rate limiting não é aplicado corretamente.

Testando comportamento com limite baixo

Para validar o funcionamento, eu utilizo um limite extremamente baixo durante os testes.

Por exemplo:

- 1 requisição por minuto

Com isso, consigo facilmente reproduzir o comportamento esperado:

- primeira requisição funciona
- segunda requisição falha

Essa estratégia facilita a validação.

Observando o erro de limite excedido

Quando o limite é atingido, a API retorna um erro específico.

O código de resposta é:

- 429 (Too Many Requests)

E a mensagem indica que o limite foi excedido e informa quando o cliente pode tentar novamente.

Esse padrão é importante porque permite que o cliente:

- entenda o erro
- implemente retry adequado
- evite comportamento agressivo

Voltando à configuração real

Depois de validar, eu restauro o valor real do limite.

No projeto, utilizo algo como:

- 90 requisições por minuto

Esse valor é mais adequado para uso normal.

Validando com testes automatizados

Assim como nas outras etapas, eu não dependo apenas de validação manual.

Eu utilizo testes automatizados para garantir que:

- requisições dentro do limite funcionam
- requisição adicional falha com 429
- o comportamento é consistente

Esses testes são fundamentais para garantir que futuras alterações não quebrem essa proteção.

Integração com Service Tokens

Um ponto importante é que o rate limiting funciona perfeitamente com Service Tokens.

Isso significa que cada token possui seu próprio limite.

Na prática:

- dois clientes diferentes não competem entre si
- cada integração tem seu próprio controle de uso

Isso é essencial em ambientes com múltiplos consumidores.

Integração com JWT

Da mesma forma, usuários autenticados via JWT também são controlados individualmente.

Isso mantém consistência no comportamento da API.

Protegendo rotas públicas

Outro benefício importante é a proteção de rotas públicas.

Mesmo sem autenticação, o uso por IP garante que:

- ataques simples sejam mitigados
- abuso em endpoints abertos seja limitado

Isso complementa a estratégia de segurança.

Impacto no uso com MCP

Quando penso no uso com MCP, o rate limiting ganha ainda mais importância.

MCPs podem:

- automatizar chamadas
- encadear operações
- executar tarefas em sequência

Sem controle, isso pode gerar cargas muito altas.

Com rate limiting:

- evito sobrecarga
- controlo custos

- mantenho estabilidade

Consolidando as camadas de proteção

Neste ponto, a aplicação possui múltiplas camadas de defesa:

- autenticação (JWT e Service Token)
- autorização (RBAC)
- controle de uso (rate limiting)

Cada uma resolve um problema diferente, e juntas formam uma proteção robusta.

Preparação para produção

Para ambientes reais, ainda posso evoluir essa implementação com:

- limites diferentes por tipo de cliente
- planos de uso (free, premium)
- monitoramento e métricas
- bloqueio automático por comportamento suspeito

Mas a base já está sólida.

NA PRÁTICA

Abra o(s) diretório(s) da Unidade 6 usando os links do GitHub exibidos no início desta aula. Localize os artefatos citados no conteúdo, reproduza o fluxo apresentado e observe o impacto em testes, estado, tracing, custo ou comportamento, conforme o tema da aula.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “O princípio de confiança zero” no fluxo apresentado?
- Que decisão de engenharia está associada a “Por que rate limiting é essencial” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório, nos testes ou no tracing para confirmar que o comportamento descrito na aula está funcionando?

RESUMO DA AULA

- O princípio de confiança zero.
- Por que rate limiting é essencial.
- Rate limiting como camada de proteção.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 5 — Usando Service Tokens em Nosso Servidor MCP + Rate Limiting

Unidade 6 • Segurança e Governança em MCPs

OBJETIVOS DE APRENDIZAGEM

- Compreender atualizando o MCP para um cenário real.
- Relacionar uso obrigatório de Service Token com inicialização controlada do servidor.
- Aplicar as decisões apresentadas em integração com variáveis de ambiente e verificar o comportamento descrito na aula.

GITHUB DA UNIDADE

[07-api-security-auth-rate-limiting-template](#)
[07-api-security-auth-rate-limiting-z](#)

CONCEITOS-CHAVE

- Atualizando o MCP para um cenário real
- Uso obrigatório de Service Token
- Inicialização controlada do servidor
- Integração com variáveis de ambiente
- Atualizando a camada de infraestrutura
- Tratamento de erros na origem



Diagrama de referência — fluxo descrito na aula.

Conteúdo da aula

Depois de implementar autenticação com JWT, autorização com RBAC, Service Tokens e rate limiting na API, eu chego a um dos momentos mais importantes do módulo: integrar tudo isso diretamente no nosso servidor MCP.

Aqui, eu deixo de tratar essas funcionalidades de forma isolada e passo a consolidar um cenário real de produção, onde o MCP precisa respeitar autenticação, autorização e limites de uso ao se comunicar com a API.

Atualizando o MCP para um cenário real

Até então, o nosso MCP estava consumindo a API de forma simples, sem considerar:

- autenticação robusta
- controle de permissões
- limite de requisições

Agora, isso muda completamente.

O MCP passa a ser um cliente real da API, sujeito às mesmas regras que qualquer outro consumidor.

Uso obrigatório de Service Token

A primeira mudança importante é que o MCP passa a exigir um Service Token para funcionar.

Esse token é fornecido via variável de ambiente. Ou seja:

- se o token não estiver configurado
- o servidor MCP simplesmente não inicia

Essa decisão é intencional.

Ela garante que ninguém consiga usar o MCP sem estar autenticado corretamente.

Inicialização controlada do servidor

Durante a inicialização, o MCP verifica:

- se existe um Service Token válido
- se ele foi configurado corretamente

Caso contrário, o processo é interrompido.

Isso é uma prática muito importante em aplicações reais. É melhor falhar na inicialização do que permitir execução em estado inválido.

Integração com variáveis de ambiente

O uso de variáveis de ambiente segue um padrão já conhecido.

O token é configurado externamente e carregado no momento da execução. Isso evita:

- exposição no código-fonte
- necessidade de hardcode
- vazamento acidental

Esse modelo é exatamente o mesmo utilizado por outros MCPs e serviços modernos.

Atualizando a camada de infraestrutura

Na camada de comunicação com a API, eu faço uma adaptação importante.

Agora, toda requisição enviada pelo MCP inclui:

- o header de autorização
- contendo o Service Token

Isso garante que a API consiga:

- autenticar o cliente
- aplicar RBAC
- aplicar rate limiting

Sem essa informação, a requisição seria rejeitada.

Tratamento de erros na origem

Outro ponto importante é o tratamento de erros.

Antes, o MCP assumia que as chamadas funcionariam corretamente. Agora, ele passa a tratar explicitamente cenários como:

- token inválido
- acesso negado
- rate limit excedido

Esses erros são capturados e transformados em respostas estruturadas.

Padronização da resposta com erro

Para lidar com esses cenários, eu passo a incluir um campo específico nas respostas:

- indicador de erro

Isso permite que o cliente, inclusive a IA, entenda claramente quando algo deu errado.

Além disso, a resposta continua sendo estruturada, o que facilita o processamento posterior.

Ajustes nas tools

As tools do MCP também são atualizadas.

Agora, além do conteúdo normal, elas podem retornar:

- respostas de sucesso
- respostas de erro estruturadas

Essa mudança é fundamental para tornar o MCP resiliente.

Testando integração com a API atualizada

Com tudo configurado, eu executo os testes novamente.

Agora, os testes do MCP:

- utilizam Service Token
- respeitam rate limiting
- validam cenários de erro

Isso garante que o MCP está totalmente alinhado com a nova versão da API.

Subindo ambiente completo

Para validar o cenário completo, eu executo:

- subida do banco de dados
- build da aplicação
- execução via container

Com isso, o MCP passa a se comunicar com uma API real, com todas as regras aplicadas.

Esse é um passo importante para sair do ambiente local isolado.

Testes específicos de erro

Nos testes do MCP, eu adiciono cenários específicos para:

- token inválido
- ausência de token
- rate limit excedido

Esses testes garantem que o MCP não apenas funciona em cenários ideais, mas também responde corretamente em situações de falha.

Integração com o VSCode

Depois de validar via testes, eu levo o MCP para o VSCode.

Aqui, a integração acontece de forma muito natural:

- o MCP é configurado com o Service Token
- o editor passa a utilizá-lo automaticamente

Esse é exatamente o fluxo esperado em produção.

Testando sem Service Token

Um teste importante é iniciar o MCP sem o token configurado.

O resultado é imediato:

- o servidor não sobe
- uma mensagem de erro é exibida

Isso confirma que a proteção está funcionando.

Testando com permissões diferentes

Depois disso, eu testo com diferentes perfis.

Com um token de member:

- o MCP funciona
- mas operações de escrita são bloqueadas

Com um token de admin:

- todas as operações são permitidas

Isso demonstra que o RBAC continua funcionando corretamente dentro do MCP.

Uso via linguagem natural

Ao testar no editor, eu utilizo comandos em linguagem natural.

Por exemplo:

- criar cliente
- remover cliente

Mesmo sem especificar a tool, o sistema consegue:

- interpretar a intenção
- escolher a tool correta
- executar a operação

E tudo isso respeitando:

- autenticação
- autorização
- rate limiting

Observando o rate limiting na prática

Outro comportamento interessante é o rate limiting.

Quando o limite é atingido:

- o MCP recebe erro da API
- transforma em resposta estruturada
- retorna para o cliente

Isso garante que o sistema não falha de forma silenciosa.

Possível bypass e considerações

Um ponto importante observado é que, se eu utilizar múltiplos Service Tokens, posso contornar parcialmente o rate limiting.

Isso acontece porque o limite é aplicado por token.

Por isso, em ambientes reais, é comum combinar:

- identificação por token
- identificação por IP

Essa combinação evita abuso mais sofisticado.

Consolidando um MCP de produção

Neste ponto, o MCP possui:

- autenticação obrigatória
- controle de permissões
- tratamento de erros
- controle de uso
- integração com ambiente real

Isso transforma o projeto em algo muito próximo de produção.

O que muda na prática

Comparado ao início do módulo, a diferença é significativa.

Antes:

- chamadas diretas
- sem controle
- sem segurança

Agora:

- autenticação obrigatória
- autorização consistente
- limites de uso definidos
- comportamento previsível

Essa evolução é exatamente o que diferencia um protótipo de uma solução real.

Próximo passo natural

Com tudo isso implementado, o MCP já está pronto para ser utilizado localmente.

O próximo passo é torná-lo acessível para outras pessoas, ou seja:

- publicação
- distribuição
- uso externo

Esse é o último passo para fechar o ciclo completo.

NA PRÁTICA

Abra o(s) diretório(s) da Unidade 6 usando os links do GitHub exibidos no início desta aula. Localize os artefatos citados no conteúdo, reproduza o fluxo apresentado e observe o impacto em testes, estado, tracing, custo ou comportamento, conforme o tema da aula.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Atualizando o MCP para um cenário real” no fluxo apresentado?
- Que decisão de engenharia está associada a “Uso obrigatório de Service Token” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório, nos testes ou no tracing para confirmar que o comportamento descrito na aula está funcionando?

RESUMO DA AULA

- Atualizando o MCP para um cenário real.
- Uso obrigatório de Service Token.
- Inicialização controlada do servidor.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Revisão da Unidade 6

Use esta página como revisão rápida antes de avançar. Você deve conseguir relacionar as aulas da unidade em um único fluxo de engenharia.



Fluxo visual da Unidade 6.

CHECKLIST DA UNIDADE

- [Revisar a Aula 1: Template Inicial e Arquitetura + Boas Práticas de Organização de Código e Estrutura de Projeto](#)
- [Revisar a Aula 2: Implementando Autenticação com RBAC e Autenticação com JWT em Web API](#)
- [Revisar a Aula 3: Autenticação e Autorização com Service Tokens em Web APIs](#)
- [Revisar a Aula 4: Controlando Acesso por Tokens com Rate Limiting](#)
- [Revisar a Aula 5: Usando Service Tokens em Nosso Servidor MCP + Rate Limiting](#)

GITHUB DA UNIDADE

[Repositório oficial da disciplina](#)

[07-api-security-auth-rate-limiting-template](#)

[07-api-security-auth-rate-limiting-z](#)

Unidade 7

MCP em Produção: Publicação e Transports

Publicação de servidores MCP em registries público e privado, execução como pacote e comparação de transports como STDIO, HTTP, streaming e Server-Sent Events.



Fluxo visual da Unidade 7 — reorganização dos elementos descritos nas aulas.

GITHUB DA UNIDADE

[Repositório oficial da disciplina](#)

[08-publishing-mcps-private-npm](#)

Aula	Tema
1	Publicando Servidores MCP em NPM Registry (público) e Verdaccio (privado)
2	Indo Além em Servidores MCP: Diferentes Transports e Ideias para seu Próximo Servidor

Aula 1 — Publicando Servidores MCP em NPM Registry (público) e Verdaccio (privado)

Unidade 7 • MCP em Produção: Publicação e Transports

OBJETIVOS DE APRENDIZAGEM

- Compreender por que publicar um MCP.
- Relacionar dois cenários de publicação com quando usar registry privado.
- Aplicar as decisões apresentadas em subindo um registry privado com Verdaccio e verificar o comportamento descrito na aula.

GITHUB DA UNIDADE

[08-publishing-mcps-private-npm](#)

CONCEITOS-CHAVE

- Por que publicar um MCP
- Dois cenários de publicação
- Quando usar registry privado
- Subindo um registry privado com Verdaccio
- Criando usuário no registry
- Versionamento do pacote



Diagrama de referência — fluxo descrito na aula.

Conteúdo da aula

Depois de construir um servidor MCP completo, com autenticação, autorização, Service Tokens e rate limiting, eu chego ao último passo natural do projeto: distribuição. Nesta etapa, eu mostro como transformar o MCP em um pacote reutilizável, publicando tanto em um registry privado quanto no NPM público.

Esse é o momento em que o projeto deixa de ser apenas local e passa a poder ser consumido por outras pessoas, equipes ou sistemas.

Por que publicar um MCP

Enquanto o MCP está rodando localmente, ele só pode ser utilizado naquele ambiente específico.

Ao publicar o servidor como um pacote:

- eu permito reutilização
- facilito distribuição
- padronizo a instalação
- torno o MCP acessível para outros times

Esse modelo é exatamente o que vemos em ferramentas modernas que distribuem integrações prontas.

Dois cenários de publicação

Nesta aula, eu trabalho com dois cenários principais:

- publicação privada, usando Verdaccio
- publicação pública, usando NPM Registry

Cada um atende a uma necessidade diferente.

Quando usar registry privado

O registry privado é ideal quando:

- o código não pode ser exposto publicamente
- o MCP contém regras de negócio sensíveis
- o uso é restrito à empresa

Nesse cenário, o Verdaccio entra como solução leve e eficiente.

Subindo um registry privado com Verdaccio

Para começar, eu utilizo um ambiente já preparado com Docker.

O Verdaccio roda como um serviço local e expõe uma interface web onde eu posso:

- criar usuários
- publicar pacotes
- visualizar versões

Esse ambiente simula um registry completo, mas sob meu controle.

Criando usuário no registry

Antes de publicar qualquer pacote, eu preciso me autenticar.

Então eu:

- acesso a interface web
- crio um usuário
- faço login via terminal

Esse passo é obrigatório, pois o registry precisa saber quem está publicando.

Versionamento do pacote

Antes de publicar, eu preciso versionar o pacote corretamente.

Aqui entra o versionamento semântico:

Cada publicação precisa ter uma versão única.

Isso é fundamental para evitar conflitos e garantir rastreabilidade.

Publicando no Verdaccio

Com tudo configurado, eu executo o processo de publicação.

O fluxo é simples:

- atualizar versão
- executar publish
- validar no registry

Depois disso, o pacote já aparece listado, com:

- versão
- descrição
- README
- instruções de uso

Ou seja, ele se comporta como qualquer pacote NPM.

Preparando o MCP como executável

Para que o MCP possa ser executado via linha de comando, eu preciso configurá-lo corretamente.

Isso envolve:

- definir um comando executável
- apontar para o arquivo de entrada
- garantir que o runtime saiba como executar

Esse passo é essencial para permitir uso com ferramentas como npx.

Tornando o arquivo executável

O arquivo principal do MCP precisa ser tratado como executável.

Para isso, eu adiciono uma instrução que indica qual runtime deve executá-lo.

Isso permite que o sistema operacional trate o arquivo como um comando.

Lidando com TypeScript no contexto de pacote

Aqui surge um problema importante.

O TypeScript nativo do Node funciona bem em projetos locais, mas pode apresentar limitações quando o código está dentro de `node_modules`.

Para resolver isso, eu utilizo uma abordagem que permite executar TypeScript diretamente, sem necessidade de build prévio.

Isso garante compatibilidade com o ambiente de execução do pacote.

Atualizando e republicando

Sempre que faço uma alteração:

- incremento a versão
- publico novamente

Esse ciclo é repetido até que o pacote esteja estável.

Por isso, o uso do registry privado é tão importante nessa fase.

Consumindo o pacote publicado

Depois de publicado, o MCP pode ser utilizado diretamente.

Em vez de executar um arquivo local, eu passo a usar:

- execução via npx
- nome do pacote publicado
- argumentos necessários

Isso muda completamente a forma de consumo.

Configurando o MCP no ambiente

Para usar o pacote, eu atualizo a configuração do MCP no editor.

Agora, em vez de apontar para um arquivo local, eu:

- utilizo npx
- passo o nome do pacote
- informo o registry (no caso privado)

Isso faz com que o pacote seja baixado e executado automaticamente.

Validando funcionamento

Após a configuração, eu inicio o MCP.

Se tudo estiver correto:

- as tools aparecem
- os prompts são carregados
- o servidor responde normalmente

Isso confirma que o pacote está funcionando fora do ambiente original.

Publicação no NPM público

Depois de validar no ambiente privado, eu passo para o NPM público.

Esse passo exige:

- criação de conta
- autenticação
- ajuste do nome do pacote

Aqui, o pacote passa a ser acessível globalmente.

Diferença entre público e privado

A principal diferença é:

No NPM, qualquer pessoa pode instalar e utilizar o pacote.

Processo de publicação no NPM

O fluxo é muito parecido com o privado:

- login
- versionamento
- publish

A diferença é que o registry padrão já é o NPM.

Boas práticas de publicação

Uma prática importante que reforço é:

- testar no registry privado antes
- validar funcionamento completo
- só então publicar no NPM

Isso evita:

- versões quebradas em produção
- necessidade de correções frequentes
- impacto em usuários

Consumindo via NPM público

Depois de publicado, o uso se torna ainda mais simples.

Não é mais necessário informar registry.

Basta:

- usar npx
- informar o nome do pacote

O sistema resolve automaticamente.

Integração com o editor

Ao configurar o MCP com o pacote público:

- o editor baixa automaticamente
- executa o servidor
- expõe tools e prompts

Isso mostra o verdadeiro poder da distribuição.

Observando o comportamento completo

Com tudo funcionando, eu consigo:

- usar tools normalmente
- respeitar autenticação
- aplicar rate limiting
- executar fluxos completos

Tudo a partir de um pacote distribuído.

O que isso representa

Neste ponto, o MCP:

- não depende mais do ambiente local
- pode ser compartilhado
- pode ser versionado
- pode ser reutilizado

Isso transforma completamente o valor do projeto.

NA PRÁTICA

Abra o(s) diretório(s) da Unidade 7 usando os links do GitHub exibidos no início desta aula. Localize os artefatos citados no conteúdo, reproduza o fluxo apresentado e observe o impacto em testes, estado, tracing, custo ou comportamento, conforme o tema da aula.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “Por que publicar um MCP” no fluxo apresentado?
- Que decisão de engenharia está associada a “Dois cenários de publicação” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório, nos testes ou no tracing para confirmar que o comportamento descrito na aula está funcionando?

RESUMO DA AULA

- Por que publicar um MCP.
- Dois cenários de publicação.
- Quando usar registry privado.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Aula 2 — Indo Além em Servidores MCP: Diferentes Transports e Ideias para seu Próximo Servidor

Unidade 7 • MCP em Produção: Publicação e Transports

OBJETIVOS DE APRENDIZAGEM

- Compreender o transporte que utilizamos até aqui.
- Relacionar por que STDIO é o padrão com outras formas de transporte.
- Aplicar as decisões apresentadas em MCP via HTTP e verificar o comportamento descrito na aula.

GITHUB DA UNIDADE

[08-publishing-mcps-private-npm](#)

CONCEITOS-CHAVE

- O transporte que utilizamos até aqui
- Por que STDIO é o padrão
- Outras formas de transporte
- MCP via HTTP
- Streaming via HTTP
- Server-Sent Events (SSE)

Quadro comparativo — transports discutidos na aula

Transporte	Uso destacado no conteúdo
STDIO	Execução local, simplicidade, editores, automação e pipelines simples.
HTTP	Serviços centralizados, múltiplos clientes e necessidade de escalabilidade.
Streaming / SSE	Cenários em tempo real ou processamento contínuo.

Conteúdo da aula

Antes de finalizar o módulo, eu faço uma pausa importante para ampliar a visão sobre o que construímos até aqui. Ao longo das aulas, eu foquei no uso de MCP via STDIO, principalmente porque esse é o modelo mais comum e mais simples de trabalhar no dia a dia.

Mas isso não significa que seja a única forma de utilizar o protocolo. Muito pelo contrário. O MCP foi desenhado para ser flexível, e entender essas possibilidades abre um leque enorme de aplicações.

O transporte que utilizamos até aqui

Durante todo o projeto, eu utilizei o transporte via STDIO.

Isso significa que o MCP:

- é executado localmente
- roda como um processo no ambiente do cliente
- se comunica por entrada e saída padrão
- normalmente é distribuído como pacote NPM

Esse modelo é extremamente eficiente porque:

- não exige infraestrutura adicional
- é simples de instalar
- funciona muito bem com editores e ferramentas locais

Na prática, é por isso que utilizamos comandos como execução via npx.

Por que STDIO é o padrão

O STDIO se tornou o padrão porque resolve a maioria dos problemas com baixo custo de implementação.

Ele é ideal para:

- ferramentas locais
- automação no ambiente do desenvolvedor
- integração com editores
- uso em pipelines simples

Além disso, reduz bastante a complexidade de segurança, já que não expõe serviços diretamente na rede.

Outras formas de transporte

Apesar disso, existem outros modelos de transporte que podem ser utilizados dependendo do cenário.

O MCP permite diferentes estratégias de comunicação, e cada uma delas atende a necessidades específicas.

MCP via HTTP

Uma das alternativas é utilizar HTTP como meio de transporte.

Nesse modelo, o MCP:

- é exposto como um serviço web
- recebe requisições via rede
- responde de forma síncrona ou assíncrona

Esse tipo de abordagem é interessante quando:

- o serviço precisa ser centralizado
- múltiplos clientes acessam o mesmo MCP
- há necessidade de escalabilidade

Também abre espaço para cenários mais distribuídos.

Streaming via HTTP

Outra possibilidade é trabalhar com streaming.

Nesse caso, o servidor não responde apenas com uma única resposta. Ele pode enviar dados continuamente, conforme são processados.

Isso é especialmente útil para:

- processamento de vídeo
- análise de áudio
- geração incremental de conteúdo
- pipelines de dados

Esse modelo muda a forma de pensar o MCP, pois ele deixa de ser apenas request-response e passa a lidar com fluxo contínuo.

Server-Sent Events (SSE)

Uma abordagem mais avançada é o uso de Server-Sent Events.

Nesse modelo:

- o servidor mantém uma conexão aberta
- envia eventos continuamente
- o cliente recebe dados em tempo real

Isso permite construir integrações mais reativas.

Por exemplo:

- notificações em tempo real
- atualizações contínuas de estado
- monitoramento de eventos

Embora mais complexo, esse modelo é muito poderoso.

Comparando os modelos

Cada tipo de transporte tem seu espaço.

O STDIO é ideal para execução local e simplicidade.

O HTTP é ideal para serviços centralizados.

O streaming e SSE são ideais para cenários em tempo real ou de processamento contínuo.

A escolha depende diretamente do problema que você quer resolver.

Exemplo prático com ferramentas reais

Um exemplo interessante é o uso de MCP em ferramentas como dashboards e sistemas de monitoramento.

Essas ferramentas podem:

- consumir dados continuamente
- reagir a eventos
- integrar múltiplas fontes

Nesse contexto, o uso de SSE ou HTTP faz muito mais sentido do que STDIO.

MCP rodando em containers

Outra possibilidade é executar MCPs dentro de containers.

Nesse modelo, o servidor:

- roda isolado
- pode ser distribuído via Docker
- facilita padronização de ambiente

Isso é comum em empresas que já trabalham com infraestrutura mais robusta.

Quando usar Docker

O uso de Docker faz sentido quando:

- o ambiente precisa ser padronizado
- há dependências complexas
- o MCP é parte de um sistema maior

Por outro lado, para ferramentas simples, pode ser um excesso de complexidade.

Simplicidade versus robustez

Aqui entra uma decisão importante de arquitetura.

Se o objetivo é:

- facilitar adoção
- reduzir barreira de entrada
- atingir desenvolvedores rapidamente

O modelo via NPM e STDIO costuma ser melhor.

Se o objetivo é:

- escalar
- centralizar
- controlar infraestrutura

Modelos baseados em HTTP ou containers fazem mais sentido.

Explorando o ecossistema

Uma forma prática de evoluir no uso de MCP é explorar o ecossistema.

Existem listas e repositórios que reúnem exemplos de servidores MCP para diferentes usos.

Esses exemplos mostram integrações com:

- sistemas operacionais
- ferramentas de produtividade
- serviços externos
- plataformas de comunicação

Esse tipo de exploração ajuda a entender o potencial real do protocolo.

MCP como camada de automação

Ao olhar para esses exemplos, fica claro que MCP não é apenas uma ferramenta técnica.

Ele pode ser usado como uma camada de automação.

Por exemplo:

- ler eventos externos
- disparar ações automáticas
- integrar múltiplos sistemas
- orquestrar fluxos de trabalho

Isso amplia muito o escopo do que pode ser construído.

Integração com ferramentas do dia a dia

No contexto prático, MCP pode ser integrado com ferramentas como:

- sistemas de gestão de projetos
- plataformas de comunicação
- serviços de e-mail
- agendas e calendários

Com isso, é possível construir fluxos como:

- receber uma notificação
- processar com IA

- gerar resumo
- atualizar sistema
- enviar mensagem

Tudo de forma automatizada.

Pensando no próximo projeto

Neste ponto, eu incentivo a pensar além do exemplo construído.

Algumas ideias que podem ser exploradas:

- MCP para automação de tarefas pessoais
- integração com ferramentas de trabalho
- processamento de dados em tempo real
- orquestração de workflows
- integração com APIs externas

O importante é perceber que o MCP não limita o tipo de aplicação.

O papel da criatividade

Com as ferramentas disponíveis hoje, especialmente com IA integrada, o limite passa a ser muito mais criativo do que técnico.

Qualquer evento pode virar uma ação.

Qualquer integração pode virar um fluxo automatizado.

E o MCP funciona como a ponte entre esses mundos.

Consolidando o aprendizado

Ao longo do módulo, eu construí:

- um servidor MCP completo
- integração com API legada
- autenticação e autorização
- controle de uso
- distribuição via NPM

Agora, com essa visão ampliada, fica claro que isso é apenas o começo.

Conclusão

Nesta aula, eu amplio a visão sobre o uso de MCP, mostrando que existem diferentes formas de transporte e que o protocolo pode ser aplicado em uma variedade muito maior de cenários do que apenas execução local via STDIO.

Com isso, eu encerro o módulo deixando um direcionamento claro: o MCP é uma base poderosa, mas o valor real está em como você decide utilizá-lo.

A partir daqui, o próximo passo não é mais aprender a tecnologia, mas explorar possibilidades, integrar com o seu contexto e construir soluções que realmente façam sentido no seu dia a dia.

NA PRÁTICA

Abra o(s) diretório(s) da Unidade 7 usando os links do GitHub exibidos no início desta aula. Localize os artefatos citados no conteúdo, reproduza o fluxo apresentado e observe o impacto em testes, estado, tracing, custo ou comportamento, conforme o tema da aula.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “O transporte que utilizamos até aqui” no fluxo apresentado?
- Que decisão de engenharia está associada a “Por que STDIO é o padrão” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório, nos testes ou no tracing para confirmar que o comportamento descrito na aula está funcionando?

RESUMO DA AULA

- O transporte que utilizamos até aqui.
- Por que STDIO é o padrão.
- Outras formas de transporte.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Revisão da Unidade 7

Use esta página como revisão rápida antes de avançar. Você deve conseguir relacionar as aulas da unidade em um único fluxo de engenharia.



Fluxo visual da Unidade 7.

CHECKLIST DA UNIDADE

- [Revisar a Aula 1: Publicando Servidores MCP em NPM Registry \(público\) e Verdaccio \(privado\)](#)
- [Revisar a Aula 2: Indo Além em Servidores MCP: Diferentes Transports e Ideias para seu Próximo Servidor](#)

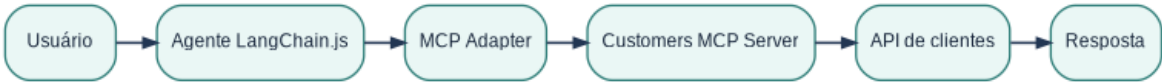
GITHUB DA UNIDADE

[Repositório oficial da disciplina](#)
[08-publishing-mcps-private-npm](#)

Unidade 8

Consumindo o MCP com LangChain.js

Criação de um agente LangChain.js que consome o Customers MCP Server, reutiliza Service Token e disponibiliza operações de clientes por meio de tools descobertas no servidor.



Fluxo visual da Unidade 8 — reorganização dos elementos descritos nas aulas.

GITHUB DA UNIDADE

[Repositório oficial da disciplina](#)

[09-using-mcp-with-langchain](#)

Aula	Tema
1	Criando um Agente para Gerenciar Operações de Clientes com Nosso Customers MCP Server e LangChain.js

Aula 1 — Criando um Agente para Gerenciar Operações de Clientes com Nosso Customers MCP Server e LangChain.js

Unidade 8 • Consumindo o MCP com LangChain.js

OBJETIVOS DE APRENDIZAGEM

- Compreender o objetivo da aplicação.
- Relacionar reaproveitando o que foi construído com estrutura geral do projeto.
- Aplicar as decisões apresentadas no papel do prompt e verificar o comportamento descrito na aula.

GITHUB DA UNIDADE

[09-using-mcp-with-langchain](#)

CONCEITOS-CHAVE

- O objetivo da aplicação
- Reaproveitando o que foi construído
- Estrutura geral do projeto
- O papel do prompt
- Usando múltiplos MCPs
- Consumo do Customers MCP via NPM



Diagrama de referência — fluxo descrito na aula.

Conteúdo da aula

Nesta aula, eu fecho o módulo mostrando o que, para mim, é uma das aplicações mais práticas de tudo o que construímos até aqui: usar o nosso servidor MCP como uma peça real dentro de uma aplicação com LangChain.js.

Até aqui, eu já tinha mostrado como criar o servidor, expor tools, resources e prompts, proteger a API com autenticação, autorização e rate limiting, publicar o pacote e até consumir esse MCP no editor. Agora, eu junto essas partes em um cenário que representa muito bem o uso no mundo real: um agente que recebe uma intenção em linguagem natural e decide, com autonomia, como usar o nosso Customers MCP Server para executar operações sobre clientes.

A grande ideia aqui é simples e poderosa ao mesmo tempo. Eu não quero apenas publicar um MCP e deixá-lo disponível por aí. Eu quero que ele vire uma capacidade real do meu sistema. Quero que uma IA consiga enxergá-lo como mais um conjunto de ferramentas disponíveis, do mesmo jeito que já fizemos com File System, banco de dados e outros servidores MCP ao longo do curso.

O objetivo da aplicação

Para esta etapa, eu já deixo um projeto completo preparado, justamente porque a estrutura geral não muda tanto do que já vimos anteriormente.

O foco desta aula não está em repetir toda a construção da arquitetura do zero, mas em mostrar como o MCP publicado pode ser consumido em uma aplicação baseada em LangChain.js como se fosse qualquer outro servidor disponível por NPM. Esse é o ponto central.

Na prática, eu quero que o agente consiga:

- entender a solicitação do usuário
- decidir quando deve usar o MCP de clientes
- executar operações como criar, listar e remover clientes
- combinar isso com outras tools disponíveis
- devolver uma resposta final coerente

Ou seja, eu não estou mais apenas demonstrando um protocolo. Estou demonstrando autonomia aplicada.

Reaproveitando o que foi construído

Uma coisa importante nesta aula é perceber o valor do trabalho feito antes.

Eu reaproveito:

- o Customers MCP Server já publicado
- a autenticação via Service Token
- a estrutura de integração com LangChain.js
- a estratégia de tools via MCP
- a organização que já vínhamos usando nos projetos anteriores

Isso deixa muito claro que construir um MCP bem feito não é um exercício isolado. É uma fundação. Depois que essa fundação existe, ela pode ser conectada a outras aplicações de forma extremamente natural.

Estrutura geral do projeto

A aplicação segue um desenho bastante simples.

No grafo, eu tenho basicamente um fluxo enxuto, em que a mensagem do usuário é recebida e encaminhada para um nó responsável por resolver a solicitação usando as tools disponíveis.

Esse ponto é importante porque mostra uma característica muito forte desse modelo: eu não preciso mais escrever um fluxo imperativo enorme dizendo passo a passo o que a IA deve fazer em cada cenário.

Em vez disso, eu deixo o prompt orientar o comportamento geral e entrego ao agente um conjunto de capacidades. A partir daí, ele decide o que fazer.

O papel do prompt

O prompt aqui é intencionalmente genérico.

A lógica que eu defino é a seguinte:

- se a intenção do usuário estiver relacionada a clientes, o agente deve usar as tools de clientes
- se não estiver, ele pode recorrer às demais ferramentas disponíveis

Essa abordagem é muito interessante porque aproxima o projeto do tipo de sistema que eu realmente quero construir em produção. Eu não quero um agente que só saiba fazer uma coisa. Eu quero um agente que conviva com múltiplos conjuntos de capacidades e escolha o recurso adequado conforme a necessidade.

Isso é um passo importante de maturidade.

Usando múltiplos MCPs

Para deixar o exemplo mais rico, eu adiciono mais de um MCP no projeto.

No caso desta aula, eu uso:

- o File System, que já vinha aparecendo em projetos anteriores
- o nosso Customers MCP Server, que foi construído ao longo do módulo

Isso mostra, na prática, que um agente pode operar com múltiplos servidores MCP ao mesmo tempo. Ele não precisa ficar preso a uma única integração.

Esse ponto é especialmente relevante porque, no mundo real, dificilmente uma aplicação inteligente vai trabalhar com apenas um sistema. O mais comum é ela combinar diferentes fontes e diferentes capacidades.

Consumo do Customers MCP via NPM

Outro detalhe importante é que agora o MCP de clientes já não precisa mais estar local dentro da aplicação como código-fonte.

Eu o consumo como pacote publicado.

Esse é um momento muito forte do módulo porque deixa claro que o MCP virou um artefato reutilizável. Ele está sendo tratado como qualquer outro pacote do ecossistema.

Na configuração, eu aponto para o pacote publicado e forneço as variáveis de ambiente necessárias, especialmente o Service Token, que é obrigatório para que o MCP consiga autenticar suas chamadas contra a API protegida.

O papel do Service Token

Como a API está protegida, o agente não pode simplesmente sair fazendo chamadas sem credencial.

Por isso, o projeto continua reutilizando o modelo de Service Token que eu implementei antes. Toda vez que o agente for usar o MCP de clientes, esse token precisa estar disponível no ambiente.

Esse detalhe é essencial porque mostra o MCP operando em um cenário realista. Não estamos trabalhando com uma integração aberta e simplificada. Estamos trabalhando com um serviço que exige autenticação e respeita as regras de segurança definidas anteriormente.

Atualização automática do token no ambiente

Para facilitar a execução, eu já deixo preparado um fluxo para gerar o Service Token e atualizar automaticamente o arquivo de ambiente do projeto.

Isso melhora bastante a experiência durante o desenvolvimento, porque evita aquele processo repetitivo de copiar token manualmente toda vez que o ambiente é reiniciado ou que um novo token é emitido.

Mais uma vez, essa é uma pequena automação, mas que mostra uma preocupação importante com ergonomia de uso.

A primeira demonstração prática

Para mostrar o agente funcionando, eu preparo uma solicitação relativamente rica. Em vez de fazer uma pergunta simples, eu peço algo como:

- criar três clientes de teste
- salvar esses clientes em um arquivo
- depois listar os clientes disponíveis

Esse tipo de prompt é interessante porque força o agente a encadear diferentes ações.

Ele precisa:

- entender que deve usar o MCP de clientes para criar registros
- entender que deve usar o File System para salvar o resultado
- voltar ao MCP para listar o estado final

Esse encadeamento é exatamente o tipo de comportamento que eu quero evidenciar.

Orquestração real de chamadas

Ao executar esse fluxo, o agente mostra claramente sua autonomia.

Ele cria os clientes um por um, faz as chamadas necessárias, persiste os dados e depois retorna a lista consolidada com o resultado final.

O mais interessante aqui é perceber que eu não precisei escrever um controlador detalhado dizendo:

- primeiro faça isso
- depois faça aquilo
- em seguida execute tal chamada

Eu apenas forneci o problema, o contexto e as capacidades disponíveis. O agente fez o restante.

Persistência com File System

A presença do File System na demonstração não é um detalhe decorativo.

Ela reforça a ideia de que o agente pode combinar o MCP de clientes com outras ferramentas do sistema. No exemplo, ele cria os dados e também grava essas informações em disco.

Isso é extremamente útil porque torna o fluxo mais próximo de um caso real de automação. Não é só manipular dados em memória. É executar uma operação completa, com resultado persistido.

Listando clientes existentes

Depois da criação, eu também faço perguntas mais simples, como pedir para o agente mostrar os clientes disponíveis.

Nesse cenário, ele utiliza diretamente o MCP de clientes para consultar os dados e devolver o resultado. Isso ajuda a validar que a integração está funcionando de forma consistente e que o agente reconhece corretamente quando a intenção do usuário está relacionada ao domínio de clientes.

Removendo clientes com linguagem natural

Um dos momentos mais interessantes da aula é quando eu peço algo como remover todos os clientes.

Esse teste é relevante porque ele mostra o agente utilizando as capacidades do MCP para percorrer a lista de clientes e executar as remoções necessárias. Ou seja, mais uma vez ele não está apenas respondendo. Ele está agindo.

Esse é o ponto em que o MCP realmente mostra seu valor dentro de uma arquitetura com agentes.

Limitações do modelo e papel da memória

Em um dos testes, eu também deixo explícita uma limitação importante do projeto naquele estado.

Como o fluxo não está usando memória de conversa consolidada, quando faço perguntas muito dependentes do contexto anterior, como pedir o identificador de um cliente recém-criado sem repetir claramente o contexto, o agente pode não responder como eu gostaria.

Esse ponto é importante porque mostra que o MCP resolve a parte de integração, mas a qualidade da experiência completa também depende de:

- gestão de histórico
- memória entre interações
- desenho adequado do grafo
- estratégia de persistência de contexto

Ou seja, MCP não substitui arquitetura de agente. Ele a complementa.

O que essa aula demonstra de forma mais profunda

Esta aula, na verdade, demonstra várias coisas ao mesmo tempo.

Primeiro, mostra que o MCP pode ser tratado como um pacote NPM e usado como dependência real de uma aplicação.

Segundo, mostra que um agente pode consumir múltiplos MCPs ao mesmo tempo.

Terceiro, mostra que uma IA pode operar com autonomia real sobre sistemas legados modernizados via MCP.

E, por fim, mostra que todo o esforço de modelagem, segurança, publicação e abstração vale a pena quando chegamos no momento de uso.

O ganho arquitetural

Do ponto de vista de arquitetura, o ganho é enorme.

Em vez de eu criar uma aplicação acoplada diretamente à API de clientes, com lógica espalhada e regras imperativas no meio do código, eu passo a ter uma camada MCP reutilizável, versionada, protegida e pronta para ser consumida por qualquer outra aplicação.

Isso significa que a inteligência do sistema não precisa conhecer os detalhes da API original. Ela conhece apenas as ações que o MCP decidiu expor.

Esse desacoplamento é um dos maiores ganhos de todo o módulo.

MCP como ponte de modernização

Ao longo do módulo inteiro, eu venho insistindo nessa ideia, e aqui ela fica ainda mais evidente: o MCP é uma excelente ponte para modernizar sistemas existentes.

Você pega uma API legada, que muitas vezes foi feita para consumo humano ou para integrações tradicionais, e a transforma em uma interface muito mais adequada para agentes e LLMs.

Não é uma substituição completa do legado. É uma camada de adaptação inteligente.

O fechamento do módulo

Esta aula também funciona como um fechamento natural do módulo.

Aqui eu reúno praticamente todos os conceitos que foram trabalhados:

- construção de tools
- resources e prompts
- autenticação e autorização
- service tokens
- rate limiting
- publicação em registry
- consumo em LangChain.js
- autonomia de agentes

É como se todas as peças finalmente se encaixassem em um cenário de uso real.

Conclusão

Nesta aula, eu demonstro como usar o nosso Customers MCP Server dentro de uma aplicação com LangChain.js para criar um agente capaz de gerenciar operações de clientes com autonomia real.

O ponto mais importante aqui é perceber que o MCP deixa de ser apenas um projeto técnico e passa a virar uma capacidade viva da aplicação. O agente entende a intenção do usuário, escolhe as tools corretas, encadeia ações e interage com sistemas protegidos como se estivesse operando sobre um conjunto natural de habilidades.

Esse é, para mim, o grande valor de tudo o que foi construído: não apenas publicar servidores MCP, mas transformá-los em blocos reutilizáveis para aplicações inteligentes, modernas e realmente úteis.

NA PRÁTICA

Abra o(s) diretório(s) da Unidade 8 usando os links do GitHub exibidos no início desta aula. Localize os artefatos citados no conteúdo, reproduza o fluxo apresentado e observe o impacto em testes, estado, tracing, custo ou comportamento, conforme o tema da aula.

CHECKPOINT DE APRENDIZAGEM

- Como você explicaria, com suas palavras, o papel de “O objetivo da aplicação” no fluxo apresentado?
- Que decisão de engenharia está associada a “Reaproveitando o que foi construído” e qual problema ela procura evitar?
- Que evidência você verificaria no repositório, nos testes ou no tracing para confirmar que o comportamento descrito na aula está funcionando?

RESUMO DA AULA

- O objetivo da aplicação.
- Reaproveitando o que foi construído.
- Estrutura geral do projeto.

[↑ Voltar ao mapa da disciplina](#) • [Abrir GitHub da unidade](#)

Revisão da Unidade 8

Use esta página como revisão rápida antes de avançar. Você deve conseguir relacionar as aulas da unidade em um único fluxo de engenharia.



Fluxo visual da Unidade 8.

CHECKLIST DA UNIDADE

- [Revisar a Aula 1: Criando um Agente para Gerenciar Operações de Clientes com Nosso Customers MCP Server e LangChain.js](#)

GITHUB DA UNIDADE

[Repositório oficial da disciplina](#)

[09-using-mcp-with-langchain](#)

Revisão final da disciplina

Ao concluir a disciplina, o percurso vai da evolução de tools e function calling para um protocolo completo, passa pela construção e adaptação de servidores MCP, segurança, publicação e termina no consumo do servidor por um agente LangChain.js. O mapa abaixo reorganiza somente elementos trabalhados nas transcrições.



Fluxo de revisão final da disciplina.

REVISÃO DE DOMÍNIO

- Explique a evolução de plugins e function calling até o MCP e a diferença entre uma lista de funções e um protocolo com tools, resources e prompts.
- Reconstrua o pipeline com parsing da entrada, tool de CSV, MongoDB e File System, indicando o que fica sob decisão do modelo e o que permanece determinístico.
- Compare instructions, agents e skills e explique por que skills são apresentadas como conhecimento modular carregado conforme a necessidade.
- Descreva como testes via MCP Client, tools, resources, prompts e Inspector estruturam a construção de um servidor MCP do zero.
- Explique por que transformar uma API legada em MCP não significa espelhar endpoints e como HTTP Client, service e tools de domínio se relacionam.
- Diferencie JWT, RBAC e Service Tokens e explique o papel do rate limiting na governança do acesso.
- Compare publicação pública e privada e relacione STDIO, HTTP, streaming e SSE aos cenários discutidos nas aulas.
- Mostre como o agente LangChain.js consome o Customers MCP Server e reutiliza as capacidades publicadas sem reimplementar a API de clientes.

Anexo A — GitHub da disciplina

Repositório oficial informado para esta disciplina:

<https://github.com/unipds-engenharia-de-ia-aplicada/engenharia-de-software-com-ia-aplicada/tree/main/modulo03-mcp-na-pratica>



Diretórios listados no repositório para este módulo:

- [01-multiple-mcp-tools-template](#)
- [01-multiple-mcp-tools-z](#)
- [02-google-trends-agent](#)
- [03-dev-instructions-agents/.github/agents](#)
- [04-skills](#)
- [05-mcps-do-zero-template](#)
- [05-mcps-do-zero-z](#)
- [06-your-legacy-api-as-mcp](#)
- [07-api-security-auth-rate-limiting-template](#)
- [07-api-security-auth-rate-limiting-z](#)
- [08-publishing-mcps-private-npm](#)
- [09-using-mcp-with-langchain](#)